



**ESCUELA POLITÉCNICA
SUPERIOR DE CÓRDOBA**
Universidad de Córdoba



TRABAJO FIN DE GRADO

*Grado en Ingeniería Informática
Mención en Ingeniería del Software*

**Ampliación y mejora de un Dashboard interactivo para el
análisis de supervivencia aplicado al Abandono Escolar**

Universitario

**Expansion and Improvement of an Interactive Dashboard
for Survival Analysis Applied to University Dropout**

Autor: Alfonso Muñoz Cuesta

Director: D. Juan Alfonso Lara Torralbo

Manual Técnico

mayo, 2026



UNIVERSIDAD
DE
CÓRDOBA



DECLARACIÓN DE ORIGINALIDAD DEL TFG

Ampliación y mejora de un Dashboard interactivo para el análisis de supervivencia aplicado al Abandono Escolar Universitario

Resumen

Palabras clave

Expansion and Improvement of an Interactive Dashboard for Survival Analysis Applied to University Dropout

Abstract

Keywords



UNIVERSIDAD
DE
CÓRDOBA

Trabajo Fin de Grado

*Dedicado a
mi familia*

Agradecimientos

En primer lugar, me gustaría expresar...

Índice general

Índice de figuras	XI
Índice de tablas	XVI
1. Introducción	1
2. Objetivos	3
3. Antecedentes	5
4. Planificación temporal	7
5. Marco teórico	10
5.1. Fundamentos del análisis de supervivencia	10
5.2. Técnicas clásicas de análisis de supervivencia	11
5.2.1. Kaplan-Meier	11
5.2.2. Regresión de Cox	12
5.2.3. Test de Log-Rank	12
5.3. Técnicas incorporadas en la ampliación	13
5.3.1. Modelo exponencial	13
5.3.2. Modelo de Weibull	14
5.3.3. Random Survival Forest	15
6. Recursos	17
6.1. Recursos humanos	17
6.2. Recursos hardware	18
6.3. Recursos software	19
6.4. Recursos de datos	20
7. Especificación de requisitos	21
7.1. Requisitos funcionales	21
7.2. Requisitos no funcionales	22
7.3. Requisitos de información	23

8. Análisis del problema	24
8.1. Identificación de actores	24
8.2. Identificación de casos de uso	25
8.3. Diagrama de casos de uso	27
8.4. Especificación de los casos de uso	28
8.4.1. CU-1: EjecutarModeloExponencial	28
8.4.2. CU-2: EjecutarModeloWeibull	30
8.4.3. CU-3: EjecutarRandomSurvivalForest	32
8.4.4. CU-4: ConsultarComparacionDeTecnica	34
8.4.5. CU-5: ExportarInformesYResultados	37
8.4.6. CU-6: CambiarIdiomaInterfaz	39
8.4.7. CU-7: AnalizarVariablesNoBinarias	40
8.4.8. CU-8: MejorarInterpretacionIA	42
8.4.9. CU-9: AmpliarVisualizacionesCoxTestLogRank	43
8.5. Matriz de trazabilidad	44
8.6. Diagramas de actividad	45
8.6.1. CU-1: EjecutarModeloExponencial	45
8.6.2. CU-2: EjecutarModeloWeibull	45
8.6.3. CU-3: EjecutarRandomSurvivalForest	46
8.6.4. CU-4: ConsultarComparacionDeTecnica	46
8.6.5. CU-5: ExportarInformesYResultados	47
8.6.6. CU-6: CambiarIdiomaInterfaz	48
8.6.7. CU-7: AnalizarVariablesNoBinarias	48
8.6.8. CU-8: MejorarInterpretacionIA	49
8.6.9. CU-9: AmpliarVisualizacionesCoxTestLogRank	49
9. Diseño del sistema	50
9.1. Arquitectura del sistema	50
9.2. Diseño de los datos de análisis	52
9.3. Diseño de la interfaz	54
9.3.1. Interfaz de usuario (UI)	54
9.4. Diagrama de clases y componentes	71
9.5. Diagramas de secuencia	73
10. Implementación	82
10.1. Tecnologías y librerías utilizadas	82
10.2. Estructura del código fuente	83
10.3. Carga y preprocesamiento de datos	85
10.4. Implementación de los modelos	86
10.4.1. Modelo exponencial	86

10.4.2. Modelo de Weibull	86
10.4.3. Random Survival Forest	88
10.4.4. Regresión de Cox y Test de Log-Rank	89
10.5. Generación de gráficas	90
10.6. Integración con callbacks	91
10.7. Interpretación mediante inteligencia artificial	91
10.8. Exportación de informes	93
10.9. Internacionalización del Dash	95
10.10 Aspectos destacables de la implementación	96
11. Pruebas	98
11.1. CU-1: EjecutarModeloExponencial	98
11.2. CU-2: EjecutarModeloWeibull	100
11.3. CU-3: EjecutarRandomSurvivalForest	101
11.4. CU-4: ConsultarComparacionDeTecnicas	103
11.5. CU-5: ExportarInformesYResultados	105
11.6. CU-6: CambiarIdiomaInterfaz	107
11.7. CU-7: AnalizarVariablesNoBinarias	109
11.8. CU-8: MejorarInterpretacionIA	111
11.9. CU-9: AmpliarVisualizacionesCoxTestLogRank	115
12. Discusión y conclusiones finales	118
12.1. Análisis de los objetivos iniciales	118
12.2. Problemas encontrados en el desarrollo	119
12.3. Conclusiones y posibles mejoras	120
Bibliografía	122
A. Anexo del Manual de Usuario	125
A.1. Introducción	125
A.2. Instalación	125
A.2.1. Requisitos previos	125
A.2.2. Descarga del código	127
A.2.3. Instalación de las librerías de Python requeridas	128
A.2.4. Ejecución de la aplicación	128
A.2.5. Ejecución del módulo de inteligencia artificial	129
A.2.6. Desinstalación del proyecto	130
A.3. Interfaz de usuario	130
A.3.1. Pantalla principal	131
A.3.2. Selector de idioma	132
A.3.3. Carga del dataset	132

A.3.4.	Preprocesamiento del dataset	133
A.3.5.	Navegación entre módulos	134
A.4.	Visualización del dataset	134
A.4.1.	Consulta del dataset cargado	134
A.4.2.	Consulta del dataset preprocesado	135
A.5.	Análisis de covariables	136
A.5.1.	Selección de covariables	136
A.5.2.	Interpretación de tablas y gráficas	136
A.6.	Técnicas de análisis de supervivencia	137
A.6.1.	Kaplan-Meier	137
A.6.2.	Regresión de Cox	138
A.6.3.	Test Log-Rank	139
A.6.4.	Modelo Weibull	140
A.6.5.	Modelo Exponencial	141
A.6.6.	Random Survival Forest	142
A.6.7.	Comparación de técnicas	143
A.7.	Interpretación con inteligencia artificial	144
A.7.1.	Arranque del servidor local de IA	144
A.7.2.	Generación de explicaciones automáticas	145
A.7.3.	Limitaciones de las interpretaciones generadas	145
A.8.	Exportación de informes PDF	146
A.8.1.	Selección de contenido del informe	146
A.8.2.	Generación y descarga del PDF	147
A.8.3.	Ubicación de los informes generados	148
A.9.	Errores frecuentes	148
A.9.1.	Problemas al cargar el dataset	149
A.9.2.	Problemas al acceder a módulos sin preprocesar	149
A.9.3.	Problemas durante la instalación de dependencias	150
A.9.4.	Problemas al arrancar la aplicación	151
A.9.5.	Problemas con el módulo de inteligencia artificial	152
A.9.6.	Problemas al generar informes PDF	152
A.10.	Flujo completo recomendado	153
B.	Anexo del Manual de código fuente	155
B.1.	Finalidad del manual de código fuente	155
B.2.	Criterios de organización del código	155
B.3.	Diccionario de variables principales	156
B.3.1.	Variables principales del dataset	157
B.3.2.	Variables internas de la aplicación	160
B.4.	Funciones principales por archivo	161

B.4.1. Preprocesamiento del dataset	161
B.4.2. Ajuste de Random Survival Forest	162
B.4.3. Llamada al servidor local de inteligencia artificial	163
B.5. Resumen de archivos principales	164
B.6. Flujo interno de ejecución	166
B.7. Criterios para ampliar el código	167
B.8. Puntos delicados para el mantenimiento	167

Índice de figuras

1.	Ejemplo de funciones de supervivencia exponenciales	14
2.	Ejemplo de distribuciones de Weibull	15
3.	Ejemplo de curvas predichas mediante Random Survival Forest	16
4.	Diagrama de casos de uso de la ampliación del sistema	27
5.	Diagrama de actividad del CU-2: EjecutarModeloExponencial	45
6.	Diagrama de actividad del CU-2: EjecutarModeloWeibull	45
7.	Diagrama de actividad del CU-3: EjecutarRandomSurvivalForest	46
8.	Diagrama de actividad del CU-4: ConsultarComparacionDeTécnicas	46
9.	Diagrama de actividad del CU-5: ExportarInformesYResultados	47
10.	Diagrama de actividad del CU-6: CambiarIdiomaInterfaz	48
11.	Diagrama de actividad del CU-7: AnalizarVariablesNoBinarias	48
12.	Diagrama de actividad del CU-8: MejorarInterpretacionIA	49
13.	Diagrama de actividad del CU-9: AmpliarVisualizacionesCoxTestLogRank	49
14.	Pantalla principal del dashboard	55
15.	Visualización del archivo bruto cargado en el sistema	55
16.	Visualización del archivo limpio tras el preprocesamiento	56
17.	Vista inicial de la sección Ver dataset	56
18.	Detalle de la visualización del dataset dentro del dashboard	57
19.	Sección de análisis de covariables	57
20.	Sección de análisis de supervivencia	58
21.	Vista general del análisis Kaplan-Meier	59
22.	Curva de supervivencia generada mediante Kaplan-Meier	59
23.	Filtros disponibles en el análisis Kaplan-Meier	60
24.	Exportación a PDF de los resultados de Kaplan-Meier	61
25.	Vista principal de la regresión de Cox	62
26.	Filtros utilizados en la regresión de Cox	62
27.	Gráfica generada a partir del modelo de regresión de Cox	62
28.	Botón de exportación a PDF utilizado en Cox y también en el Test de Log-Rank	63
29.	Vista principal del Test de Log-Rank	63
30.	Filtros compartidos por Cox y el Test de Log-Rank	64
31.	Ejemplo de resultado generado mediante el Test de Log-Rank	64
32.	Vista inicial del análisis mediante el modelo de Weibull	65

33.	Resultados y visualizaciones del modelo de Weibull	65
34.	Botón de exportación combinada utilizado en Weibull y Exponencial, ya que ambos modelos exportan el mismo tipo de resultados	66
35.	Vista inicial del análisis mediante el modelo exponencial	67
36.	Resultados y visualizaciones del modelo exponencial	67
37.	Vista inicial del análisis mediante Random Survival Forest	68
38.	Resultados generales generados por Random Survival Forest	68
39.	Visualización complementaria del análisis Random Survival Forest . .	69
40.	Página de comparación de técnicas de supervivencia	69
41.	Detalle explicativo de la comparación de técnicas	70
42.	Botón para consultar el dataset bruto	70
43.	Botón para consultar el dataset limpio	70
44.	Botón para solicitar una interpretación mediante inteligencia artificial	70
45.	Botón de exportación de resultados	71
46.	Botón de confirmación para generar y descargar el informe PDF . . .	71
47.	Botón de exportación combinada para Weibull y Exponencial	71
48.	Botón de simulación de perfiles en Random Survival Forest	71
49.	Diagrama de secuencia del CU-1: EjecutarModeloExponencial	74
50.	Diagrama de secuencia del CU-2: EjecutarModeloWeibull	75
51.	Diagrama de secuencia del CU-3: EjecutarRandomSurvivalForest . . .	76
52.	Diagrama de secuencia del CU-4: ConsultarComparacionDeTecnicas .	77
53.	Diagrama de secuencia del CU-5: ExportarInformesYResultados . . .	78
54.	Diagrama de secuencia del CU-6: CambiarIdiomaInterfaz	79
55.	Diagrama de secuencia del CU-7: AnalizarVariablesNoBinarias	80
56.	Diagrama de secuencia del CU-8: MejorarInterpretacionIA	80
57.	Diagrama de secuencia del CU-9: AmpliarVisualizacionesCoxTestLogRank	81
58.	Fragmento de la implementación del modelo de Weibull, donde se ajustan los modelos paramétricos y se obtienen métricas de comparación	87
59.	Fragmento de la implementación de Random Survival Forest, incluyendo la construcción del objeto de supervivencia y el ajuste del modelo	89
60.	Definición del prompt académico utilizado para orientar las respuestas generadas por el módulo de inteligencia artificial	93
61.	Llamada al modelo local de inteligencia artificial mediante una petición HTTP y tratamiento posterior de la respuesta generada . . .	93
62.	Ejemplos de informes PDF generados por el sistema a partir de distintos análisis	95
63.	Implementación del sistema de traducciones empleado en la internacionalización del dashboard	96

64.	Evidencia de ejecución correcta del CU-1: EjecutarModeloExponencial	99
65.	Evidencia de ejecución correcta del CU-2: EjecutarModeloWeibull	100
66.	Evidencia de ejecución correcta del CU-3: EjecutarRandomSurvivalForest	102
67.	Evidencia de ejecución correcta del CU-4: ConsultarComparacionDeTecnicas	104
68.	Evidencia de exportación correcta del CU-5: ExportarInformesYResultados	106
69.	Evidencia de análisis de variables en el CU-7: AnalizarVariablesNoBinarias	110
70.	Evidencia de acceso a la interpretación mediante IA del CU-8: MejorarInterpretacionIA	112
71.	Evidencia de visualización ampliada del CU-9: AmpliarVisualizacionesCoxTestLogRank	111
72.	Descarga del proyecto desde GitHub como archivo ZIP	127
73.	Pantalla principal de la aplicación	131
74.	Botón que cambia el idioma de la aplicación	132
75.	Archivo CSV cargado antes de iniciar el preprocesamiento	133
76.	Confirmación del dataset preprocesado	133
77.	Consulta del dataset limpio desde la sección de visualización	135
78.	Descripción de variables disponible para interpretar el dataset	135
79.	Selección de covariables para la exploración descriptiva	136
80.	Página de acceso a las técnicas de análisis de supervivencia	137
81.	Resultado esperado al generar una curva Kaplan-Meier estratificada	138
82.	Visualización de los efectos obtenidos tras ejecutar la regresión de Cox	139
83.	Resultado mostrado al comparar grupos mediante el Test Log-Rank	140
84.	Resultados que debe revisar el usuario tras ejecutar el modelo Weibull	141
85.	Resultados que debe revisar el usuario tras ejecutar el modelo Exponencial	142
86.	Curvas y métricas generadas por Random Survival Forest	143
87.	Vista utilizada para consultar la comparación entre técnicas disponibles	144
88.	Botón de interpretación mediante inteligencia artificial	145
89.	Ejemplo de explicación generada para apoyar la interpretación de resultados	145
90.	Botón que inicia la exportación desde un módulo de análisis	146
91.	Opciones de contenido para el informe PDF	147
92.	Ejemplo de informe PDF generado a partir de los resultados seleccionados	148
93.	Mensaje mostrado cuando el archivo cargado no corresponde al dataset esperado	149
94.	Aviso mostrado al acceder a una sección que requiere el dataset preprocesado	150
95.	Fragmentos de la función de preprocesamiento del dataset	162
96.	Fragmento de código encargado del ajuste de Random Survival Forest	163

97. Fragmento de código utilizado para llamar al servidor local de inteligencia artificial	164
---	-----

Índice de tablas

1.	Identificación de casos de uso de la ampliación del sistema	26
2.	Especificación del CU-1: Ejecutar modelo exponencial	28
3.	Especificación del CU-2: Ejecutar modelo Weibull	30
4.	Especificación del CU-3: Ejecutar Random Survival Forest	32
5.	Especificación del CU-4: Consultar comparación de técnicas	34
6.	Especificación del CU-5: Exportar informes y resultados	37
7.	Especificación del CU-6: Cambiar idioma de la interfaz	39
8.	Especificación del CU-7: Analizar variables no binarias	40
9.	Especificación del CU-8: Mejorar interpretación mediante IA	42
10.	Especificación del CU-9: Ampliar visualizaciones Cox y Test de Log-Rank	43
11.	Matriz de trazabilidad entre requisitos funcionales y casos de uso . . .	44
12.	Componentes principales de la arquitectura del sistema	51
13.	Variables principales utilizadas y justificación de su selección	53
14.	Vista de diseño basada en clases y componentes del sistema	73
15.	Tecnologías con mayor peso en la ampliación implementada	83
16.	Distribución de responsabilidades en el código fuente	84
17.	Prueba válida del CU-1: EjecutarModeloExponencial	98
18.	Prueba errónea del CU-1: EjecutarModeloExponencial	99
19.	Prueba válida del CU-2: EjecutarModeloWeibull	100
20.	Prueba errónea del CU-2: EjecutarModeloWeibull	101
21.	Prueba válida del CU-3: EjecutarRandomSurvivalForest	102
22.	Prueba errónea del CU-3: EjecutarRandomSurvivalForest	103
23.	Prueba válida del CU-4: ConsultarComparacionDeTecnicas	104
24.	Prueba errónea del CU-4: ConsultarComparacionDeTecnicas	105
25.	Prueba válida del CU-5: ExportarInformesYResultados	106
26.	Prueba errónea del CU-5: ExportarInformesYResultados	107
27.	Prueba válida del CU-6: CambiarIdiomaInterfaz	108
28.	Prueba errónea del CU-6: CambiarIdiomaInterfaz	109
29.	Prueba válida del CU-7: AnalizarVariablesNoBinarias	110
30.	Prueba errónea del CU-7: AnalizarVariablesNoBinarias	111
31.	Prueba válida del CU-8: MejorarInterpretacionIA	112

32.	Prueba errónea del CU-8: MejorarInterpretacionIA	113
33.	Configuración utilizada para medir el rendimiento del módulo de inteligencia artificial	114
34.	Resultados de rendimiento del modelo de inteligencia artificial por técnica de análisis	114
35.	Prueba válida del CU-9: AmpliarVisualizacionesCoxTestLogRank . .	115
36.	Prueba errónea del CU-9: AmpliarVisualizacionesCoxTestLogRank . .	117
37.	Variables principales procedentes del dataset académico	157
38.	Variables internas más relevantes para entender el flujo de la aplicación	160
39.	Resumen de archivos principales del código fuente	165
40.	Pasos recomendados para incorporar una nueva funcionalidad	167

1 Introducción

El estudio del comportamiento académico del alumnado universitario ha ido ganando peso en los últimos años dentro del ámbito de la educación superior. No es una preocupación menor. Detrás de cada estudiante que se abandona hay algo más que una baja en el sistema: hay una trayectoria truncada, recursos que no llegan a ningún puerto y, con demasiada frecuencia, una señal de alerta que nadie supo detectar a tiempo. Por eso, entender qué factores influyen en la continuidad académica no es un ejercicio teórico, sino algo con consecuencias muy reales para personas e instituciones. Y es que cuando una universidad pierde a un estudiante a mitad del camino, no solo falla el sistema de retención, sino que se pierde también una oportunidad de intervenir que, con las herramientas adecuadas, podría haberse aprovechado.

En ese escenario, el análisis de datos se ha convertido en un aliado cada vez más valioso. No porque los números lo expliquen todo, sino porque permiten ver patrones que a simple vista resultan invisibles como qué perfiles tienen más riesgo, en qué momentos del curso se concentran las bajas o qué variables marcan la diferencia entre quien continúa y quien no. Aplicar estas técnicas al abandono universitario abre una vía verdaderamente útil para pasar de la intuición a la evidencia, y de la evidencia a la acción.

Este Trabajo Fin de Grado nace precisamente de esa inquietud. Toma como punto de partida un dashboard interactivo desarrollado en un trabajo previo, orientado al análisis del abandono universitario a partir de datos reales de estudiantes. Aquel sistema fue un primer paso valioso, pero dejó abiertas varias puertas: modelos estadísticos limitados, una arquitectura mejorable y un enfoque todavía demasiado descriptivo. El objetivo de este Trabajo de Fin de Grado es dar el siguiente salto y construir sobre esa base algo más ambicioso.

Para ello, se plantean mejoras en varios frentes. A nivel técnico, se refuerza la arquitectura de la aplicación para ganar en escalabilidad, rendimiento y estabilidad. A nivel analítico, se incorporan nuevas técnicas estadísticas y modelos con mayor capacidad predictiva, capaces de identificar patrones de comportamiento y anticipar posibles situaciones de abandono antes de que sea demasiado tarde para intervenir. Y a nivel de usabilidad, se desarrollan herramientas de visualización más claras e interpretables, se añade soporte multilingüe y se amplían las opciones de exportación

CAPÍTULO 1. INTRODUCCIÓN

de resultados, pensando siempre en quienes, al final del día, tienen que tomar decisiones a partir de esa información.

El objetivo final además de mostrar información es transformarla en conocimiento útil. Una buena visualización puede resultar atractiva y estar bien construida técnicamente, pero su verdadero valor aparece cuando ayuda a entender qué está ocurriendo y, sobre todo, qué podría hacerse a continuación. Eso es lo que se busca con este proyecto: contribuir a que los responsables académicos dispongan de una herramienta real para comprender, anticipar y, en la medida de lo posible, prevenir el abandono en el entorno universitario.

2 Objetivos

El objetivo principal de este Trabajo de Fin de Grado es ampliar y mejorar una aplicación interactiva orientada al análisis de supervivencia, aplicada al estudio del abandono académico en el ámbito universitario. No se trata solo de añadir nuevas funciones por añadirlas, sino de hacer que la herramienta sea más útil, más clara y más capaz de ayudar a interpretar un problema que, en la práctica, afecta tanto a los estudiantes como a las instituciones. Para ello, se plantea el desarrollo de nuevas funcionalidades que permitan analizar con mayor precisión los factores que influyen en este fenómeno.

La solución propuesta, al igual que en el trabajo previo, se desarrollará utilizando tecnologías como Python y Dash. A partir de estas herramientas se busca construir una plataforma que facilite la realización de análisis interactivos, la visualización de resultados y la aplicación de técnicas estadísticas sobre datos educativos reales. Como punto de partida se utilizará el conjunto de datos empleado en el Trabajo de Fin de Grado anterior, lo que permitirá mantener una base común para comparar resultados y valorar con más justicia las mejoras introducidas.

Además, el proyecto no se limita únicamente al análisis del abandono académico. La idea es ir un paso más allá y mejorar la utilidad real de la herramienta desarrollada. En este sentido, se pretende evolucionar la aplicación hacia un sistema más completo, capaz de servir como apoyo en la toma de decisiones por parte de responsables académicos y de contribuir al diseño de estrategias orientadas a mejorar la retención del alumnado.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

- Implementar y probar una inteligencia artificial más rápida que la del trabajo previo, con el fin de optimizar los tiempos de generación y visualización de resultados. Para comprobar esta mejora, se medirán aspectos como el tiempo de respuesta, el tiempo de generación de resultados y el consumo de recursos.
- Incorporar soporte multilingüe, permitiendo que la aplicación pueda utilizarse en distintos idiomas, concretamente castellano e inglés. Además, se verificará que los elementos de la interfaz estén correctamente traducidos, de forma que la herramienta resulte más accesible para distintos perfiles de usuario.

CAPÍTULO 2. OBJETIVOS

- Ampliar el conjunto de variables utilizadas en el modelo, incorporando nuevos atributos y variables no binarias que permitan enriquecer el análisis y mejorar la capacidad explicativa de los modelos. Asimismo, se justificará la selección de dichas variables, explicando los criterios seguidos para incluirlas y valorando su relevancia frente a otros posibles atributos no considerados.
- Incluir gráficos y métodos estadísticos complementarios basados en el Test de Log-Rank y en el modelo de riesgos proporcionales de Cox. Con ello se pretende facilitar el análisis comparativo entre distintos grupos de estudiantes y ofrecer una interpretación más completa de los resultados.
- Explorar e integrar nuevas técnicas de análisis de supervivencia, como modelos paramétricos del tipo Weibull y Exponencial, junto con enfoques basados en aprendizaje automático como Random Survival Forest. La finalidad será comparar distintos enfoques, estudiar sus factores asociados y mejorar la precisión y versatilidad del sistema.
- Permitir la exportación automática de informes, imágenes y representaciones gráficas, facilitando la documentación de los resultados y su uso en contextos de investigación o gestión académica.

En conjunto, estos objetivos marcan el camino del proyecto y permiten comprobar, de forma ordenada, si las mejoras introducidas cumplen realmente con lo planteado en el anteproyecto. Además, este proceso de validación ayudará a evaluar la fiabilidad, la utilidad y el alcance de la herramienta desarrollada, evitando que la mejora quede solo en una ampliación técnica sin impacto práctico.

3 Antecedentes

El análisis de supervivencia es una técnica estadística diseñada para estudiar el tiempo que transcurre hasta que ocurre un evento determinado. Su característica más valiosa es que permite trabajar con datos censurados, es decir, con casos en los que el evento todavía no se ha producido durante el periodo de observación, algo que otros métodos suelen ignorar. En medicina se usa para estudiar la evolución de pacientes y en ingeniería para estimar la vida útil de componentes. En educación, aplicado al abandono universitario, resulta especialmente útil porque no solo permite saber cuántos estudiantes abandonan, sino también cuándo lo hacen y bajo qué circunstancias es más probable que ocurra e

Dentro de este campo, los modelos más consolidados son Kaplan-Meier, que estima la probabilidad de permanencia a lo largo del tiempo, el Test de Log-Rank, que permite comparar trayectorias entre grupos, y la regresión de Cox, que identifica qué variables —rendimiento académico, edad, situación socioeconómica, entre otras— influyen de forma significativa en el riesgo de abandono. Más recientemente, han ganado terreno enfoques como los modelos paramétricos de Weibull o los Random Survival Forests, que incorporan la potencia del aprendizaje automático para mejorar la capacidad predictiva del análisis.

La literatura sobre abandono universitario aborda el problema desde distintos ángulos. Algunos trabajos se centran en los factores personales, académicos o institucionales que condicionan la trayectoria del estudiante. Otros apuntan directamente a la detección temprana de perfiles de riesgo. Esta segunda línea es la que más se aproxima al enfoque de este proyecto, donde el análisis no es un fin en sí mismo sino una herramienta para anticipar y prevenir.

El antecedente más directo es el Trabajo Fin de Grado de Lucía Cabezuelo Pérez (2025), titulado *“Análisis, Diseño e Implementación de un Dashboard Interactivo para el Análisis de Supervivencia aplicado al Abandono Escolar Universitario”*. En él se desarrolló una aplicación web en Python y Dash que integraba los modelos de Kaplan-Meier, regresión de Cox y Test de Log-Rank, con visualizaciones interactivas y un módulo de inteligencia artificial basado en Llama 3 a través de Ollama, capaz de generar interpretaciones automáticas de los resultados. Fue un trabajo sólido, que demostró la viabilidad del enfoque y dejó una base técnica sobre la que merece la pena seguir construyendo.

CAPÍTULO 3. ANTECEDENTES

Este proyecto recoge ese testigo y lo lleva más lejos. Se amplía el abanico de modelos estadísticos disponibles, se incorporan técnicas de aprendizaje automático aplicadas a la supervivencia, se sustituye el módulo de IA por una solución más rápida y eficiente, y se enriquece la plataforma con soporte multilingüe, exportación de resultados y una interfaz rediseñada. No se trata de rehacer lo que ya funcionaba, sino de completar lo que faltaba.

4 Planificación temporal

La planificación temporal de este Trabajo Fin de Grado se organiza en varias fases, pensadas para abordar de forma ordenada la ampliación y mejora de la aplicación existente. La verdad es que, en un proyecto de este tipo, contar con una planificación clara ayuda mucho: permite avanzar sin perder de vista los objetivos principales y evita que las mejoras técnicas queden desconectadas entre sí. Cada fase se centra en un conjunto de tareas concretas, desde la preparación inicial del entorno hasta la validación final de los resultados obtenidos.

1. Preparación del entorno y revisión del sistema existente

En esta primera fase se realizará una revisión completa del TFG original y de la aplicación desarrollada previamente. El objetivo será comprender con calma su funcionamiento interno, sus dependencias, la estructura del código y la forma en la que se organizan los datos. Además, se configurará el entorno de trabajo en el equipo local, incluyendo las herramientas de desarrollo necesarias, las bibliotecas de Python, el entorno de Dash y los archivos de datos empleados por la aplicación.

Esta etapa servirá como punto de partida técnico. Antes de incorporar nuevas funcionalidades, resulta necesario asegurarse de que el sistema base funciona correctamente y de que se conocen sus límites.

2. Análisis inicial y planificación

En esta fase se identificarán las limitaciones del sistema original y las áreas que requieren una mejora más clara. También se definirán los nuevos requisitos funcionales, no funcionales y de información, así como los objetivos técnicos y metodológicos del proyecto. Además, se elaborará un plan de trabajo que permita repartir mejor los recursos, los tiempos y las herramientas que se utilizarán durante el desarrollo.

3. Diseño de mejoras y ampliaciones

A partir del análisis previo, se planificarán las mejoras principales de la aplicación. En esta parte se estudiará cómo incorporar el soporte multilingüe, la implementación de gráficos adicionales basados en el Test de Log-Rank y en la regresión de Cox, y los cambios necesarios en la interfaz para que la experiencia de uso resulte más clara. También se valorará el tratamiento de atributos no binarios del dataset, junto con la integración de una inteligencia artificial más rápida y eficiente.

CAPÍTULO 4. PLANIFICACIÓN TEMPORAL

4. Desarrollo e implementación

Durante esta fase se llevará a cabo la programación de las mejoras diseñadas. Será, previsiblemente, la parte más extensa del trabajo, ya que aquí las ideas definidas en las fases anteriores se convierten en funcionalidades reales. Se implementarán nuevos módulos, se integrarán funcionalidades interactivas y se añadirán modelos de análisis de supervivencia adicionales. Asimismo, se actualizará la interfaz de usuario en Dash y se optimizará el código para mejorar el rendimiento general del sistema.

5. Pruebas y validación

En esta fase se realizarán pruebas para comprobar la estabilidad y fiabilidad del sistema mejorado. No basta con que la aplicación funcione una vez; debe responder de forma coherente ante distintos escenarios. Por ello, se evaluará el comportamiento de las nuevas funcionalidades, la precisión de los modelos estadísticos, la exportación de informes e imágenes y la velocidad de respuesta de la inteligencia artificial. Se emplearán datos reales y, cuando sea necesario, datos de prueba para comprobar la calidad de los resultados.

6. Documentación técnica y manuales

Se elaborará la documentación completa del proyecto, incluyendo el manual técnico, el manual de usuario y una guía de instalación y ejecución. En estos documentos se explicará la estructura de la aplicación, el funcionamiento de las nuevas funcionalidades y las instrucciones necesarias para utilizar el sistema. Además, se documentará el código fuente de forma clara, de manera que el trabajo pueda entenderse y mantenerse en el futuro.

7. Evaluación final y conclusiones

En la última fase se analizará el cumplimiento de los objetivos planteados y se evaluará el impacto de las mejoras implementadas sobre la aplicación original. También se expondrán las conclusiones generales del trabajo, los problemas encontrados durante el desarrollo y las posibles líneas de mejora que podrían abordarse en el futuro.

CAPÍTULO 4. PLANIFICACIÓN TEMPORAL

Cronograma de trabajo:

A continuación se presenta una estimación de la distribución temporal del proyecto, con una duración total aproximada de 300 horas:

FASE	TIEMPO (HORAS)
Preparación	30
Análisis y diseño	60
Desarrollo e implementación	120
Pruebas y validación	50
Documentación y manuales	25
Evaluación final y conclusiones	15
TOTAL ESTIMADO	300 HORAS

La tabla muestra cómo se reparten las horas de trabajo entre las distintas fases del proyecto. Como es lógico, la parte que más dedicación requiere es la de desarrollo e implementación, ya que en ella se concentran la mayoría de las mejoras de la aplicación. También tienen un peso importante las fases de análisis, diseño y validación, porque ayudan a definir bien el camino antes de programar y a comprobar después que los resultados obtenidos son fiables. En conjunto, esta distribución ofrece una visión realista del esfuerzo previsto y permite organizar el proyecto con mayor claridad.

5 Marco teórico

En este capítulo se presentan los conceptos teóricos sobre los que se apoya el proyecto. Al tratarse de una ampliación de un dashboard ya existente, no se parte desde cero, sino desde una base previa centrada en el análisis de supervivencia aplicado al abandono académico universitario. Por ello, el marco teórico se organiza comenzando por los fundamentos generales de esta técnica, continúa con los modelos clásicos ya presentes en el sistema original y, finalmente, introduce las técnicas incorporadas en esta nueva versión.

5.1. Fundamentos del análisis de supervivencia

El análisis de supervivencia es un conjunto de técnicas estadísticas orientadas al estudio del tiempo que transcurre hasta que ocurre un determinado evento. Aunque el término “supervivencia” procede principalmente del ámbito médico, donde se utiliza para analizar el tiempo hasta el fallecimiento, la recuperación o la recaída de un paciente, su aplicación no se limita a ese contexto. También se emplea en ingeniería para estudiar la vida útil de componentes, en economía para analizar la duración de determinados procesos y, más recientemente, en educación para estudiar fenómenos como el abandono académico, es decir, contextos muy diferentes pero con algo en común: todos se preguntan cuánto tiempo tarda en ocurrir algo.

En este Trabajo Fin de Grado, ese “algo” es el abandono universitario. La idea principal no consiste únicamente en saber si un estudiante abandona o no, sino en analizar cuándo se produce ese abandono y qué factores pueden estar relacionados con él. Este matiz es importante, porque permite observar el fenómeno como un proceso temporal y no como un resultado aislado. Dicho de forma sencilla, no es lo mismo detectar que un estudiante abandona al inicio de sus estudios que observar que lo hace tras un periodo prolongado de permanencia.

Uno de los conceptos clave en este tipo de análisis es la función de supervivencia. Esta función representa la probabilidad de que un individuo, en este caso un estudiante, continúe en el sistema más allá de un instante determinado. Aplicado al abandono académico, permite estimar la probabilidad de permanencia del alumnado a lo largo del tiempo. A partir de esta información se pueden identificar momentos de mayor riesgo y estudiar si ciertos grupos presentan comportamientos diferentes.

CAPÍTULO 5. MARCO TEÓRICO

Otro concepto fundamental es el de censura. En muchos estudios no se observa el evento para todos los individuos durante el periodo analizado. Por ejemplo, puede ocurrir que un estudiante siga matriculado al finalizar el periodo de observación. En ese caso, no se puede afirmar que nunca vaya a abandonar, pero sí se sabe que no lo ha hecho hasta ese momento. El análisis de supervivencia permite aprovechar este tipo de información, algo especialmente valioso en el ámbito educativo, donde las trayectorias académicas no siempre finalizan dentro del intervalo de estudio disponible.

Además, el análisis de supervivencia permite incorporar variables explicativas, también denominadas covariables, que ayudan a estudiar qué factores pueden influir en el tiempo hasta el evento. En el caso del abandono universitario, estas variables pueden estar relacionadas con aspectos académicos, personales o administrativos, como el rendimiento, la edad, el género, la discapacidad, el nivel educativo previo o los créditos cursados. Gracias a ello, el análisis no se limita a describir una curva general de permanencia, sino que puede aportar una visión más rica y útil del comportamiento de distintos perfiles de estudiantes.

5.2. Técnicas clásicas de análisis de supervivencia

El sistema desarrollado en el Trabajo Fin de Grado previo ya incorporaba varias técnicas clásicas de análisis de supervivencia. Estas técnicas constituyen la base sobre la que se apoya la ampliación actual, ya que permiten estimar la permanencia del alumnado, comparar grupos y estudiar el efecto de diferentes covariables. Aunque cada método tiene un propósito distinto, todos ellos ayudan a entender mejor el abandono académico desde una perspectiva temporal.

5.2.1. Kaplan-Meier

El estimador de Kaplan-Meier es una de las técnicas más utilizadas dentro del análisis de supervivencia. Su objetivo principal es estimar la función de supervivencia a partir de los datos observados, teniendo en cuenta tanto los casos en los que ocurre el evento como aquellos que están censurados. En el contexto de este proyecto, permite representar de forma gráfica la probabilidad de que los estudiantes permanezcan en el sistema universitario a medida que avanza el tiempo.

Una de sus ventajas es que ofrece una interpretación visual bastante directa. La curva de supervivencia permite observar en qué momentos se producen descensos más marcados, lo que puede indicar periodos de mayor riesgo de abandono. Además,

CAPÍTULO 5. MARCO TEÓRICO

puede aplicarse a distintos grupos de estudiantes, por ejemplo según género, tramo de edad o nivel educativo previo, facilitando una primera comparación entre perfiles.

No obstante, Kaplan-Meier tiene también ciertas limitaciones. Principalmente, describe la supervivencia de forma general o por grupos, pero no permite modelar de manera conjunta el efecto de varias covariables. Por ello, suele utilizarse como una herramienta inicial de exploración y visualización, complementándose con otros métodos más adecuados para analizar factores explicativos.

5.2.2. Regresión de Cox

El modelo de riesgos proporcionales de Cox permite estudiar la relación entre el tiempo hasta el evento y un conjunto de covariables. A diferencia de Kaplan-Meier, que se centra en estimar curvas de supervivencia, la regresión de Cox permite analizar cómo ciertas variables influyen en el riesgo de que ocurra el evento. En este proyecto, dicho evento se corresponde con el abandono académico.

Este modelo resulta especialmente útil porque permite interpretar el efecto de cada variable mediante razones de riesgo. De forma simplificada, una variable puede asociarse con un aumento o una disminución del riesgo de abandono. Por ejemplo, determinadas características académicas podrían estar relacionadas con una mayor probabilidad de abandonar antes, mientras que otras podrían estar asociadas con una mayor permanencia.

La regresión de Cox es una técnica muy valiosa cuando se desea estudiar el papel de varios factores al mismo tiempo. Sin embargo, parte de una hipótesis importante: los riesgos entre grupos deben ser proporcionales a lo largo del tiempo. Esto significa que el efecto relativo de las covariables se mantiene aproximadamente constante durante el periodo analizado. Si esta condición no se cumple, la interpretación del modelo puede verse afectada.

5.2.3. Test de Log-Rank

El Test de Log-Rank tiene un propósito más concreto y directo: comparar curvas de supervivencia entre grupos y determinar si las diferencias observadas son estadísticamente significativas o podrían deberse al azar. En el contexto del abandono universitario, permite preguntarse, por ejemplo, si los estudiantes de distintos tramos de edad realmente siguen trayectorias diferentes o si esa diferencia no es suficientemente relevante como para sacar conclusiones.

No explica por sí solo las causas, pero sí lanza señales. Si el test detecta diferencias significativas entre grupos, es una indicación de que esa variable merece ser estudiada con más detalle. Y ahí es donde entra en juego su papel dentro del dashboard: complementa a Kaplan-Meier, que visualiza las curvas, y a Cox, que analiza las covariables. Los tres juntos forman una base analítica coherente y bastante completa para abordar el abandono desde distintos ángulos.

5.3. Técnicas incorporadas en la ampliación

La ampliación desarrollada en este Trabajo Fin de Grado incorpora nuevas técnicas de análisis de supervivencia con el fin de enriquecer la capacidad analítica del sistema. La idea no es sustituir los métodos clásicos, sino complementarlos. Cada modelo aporta una forma distinta de mirar los datos, y esa variedad permite comparar enfoques, detectar patrones y ofrecer una interpretación más completa del fenómeno estudiado.

5.3.1. Modelo exponencial

El modelo exponencial es un modelo paramétrico de análisis de supervivencia. A diferencia de métodos como Kaplan-Meier, que no imponen una forma concreta a la función de supervivencia, los modelos paramétricos asumen una determinada distribución para describir el tiempo hasta el evento [1, 2]. En el caso del modelo exponencial, se supone que el riesgo de que ocurra el evento permanece constante a lo largo del tiempo.

Esta característica lo convierte en un modelo sencillo y fácil de interpretar. Si el riesgo de abandono se considera estable durante el periodo analizado, el modelo exponencial puede ofrecer una aproximación útil. Sin embargo, esa misma simplicidad puede convertirse en una limitación cuando el riesgo cambia con el tiempo, algo que puede ocurrir en contextos educativos. Por ejemplo, puede haber periodos iniciales con mayor incertidumbre o momentos concretos del curso en los que el abandono sea más probable.

Dentro del proyecto, el modelo exponencial permite incorporar una primera aproximación paramétrica al análisis y sirve como punto de comparación frente a modelos más flexibles. Además, facilita la generación de métricas y resultados que pueden contrastarse con otras técnicas implementadas en el dashboard.

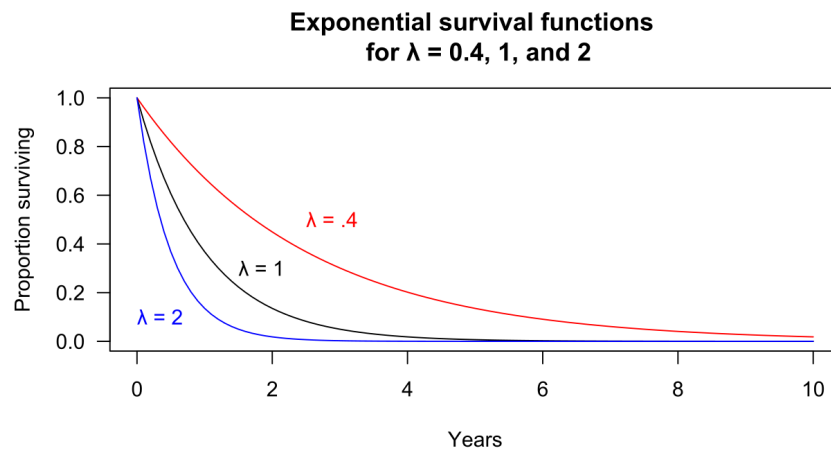


Figura 1: Ejemplo de funciones de supervivencia exponenciales con distintos valores del parámetro λ . Fuente: Michaelg2015, Wikimedia Commons [3].

5.3.2. Modelo de Weibull

El modelo de Weibull es también un modelo paramétrico, pero ofrece mayor flexibilidad que el modelo exponencial. Su principal ventaja es que permite representar situaciones en las que el riesgo no permanece constante, sino que puede aumentar o disminuir a lo largo del tiempo [1, 2]. Esa flexibilidad lo hace mucho más adecuado para capturar lo que realmente ocurre en las trayectorias académicas, donde el riesgo de abandono no es el mismo en el primer mes que en el tercer año.

Y es que la realidad es más complicada que una línea recta. Algunos estudiantes presentan mayor vulnerabilidad al principio, cuando todo es nuevo y la adaptación es más dura. Otros aguantan bien al inicio pero acumulan dificultades con el tiempo. Weibull puede capturar esos dos patrones, y eso lo convierte en una herramienta mucho más expresiva que el exponencial. Además, comparar los resultados de ambos modelos aporta información en sí misma: si Weibull ofrece un ajuste claramente mejor, es una señal de que el riesgo evoluciona con el tiempo y de que las estrategias de intervención deberían tenerlo en cuenta.

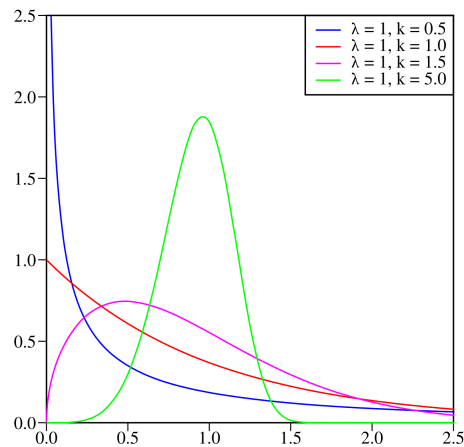


Figura 2: Ejemplo de distribuciones de Weibull con diferentes valores del parámetro de forma. Fuente: Calimo, Wikimedia Commons [4].

5.3.3. Random Survival Forest

Random Survival Forest es un salto cualitativo respecto a los modelos anteriores. Parte de la lógica de los bosques aleatorios, una técnica consolidada en aprendizaje automático, pero adaptada para trabajar con tiempos hasta evento y datos censurados [5]. Lo que lo distingue de todo lo anterior es que no exige asumir ninguna forma concreta para la función de supervivencia ni una relación lineal entre variables. Simplemente aprende de los datos.

Esa libertad tiene consecuencias prácticas importantes. En un fenómeno tan multifactorial como el abandono universitario, donde pueden intervenir al mismo tiempo el rendimiento, la situación personal, la carga académica y muchos otros factores, un modelo que capture relaciones complejas y no lineales puede detectar patrones que los métodos clásicos pasarían por alto. Además, Random Survival Forest ofrece algo especialmente útil para este proyecto: una medida de importancia de variables, que permite identificar qué factores pesan más en la predicción del abandono.

La contrapartida es que, a mayor complejidad, mayor necesidad de cautela. Los resultados de este modelo deben interpretarse con cuidado y, siempre que sea posible, contrastarse con los obtenidos por técnicas más transparentes. En este proyecto se implementa a través de `scikit-survival`, una librería especializada en análisis de tiempo hasta evento construida sobre el ecosistema de `scikit-learn` [6].

CAPÍTULO 5. MARCO TEÓRICO

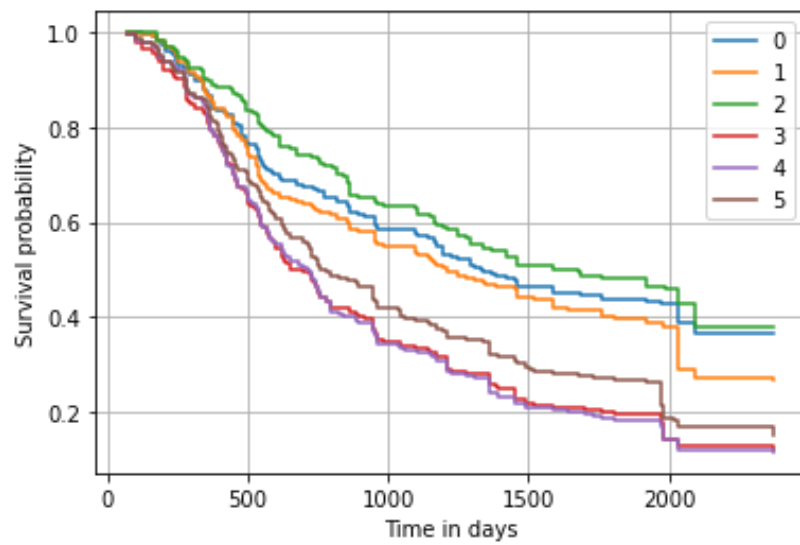


Figura 3: Ejemplo de curvas de supervivencia predichas mediante Random Survival Forest.
Fuente: documentación oficial de scikit-survival [7].

6 Recursos

6.1. Recursos humanos

El desarrollo de este Trabajo Fin de Grado ha sido realizado por Alfonso Muñoz Cuesta, estudiante del Grado de Ingeniería Informática. Su papel dentro del proyecto abarca el análisis, diseño, implementación, pruebas y documentación de la aplicación. En la práctica, esto implica no solo programar nuevas funcionalidades, sino también entender con detalle el sistema previo, detectar sus limitaciones y decidir qué mejoras aportan realmente valor.

Entre las tareas principales desarrolladas por el estudiante se encuentran las siguientes:

- Revisión del sistema desarrollado en el Trabajo Fin de Grado previo, con el fin de comprender su estructura, su funcionamiento interno y sus posibilidades de ampliación.
- Análisis de los requisitos necesarios para mejorar la aplicación y adaptar sus funcionalidades al nuevo alcance del proyecto.
- Diseño y programación de nuevas funcionalidades del dashboard interactivo, incluyendo la integración de modelos adicionales de análisis de supervivencia.
- Preparación y tratamiento de los datos utilizados por el sistema, prestando especial atención a las variables necesarias para los modelos estadísticos.
- Realización de pruebas para comprobar el correcto funcionamiento de la aplicación, la estabilidad del sistema y la coherencia de los resultados obtenidos.
- Mejora de la interfaz y de los elementos de visualización, con el objetivo de que la información generada sea más fácil de interpretar por parte del usuario.
- Redacción de la documentación asociada al proyecto, incluyendo la memoria, los manuales y los informes finales.

CAPÍTULO 6. RECURSOS

El proyecto ha contado además con la supervisión del Dr. Juan Alfonso Lara Torralbo, profesor del Departamento de Ciencia de la Computación e Inteligencia Artificial de la Universidad de Córdoba. Su labor ha sido especialmente importante para orientar el trabajo, revisar los avances y mantener el proyecto dentro de un enfoque académico y técnicamente coherente. Entre sus funciones principales destacan:

- Ayuda en la definición de los objetivos y del alcance del proyecto.
- Revisión periódica de los avances en el diseño y desarrollo de la herramienta.
- Propuesta de mejoras basadas en su experiencia académica y técnica.
- Resolución de dudas relacionadas con la implementación de funcionalidades y con las decisiones metodológicas del trabajo.

6.2. Recursos hardware

El proyecto se ha desarrollado en un ordenador personal con capacidad suficiente para ejecutar la aplicación web, procesar los conjuntos de datos y realizar los análisis estadísticos necesarios. Aunque no se trata de una infraestructura especialmente compleja, sí es importante contar con un equipo estable, ya que durante el desarrollo se ejecutan servicios locales, procesos de análisis, generación de gráficos y exportación de informes.

Las características principales del equipo utilizado son las siguientes:

- Equipo: Lenovo 82KU.
- Procesador: AMD Ryzen 7 5700U with Radeon Graphics.
- Memoria RAM: 8 GB.
- Sistema operativo: Windows 11 Home de 64 bits.
- Tarjeta gráfica: AMD Radeon Graphics integrada.

6.3. Recursos software

Para el desarrollo de la aplicación se han empleado distintos recursos software que forman la base tecnológica del sistema. El núcleo del proyecto está desarrollado en Python, mientras que la interfaz web y la interacción con el usuario se han implementado mediante Dash. Esta combinación permite construir un dashboard interactivo sin separar en exceso la parte de análisis de datos y la parte visual, algo especialmente útil en un proyecto de carácter académico y experimental.

Para la construcción de gráficos dinámicos y representaciones analíticas se utiliza Plotly. El procesamiento, limpieza y transformación de datos se realiza principalmente mediante Pandas y NumPy, dos bibliotecas fundamentales dentro del ecosistema de análisis de datos en Python. Además, para la parte estadística se emplean librerías especializadas como Lifelines, utilizada en modelos como Kaplan-Meier, Cox, Weibull y Exponencial, junto con Scikit-survival para la implementación de Random Survival Forest.

En la parte de presentación, la aplicación incorpora una hoja de estilos CSS personalizada, que permite ajustar el aspecto visual del dashboard y mejorar la experiencia de uso. Además, se utilizan ReportLab y Kaleido para la exportación de informes y gráficos en formato PDF, una funcionalidad importante porque facilita conservar y compartir los resultados obtenidos.

Como funcionalidad complementaria, el sistema incluye un módulo de interpretación asistida por inteligencia artificial ejecutado en entorno local. Para ello se utiliza un servidor compatible con una API de estilo OpenAI, mediante *llama-server*, junto con un modelo ligero de lenguaje configurado para generar explicaciones sobre los resultados estadísticos. En concreto, la configuración actual del proyecto utiliza el modelo *qwen2.5-1.5b-instruct*. Esta parte aporta una capa adicional de interpretación, de forma que los resultados no queden únicamente como tablas o gráficos, sino que puedan acompañarse de explicaciones más comprensibles.

También se han empleado herramientas de apoyo al desarrollo, como Visual Studio Code para la edición del código, Git para el control de versiones y scripts de arranque en Windows para facilitar la ejecución tanto de la aplicación como del servidor local de inteligencia artificial y Visual Paradigm para el diseño de los diagramas que le dan forma a los casos de uso.

6.4. Recursos de datos

Los datos utilizados proceden del conjunto de información heredado del Trabajo Fin de Grado previo, relacionado con el abandono académico universitario. Este conjunto de datos permite trabajar con información de estudiantes y con variables relevantes para el análisis de supervivencia, como el identificador del estudiante, el tiempo observado, el resultado final y distintos atributos académicos o personales.

Dentro del proyecto se emplean archivos de datos preparados para la carga y el análisis, como *dataset_limpio.csv* y *data/temp_data.csv*. Estos archivos permiten probar el flujo completo de la aplicación: carga, validación, preprocesamiento, análisis estadístico, visualización de resultados e interpretación posterior.

Un aspecto especialmente importante es la presencia de datos censurados. En este contexto, estos datos representan estudiantes para los que no se ha observado el abandono durante el periodo analizado. La verdad es que este punto es clave, porque el análisis de supervivencia no solo estudia a quienes abandonan, sino también a quienes permanecen en el sistema durante el tiempo de observación. Gracias a ello, el modelo puede aprovechar mejor la información disponible y ofrecer una visión más ajustada del fenómeno.

Por último, la documentación del proyecto se desarrolla mediante $\text{\LaTeX}$, utilizando la plantilla oficial proporcionada para la memoria. El código fuente de la aplicación, donde se incluyen los archivos utilizados para la carga, procesamiento, análisis de datos y generación de informes, está disponible en el repositorio de GitHub del proyecto [8].

7 Especificación de requisitos

En este capítulo se llevará a cabo un análisis más detallado y técnico de los requisitos que componen el sistema. Partiendo de los requisitos ya definidos en el TFG anterior, este trabajo mantiene las funcionalidades generales básicas del sistema, como la carga, limpieza y visualización de datos, así como la interacción con el asistente de inteligencia artificial. No obstante, los requisitos específicos asociados a los modelos clásicos ya implementados en el sistema base no se redefinen de forma individual, puesto que forman parte de la solución previa. A partir de dicha base bien fundamentada, este TFG incorpora nuevos requisitos orientados a la ampliación funcional y a la mejora de la capacidad analítica del dashboard.

7.1. Requisitos funcionales

- **RF1.** El sistema debe permitir la interacción del usuario con el asistente de inteligencia artificial integrado.
- **RF2.** El sistema debe incorporar soporte multilingüe en la interfaz, permitiendo al menos el uso en castellano e inglés.
- **RF3.** El sistema debe permitir trabajar con nuevas variables del conjunto de datos, además de las ya contempladas en el sistema anterior.
- **RF4.** El sistema debe admitir el análisis de variables no binarias dentro del flujo de estudio.
- **RF5.** El sistema debe ampliar las visualizaciones disponibles para mejorar la interpretación de los resultados en los métodos de regresión de Cox y Test de Log-Rank.
- **RF6.** El sistema debe incorporar nuevas técnicas de análisis de supervivencia que amplíen la capacidad analítica del dashboard, cada una con sus respectivas interpretaciones.
- **RF7.** El sistema debe incorporar el modelo exponencial como nueva técnica de análisis de supervivencia.
- **RF8.** El sistema debe incorporar el modelo de Weibull como nueva técnica de análisis de supervivencia.

CAPÍTULO 7. ESPECIFICACIÓN DE REQUISITOS

- **RF9.** El sistema debe incorporar Random Survival Forest como nueva técnica de análisis de supervivencia.
- **RF10.** El sistema debe permitir la comparación de resultados entre las distintas técnicas de análisis implementadas.
- **RF11.** El sistema debe incorporar una nueva solución de inteligencia artificial más rápida que la utilizada en el TFG anterior, manteniendo la generación de interpretaciones sobre los resultados estadísticos.
- **RF12.** El sistema debe mejorar la calidad de las explicaciones generadas por la inteligencia artificial a partir de los resultados estadísticos obtenidos.
- **RF13.** El sistema debe permitir la exportación de resultados, gráficas, tablas e informes en formato PDF para su uso en documentación o análisis posteriores.

7.2. Requisitos no funcionales

- **RNF1.** El sistema debe ofrecer tiempos de respuesta adecuados durante la carga, limpieza y visualización de los datos.
- **RNF2.** El sistema debe ejecutar los análisis en un tiempo razonable, incluso tras la incorporación de nuevas técnicas de análisis de supervivencia.
- **RNF3.** El sistema debe reducir el tiempo de respuesta del módulo de inteligencia artificial con respecto a la solución utilizada en el TFG anterior, teniendo en cuenta métricas como el tiempo de respuesta, el tiempo de generación y el consumo de recursos.
- **RNF4.** El sistema debe garantizar la confidencialidad de los datos académicos utilizados en el análisis.
- **RNF5.** El sistema debe validar los archivos cargados para evitar la incorporación de contenido malicioso o no válido.
- **RNF6.** El sistema debe ofrecer una interfaz clara, comprensible y fácil de utilizar para facilitar la navegación y la comprensión de la información mostrada.
- **RNF7.** El sistema debe mostrar correctamente las visualizaciones y resultados en los navegadores de uso habitual.
- **RNF8.** El sistema debe mantener la estabilidad de las funcionalidades ya implementadas en la versión previa.

CAPÍTULO 7. ESPECIFICACIÓN DE REQUISITOS

- **RNF9.** El sistema debe asegurar la coherencia entre los resultados estadísticos obtenidos, las gráficas generadas y las explicaciones producidas por la inteligencia artificial.
- **RNF10.** El sistema debe facilitar futuras ampliaciones funcionales sin necesidad de modificar por completo la estructura principal de la aplicación.
- **RNF11.** El sistema debe permitir un despliegue reproducible en entorno local con las dependencias necesarias correctamente definidas.

7.3. Requisitos de información

- **RI1.** El sistema debe gestionar un conjunto de datos estructurado que contenga información académica de los estudiantes, incluyendo variables relevantes para el análisis de supervivencia.
- **RI2.** El sistema debe permitir la incorporación de nuevas variables al conjunto de datos, incluyendo variables no binarias.
- **RI3.** El sistema debe almacenar y procesar correctamente las variables necesarias para la aplicación de modelos de análisis de supervivencia.
- **RI3.1.** El sistema debe identificar correctamente las variables mínimas necesarias para el análisis, incluyendo un identificador del estudiante, una variable temporal y una variable que represente el evento o resultado observado.
- **RI3.2.** El sistema debe representar correctamente la información asociada a datos censurados, diferenciando los casos en los que el abandono se ha producido de aquellos en los que no se ha observado durante el periodo de estudio.
- **RI4.** El sistema debe generar y mantener los resultados derivados de los distintos modelos aplicados, incluyendo métricas, estimaciones y representaciones gráficas.
- **RI5.** El sistema debe proporcionar información interpretativa sobre los resultados obtenidos, facilitando su comprensión mediante el módulo de inteligencia artificial.
- **RI6.** El sistema debe permitir la exportación de la información generada, incluyendo resultados, tablas y gráficos, en formatos reutilizables.

8 Análisis del problema

En este capítulo se presenta el análisis funcional del sistema desarrollado en el marco de este Trabajo de Fin de Grado. Al tratarse de una ampliación del dashboard desarrollado en un Trabajo de Fin de Grado previo, se mantienen sus características principales y se incorporan nuevas capacidades orientadas a mejorar el análisis, la comparación de técnicas y la interpretación de resultados.

El objetivo principal de esta ampliación es proporcionar una herramienta más flexible y completa para el estudio del abandono académico en el ámbito universitario. Para ello, se profundiza en los factores que influyen en este fenómeno mediante la incorporación de nuevas técnicas de análisis de supervivencia y funcionalidades complementarias.

El análisis parte de los requisitos funcionales, no funcionales y de información definidos en el capítulo anterior. A partir de ellos, se identifican los actores que interactúan con la aplicación y los casos de uso asociados a la ampliación propuesta. De este modo, las funcionalidades se describen desde la perspectiva del usuario y se representa su comportamiento mediante diagramas de casos de uso y diagramas de actividad, lo que facilita comprender el flujo de interacción entre los distintos componentes del sistema.

8.1. Identificación de actores

En esta primera fase de análisis del problema se mantienen los mismos actores principales que en el Trabajo de Fin de Grado anterior, ya que la aplicación está orientada al mismo tipo de público: profesorado, personal administrativo, alumnado o cualquier miembro interesado en utilizar estas técnicas a partir de los datos propuestos. Aunque la interacción principal recae sobre el usuario, también se considera el asistente de inteligencia artificial, encargado de procesar las consultas y generar respuestas o interpretaciones relacionadas con los resultados obtenidos.

- **Usuario:** Representa a cualquier persona que utiliza la aplicación para cargar datos, ejecutar análisis de supervivencia, visualizar resultados, comparar modelos y exportar la información generada por el sistema.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

- **Sistema de Inteligencia Artificial (IA):** Actúa como un componente externo que interactúa con el usuario proporcionando explicaciones, sugerencias e interpretaciones de los resultados obtenidos. En esta nueva versión del sistema, este actor adquiere especial relevancia debido a las mejoras introducidas en la calidad y rapidez de las respuestas generadas.

8.2. Identificación de casos de uso

Dado que este Trabajo Fin de Grado supone una ampliación y mejora del dashboard desarrollado previamente, no resulta necesario incluir en la lista de casos de uso las funcionalidades ya implementadas en el proyecto anterior. Tras el análisis de los objetivos generales del proyecto y la elicitación de requisitos correspondiente, en esta fase se identifican y documentan las principales acciones que los actores previamente definidos pueden llevar a cabo sobre el sistema.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Id	Nombre	Descripción
CU-1	EjecutarModeloExponencial	Permite al usuario aplicar el modelo exponencial como técnica de análisis de supervivencia para estudiar el tiempo hasta el evento.
CU-2	EjecutarModeloWeibull	Permite al usuario aplicar el modelo de Weibull para analizar el comportamiento del riesgo en función del tiempo mediante un enfoque paramétrico.
CU-3	EjecutarRandomSurvivalForest	Permite al usuario aplicar el modelo Random Survival Forest para realizar análisis de supervivencia basados en técnicas de aprendizaje automático.
CU-4	ConsultarComparaciónTécnicas	Permite al usuario acceder a una página específica del dashboard dedicada a la comparación de técnicas de análisis de supervivencia, mostrando una tabla comparativa, visualizaciones y un bloque explicativo automático.
CU-5	ExportarInformesYResultados	Permite al usuario exportar resultados, gráficas y tablas generadas por el sistema en formato reutilizable para documentación o análisis posteriores.
CU-6	CambiarIdiomaInterfaz	Permite al usuario seleccionar y cambiar el idioma de la interfaz del sistema, facilitando su uso en distintos contextos lingüísticos.
CU-7	AnalizarVariablesNoBinarias	Permite incorporar y analizar nuevas variables del conjunto de datos, incluyendo variables no binarias, ampliando las posibilidades de estudio del sistema.
CU-8	MejorarInterpretaciónIA	Permite generar explicaciones más precisas, coherentes y rápidas mediante la mejora del sistema de inteligencia artificial integrado.
CU-9	AmpliarVisualizacionesCoxLogRank	Permite generar nuevas visualizaciones asociadas a los modelos de regresión de Cox y Test de Log-Rank, mejorando la interpretación visual de los resultados.

Tabla 1: Identificación de casos de uso de la ampliación del sistema

Los casos de uso definidos se apoyan en un conjunto concreto de variables académicas y personales. La selección y justificación de estas variables se desarrolla

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

en el capítulo de diseño, dentro del apartado dedicado al diseño de los datos de análisis, ya que condiciona la estructura interna del sistema y la forma en la que los modelos reciben la información.

8.3. Diagrama de casos de uso

Tras la identificación de los casos de uso y los actores, se procede a la creación del diagrama de casos de uso. Este diagrama, elaborado mediante la herramienta Visual Paradigm, representa de forma visual las funcionalidades del sistema y las interacciones que los actores pueden realizar sobre él. Dicho diagrama se centra únicamente en los casos de uso asociados a la ampliación desarrollada, manteniendo implícitas las funcionalidades del sistema base.

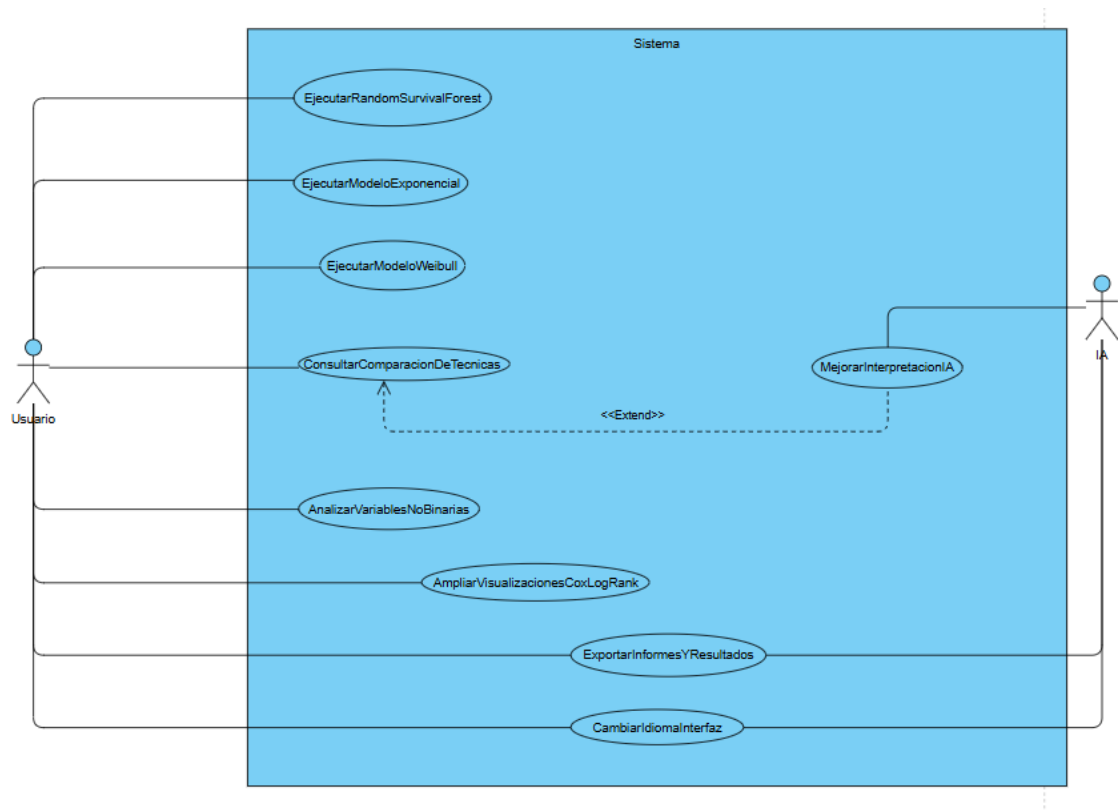


Figura 4: Diagrama de casos de uso de la ampliación del sistema

8.4. Especificación de los casos de uso

Una vez definidos los casos de uso, los actores del sistema y el diagrama general, se procede a la especificación de cada uno de ellos. En dicha especificación se describe con detalle cómo funciona cada funcionalidad desde el punto de vista del usuario, incluyendo condiciones iniciales, resultados esperados y posibles errores. Además, se identifican los actores implicados en cada caso de uso.

8.4.1. CU-1: EjecutarModeloExponencial

Tabla 2: Especificación del CU-1: Ejecutar modelo exponencial

Id	CU-1
Nombre	EjecutarModeloExponencial
Descripción	Este caso de uso describe el proceso mediante el cual el usuario aplica el modelo exponencial como técnica de análisis de supervivencia, con el objetivo de estudiar el tiempo hasta la ocurrencia del evento analizado.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	▪ El dataset debe haber sido cargado y preprocesado correctamente. ▪ El usuario debe haber accedido a la sección correspondiente al modelo exponencial.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Flujo principal	1. El caso de uso comienza cuando el usuario accede al módulo de Análisis de Supervivencia y selecciona el botón “Análisis Exponencial”. 2. El sistema carga la vista del modelo exponencial y verifica que exista un dataset previamente cargado y preprocesado. 3. El sistema detecta que hay un dataset disponible y calcula automáticamente el resultado del modelo. 4. El sistema muestra en pantalla las salidas del módulo: tabla resumen, gráfica y comparativa de ajuste. 5. El usuario decide una acción posterior: ▪ Pulsar botón “Explicar”. ▪ Pulsar botón “Exportar a PDF”. ▪ Pulsar botón “Exportar informe combinado”. 6. El sistema ejecuta la acción seleccionada y muestra la confirmación o resultado correspondiente.
Postcondiciones	El análisis mediante el modelo exponencial queda completado y los resultados quedan disponibles para su visualización, comparación o exportación posterior.
Flujos alternativos	FA1 (Ejecución del modelo): El sistema no puede ejecutar el modelo debido a datos insuficientes o incompatibles. El sistema muestra un mensaje de error indicando la imposibilidad de realizar el análisis.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.2. CU-2: EjecutarModeloWeibull

Tabla 3: Especificación del CU-2: Ejecutar modelo Weibull

Id	CU-2
Nombre	EjecutarModeloWeibull
Descripción	Este caso de uso describe el proceso mediante el cual el usuario aplica el modelo de Weibull como técnica de análisis de supervivencia, permitiendo analizar el comportamiento del riesgo a lo largo del tiempo mediante un enfoque paramétrico.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	▪ El dataset debe haber sido cargado y preprocesado correctamente. ▪ El usuario debe haber accedido a la sección correspondiente al modelo de Weibull.
Flujo principal	1. El caso de uso comienza cuando el usuario accede al módulo de Análisis de Supervivencia y selecciona el botón “Análisis Weibull”. 2. El sistema carga la vista del modelo de Weibull y verifica que exista un dataset previamente cargado y preprocesado. 3. El sistema detecta que hay un dataset disponible y calcula automáticamente el resultado del modelo. 4. El sistema muestra en pantalla las salidas del módulo: tabla resumen, gráfica y comparativa de ajuste. 5. El usuario decide una acción posterior: ▪ Pulsar botón “Explicar”. ▪ Pulsar botón “Exportar a PDF”. ▪ Pulsar botón “Exportar informe combinado”. 6. El sistema ejecuta la acción seleccionada y muestra la confirmación o resultado correspondiente.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Postcondiciones	Los resultados quedan disponibles para su visualización, comparación o exportación.
Flujos alternativos	FA1 (Ejecución del modelo): El sistema detecta un error en los datos o parámetros seleccionados. El sistema muestra un mensaje indicando la imposibilidad de ejecutar el modelo.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.3. CU-3: EjecutarRandomSurvivalForest

Tabla 4: Especificación del CU-3: Ejecutar Random Survival Forest

Id	CU-3
Nombre	EjecutarRandomSurvivalForest
Descripción	Este caso de uso describe el proceso mediante el cual el usuario aplica el modelo Random Survival Forest para realizar análisis de supervivencia mediante técnicas de aprendizaje automático.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	▪ El dataset debe haber sido cargado y preprocesado correctamente. ▪ El usuario debe haber accedido a la sección correspondiente a Random Survival Forest.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Flujo principal	1. El caso de uso comienza cuando el usuario accede al módulo de Análisis de Supervivencia y selecciona el botón “Random Survival Forest”. 2. El sistema detecta la ruta correspondiente y ejecuta automáticamente el análisis utilizando el dataset en sesión. 3. El sistema muestra en pantalla los resultados del modelo, incluyendo: ▪ Tabla resumen del modelo. ▪ Gráfica de curvas de supervivencia estimadas por niveles de riesgo. ▪ Gráfica de importancia de variables. 4. El sistema muestra el panel de simulación de perfil con valores por defecto (género, discapacidad, edad, educación, créditos) junto con una primera estimación del perfil. 5. El usuario revisa los resultados obtenidos y, si lo desea, modifica los valores del perfil. 6. El usuario puede pulsar el botón “Simular perfil” para recalcular y visualizar el detalle del perfil, incluyendo curva de supervivencia, interpretación y métricas asociadas. 7. El usuario decide una acción posterior: ▪ Pulsar el botón “Explicar”. ▪ Pulsar el botón “Exportar a PDF”. 8. El sistema ejecuta la acción seleccionada y muestra la confirmación o resultado correspondiente.
Postcondiciones	Los resultados quedan disponibles para su visualización, comparación o exportación.
Flujos alternativos	FA1 (Ejecución del modelo): El sistema no puede completar el entrenamiento o ejecución del modelo. El sistema informa al usuario mediante un mensaje de error.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.4. CU-4: ConsultarComparacionDeTecnicas

Tabla 5: Especificación del CU-4: Consultar comparación de técnicas

Id	CU-4
Nombre	ConsultarComparacionDeTecnicas
Descripción	Este caso de uso describe el proceso mediante el cual el usuario accede a una página específica del dashboard dedicada a la comparación de técnicas de análisis de supervivencia, con el objetivo de consultar información comparativa entre los distintos métodos implementados en el sistema.
Actor principal	Usuario
Actores secundarios	Sistema de Inteligencia Artificial (IA)
Precondiciones	▪ El usuario debe haber accedido previamente a la sección “Análisis de supervivencia” del dashboard. ▪ La página “Comparación de técnicas” debe estar integrada en el sistema de rutas de la aplicación. ▪ La opción de acceso a esta funcionalidad debe estar disponible como botón interno bajo las técnicas de análisis de supervivencia.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Flujo principal	1. El caso de uso comienza cuando el usuario entra en “Análisis de supervivencia” y pulsa el botón “Comparación de técnicas”. 2. El sistema valida que exista un dataset disponible en la sesión. 3. El sistema carga automáticamente una tabla comparativa de técnicas que incluye información como el tipo de modelo, hipótesis, ventajas, limitaciones y contexto de uso recomendado. 4. El sistema genera una visualización comparativa en forma de gráfico de barras, mostrando puntuaciones orientativas (capacidad predictiva, interpretabilidad y flexibilidad en una escala de 0 a 5). 5. El sistema genera un bloque explicativo textual mediante el módulo de inteligencia artificial, en el que se resumen las principales diferencias entre técnicas y se propone una estrategia metodológica de uso. 6. El usuario consulta la información mostrada y la utiliza como apoyo para el análisis e interpretación de los resultados.
Postcondiciones	La página de comparación de técnicas queda cargada correctamente y la información comparativa permanece disponible para su consulta, interpretación o posible exportación posterior.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Flujos alternativos	■ FA1 (Carga de la página): el sistema no puede cargar correctamente la página de comparación. En ese caso, muestra un mensaje de error al usuario indicando la imposibilidad de acceder a dicha funcionalidad. ■ FA2 (Generación de visualización comparativa): no es posible generar el gráfico comparativo debido a la falta de datos o a incompatibilidades entre las técnicas. El sistema mantiene visible la tabla comparativa y el bloque explicativo automático, pero omite la visualización gráfica. ■ FA3 (Generación del bloque explicativo): el sistema no puede generar el texto explicativo automático. El sistema mantiene visible la tabla comparativa y, en su caso, la visualización gráfica, mostrando un aviso al usuario.
----------------------------	---

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.5. CU-5: Exportar Informes Y Resultados

Tabla 6: Especificación del CU-5: Exportar informes y resultados

Id	CU-5
Nombre	Exportar Informes Y Resultados
Descripción	Este caso de uso describe el proceso mediante el cual el usuario exporta los resultados, gráficas y tablas generadas por el sistema en un formato reutilizable.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	■ Deben existir resultados generados previamente en el sistema. ■ El usuario debe haber accedido a la funcionalidad de exportación.
Flujo principal	1. El caso de uso comienza cuando el usuario pulsa el botón “Exportar a PDF” del módulo correspondiente. 2. El sistema abre un modal de exportación. 3. El usuario introduce, de forma opcional, el nombre del archivo y selecciona el contenido a incluir en el informe. 4. El usuario confirma la operación pulsando “Descargar PDF”. 5. El sistema valida que existan datos o resultados suficientes para realizar la exportación. 6. El sistema genera el informe en formato PDF con los bloques seleccionados (resumen, tabla, gráfica e interpretación, según el módulo). 7. El sistema entrega el archivo generado para su descarga.
Postcondiciones	El archivo de resultados queda generado correctamente y puede ser utilizado en documentación o análisis posteriores.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Flujos alternativos	FA1 (Generación del archivo): El sistema no puede generar el archivo. El sistema muestra un mensaje de error al usuario.
----------------------------	---

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.6. CU-6: CambiarIdiomaInterfaz

Tabla 7: Especificación del CU-6: Cambiar idioma de la interfaz

Id	CU-6
Nombre	CambiarIdiomaInterfaz
Descripción	Este caso de uso describe el proceso mediante el cual el usuario cambia el idioma de la interfaz del sistema.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	■ El sistema debe estar en ejecución. ■ Deben existir varios idiomas disponibles.
Flujo principal	1. El caso de uso comienza cuando el usuario accede al selector de idioma de la barra de navegación. 2. El sistema muestra los idiomas disponibles. 3. El usuario selecciona el idioma deseado. 4. El sistema procesa la selección y recarga los textos visibles de la interfaz en el idioma seleccionado. 5. Finalmente, la interfaz se muestra completamente actualizada en el idioma elegido.
Postcondiciones	El idioma de la interfaz queda actualizado correctamente.
Flujos alternativos	FA1 (Cambio de idioma): El sistema no puede aplicar el idioma seleccionado. El sistema mantiene el idioma anterior y muestra un mensaje de aviso al usuario.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.7. CU-7: Analizar Variables No Binarias

Tabla 8: Especificación del CU-7: Analizar variables no binarias

Id	CU-7
Nombre	Analizar Variables No Binarias
Descripción	Este caso de uso describe el proceso mediante el cual el usuario incorpora y analiza nuevas variables del conjunto de datos, incluyendo variables no binarias.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	■ El dataset debe haber sido cargado correctamente. ■ Deben existir variables adicionales o no binarias en el conjunto de datos.
Flujo principal	1. El caso de uso comienza cuando el usuario carga un archivo CSV y pulsa la opción “Preprocesar CSV”. 2. El sistema valida la estructura del dataset y conserva las variables no binarias soportadas por el pipeline. 3. El usuario accede al módulo de análisis de covariables o supervivencia y selecciona una variable no binaria disponible. 4. El sistema transforma la variable según la técnica aplicada (por ejemplo, agrupación por categorías o discretización por cuantiles en variables como créditos). 5. El sistema ejecuta el análisis correspondiente utilizando las variables seleccionadas. 6. El sistema muestra en pantalla los resultados actualizados, incluyendo tablas, métricas y representaciones gráficas. 7. El usuario consulta los resultados y los utiliza para interpretar el efecto de las variables analizadas.
Postcondiciones	Las nuevas variables quedan integradas en el análisis y los resultados quedan disponibles para su interpretación.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

Flujos alternativos	FA1 (Validación de variables): El sistema detecta que alguna variable no es compatible con el análisis seleccionado. El sistema informa al usuario mediante un mensaje de error.
----------------------------	---

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.8. CU-8: Mejorar Interpretación IA

Tabla 9: Especificación del CU-8: Mejorar interpretación mediante IA

Id	CU-8
Nombre	Mejorar Interpretación IA
Descripción	Este caso de uso describe el proceso mediante el cual el sistema genera explicaciones más precisas, coherentes y rápidas mediante una mejora del módulo de inteligencia artificial.
Actor principal	Sistema de Inteligencia Artificial (IA)
Actores secundarios	Usuario
Precondiciones	■ Deben existir resultados generados previamente en el sistema. ■ El módulo de inteligencia artificial debe estar disponible.
Flujo principal	1. El caso de uso comienza cuando el usuario solicita una interpretación de los resultados obtenidos. 2. El sistema recopila la información relevante del análisis ejecutado. 3. El sistema envía dicha información al módulo de inteligencia artificial. 4. El módulo de inteligencia artificial procesa los datos y genera una explicación. 5. El sistema recibe la respuesta generada y la presenta en la interfaz para su consulta por parte del usuario.
Postcondiciones	La explicación generada queda disponible para el usuario, facilitando la interpretación de los resultados.
Flujos alternativos	FA1 (Generación de respuesta): La inteligencia artificial no puede generar una respuesta adecuada. El sistema muestra un mensaje de error al usuario.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.4.9. CU-9: Ampliar Visualizaciones Cox Test Log Rank

Tabla 10: Especificación del CU-9: Ampliar visualizaciones Cox y Test de Log-Rank

Id	CU-9
Nombre	Ampliar Visualizaciones Cox Test Log Rank
Descripción	Este caso de uso describe el proceso mediante el cual el sistema genera nuevas visualizaciones asociadas a los modelos de regresión de Cox y Test de Log-Rank.
Actor principal	Usuario
Actores secundarios	Sistema
Precondiciones	■ Deben haberse ejecutado previamente análisis mediante regresión de Cox o Test de Log-Rank. ■ Los resultados correspondientes deben estar disponibles en el sistema.
Flujo principal	1. El caso de uso comienza cuando el usuario accede al módulo de análisis de supervivencia y selecciona “Regresión de Cox” o “Test de Log-Rank”. 2. El sistema muestra las opciones de visualización disponibles. 3. El usuario selecciona la visualización deseada. 4. El sistema obtiene los resultados del análisis correspondientes a la configuración activa. 5. El sistema procesa la información y genera la representación gráfica solicitada. 6. El sistema muestra la visualización en pantalla para su consulta por parte del usuario.
Postcondiciones	La visualización queda generada correctamente y disponible para su interpretación o exportación.
Flujos alternativos	FA1 (Disponibilidad de datos): No existen resultados previos de Cox o Test de Log-Rank. El sistema informa al usuario de que debe ejecutar previamente dichos análisis.

8.5. Matriz de trazabilidad

La siguiente matriz muestra la relación entre los Requisitos Funcionales (RF) definidos para el sistema y los Casos de Uso (CU) que los implementan. De este modo se garantiza la cobertura de todos los requisitos dentro del diseño y desarrollo de la aplicación.

Req	CU-1	CU-2	CU-3	CU-4	CU-5	CU-6	CU-7	CU-8	CU-9
RF1				X				X	
RF2						X			
RF3							X		
RF4							X		
RF5									X
RF6	X	X	X						
RF7	X								
RF8		X							
RF9			X						
RF10				X					
RF11								X	
RF12								X	
RF13	X	X	X		X				

Tabla 11: Matriz de trazabilidad entre requisitos funcionales y casos de uso

Como puede observarse en la matriz, todos los requisitos funcionales definidos para la ampliación del sistema quedan cubiertos por al menos un caso de uso. Esto permite comprobar que no existen requisitos funcionales aislados sin representación en el análisis. En el caso concreto de RF13, la trazabilidad se asocia al caso de uso general de exportación y también a los modelos Exponencial, Weibull y Random Survival Forest, ya que estas vistas incorporan acciones específicas para generar informes o exportar resultados. Asimismo, cada caso de uso se relaciona con una o varias necesidades concretas del sistema, lo que facilita mantener la trazabilidad entre los objetivos planteados, los requisitos especificados y las funcionalidades que finalmente se desarrollan en la aplicación.

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.6. Diagramas de actividad

8.6.1. CU-1: EjecutarModeloExponencial

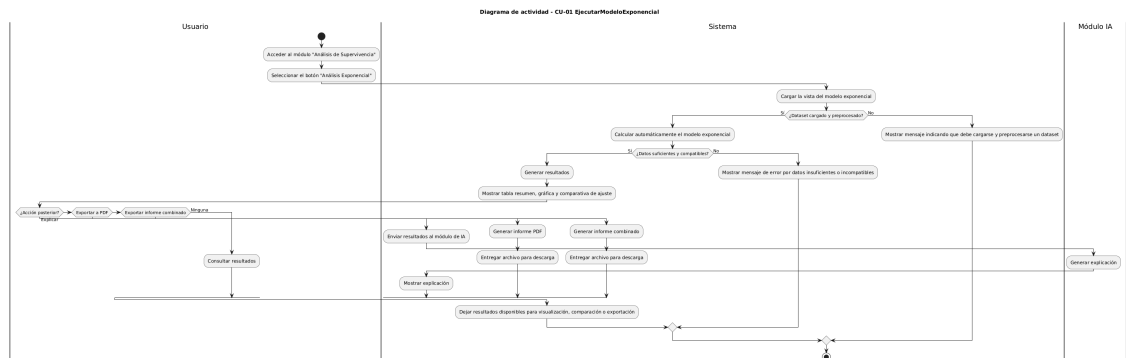


Figura 5: Diagrama de actividad del CU-2: EjecutarModeloExponencial

8.6.2. CU-2: EjecutarModeloWeibull

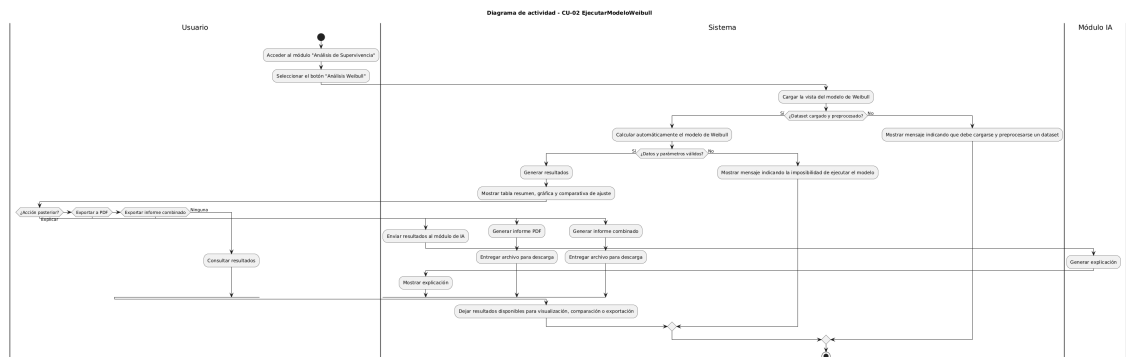


Figura 6: Diagrama de actividad del CU-2: EjecutarModeloWeibull

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.6.3. CU-3: EjecutarRandomSurvivalForest

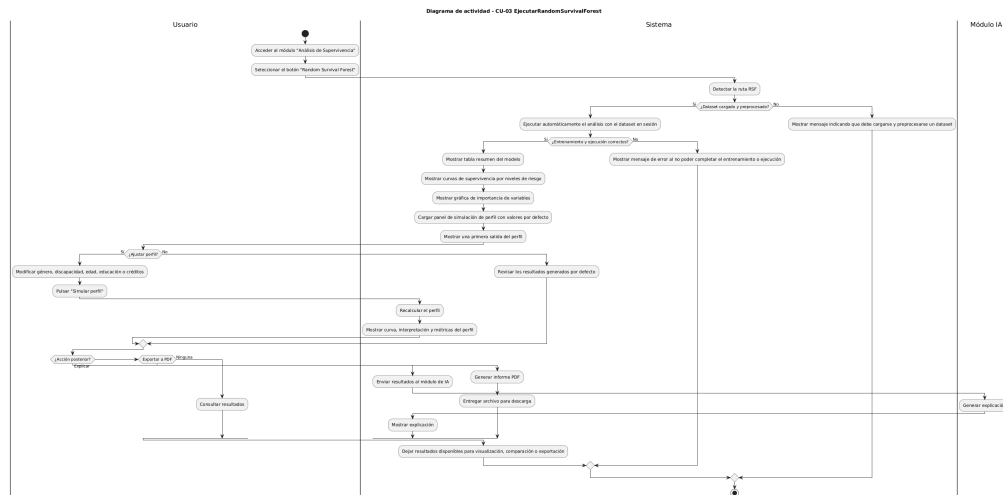


Figura 7: Diagrama de actividad del CU-3: EjecutarRandomSurvivalForest

8.6.4. CU-4: ConsultarComparacionDeTecnica

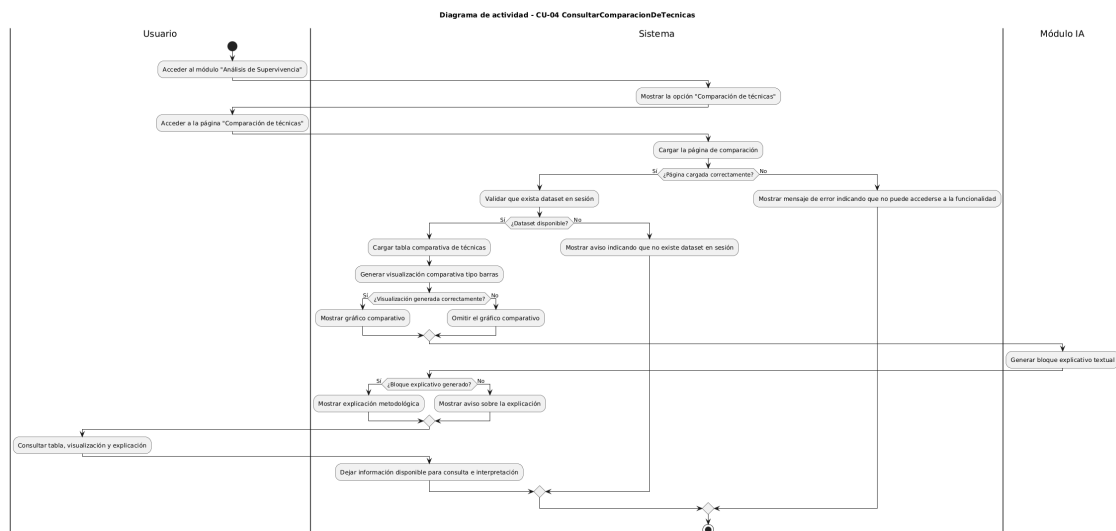


Figura 8: Diagrama de actividad del CU-4: ConsultarComparacionDeTecnica

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.6.5. CU-5: Exportar Informes Y Resultados

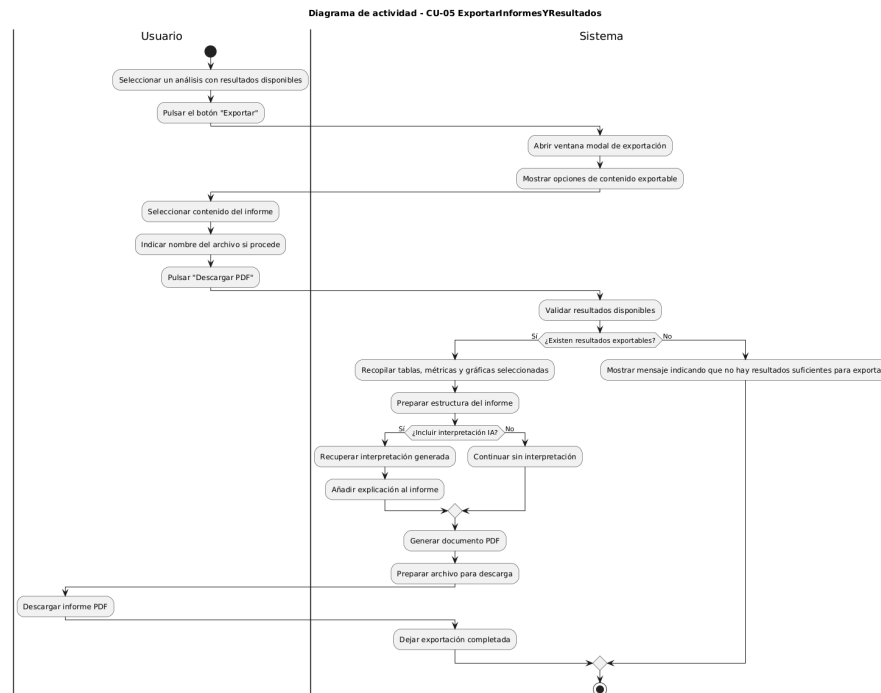


Figura 9: Diagrama de actividad del CU-5: Exportar Informes Y Resultados

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.6.6. CU-6: CambiarIdiomaInterfaz

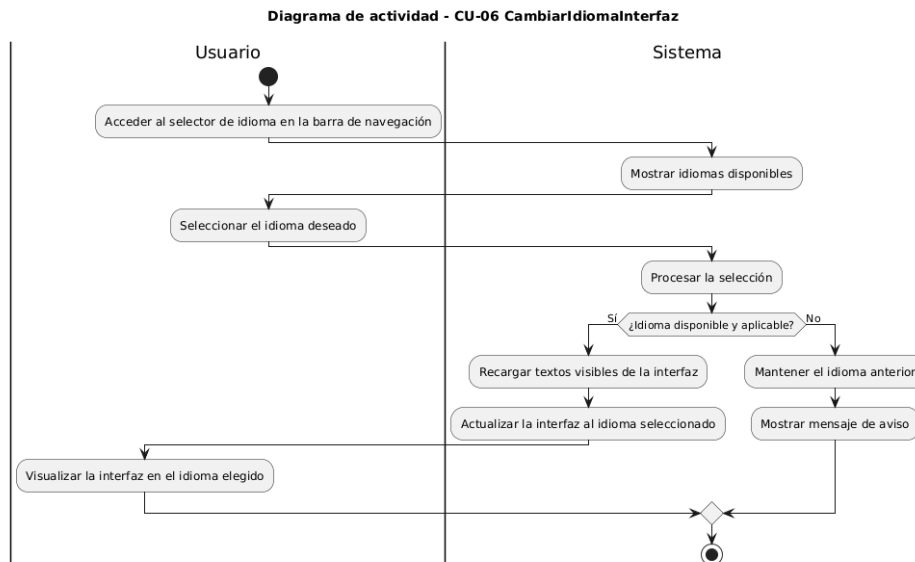


Figura 10: Diagrama de actividad del CU-6: CambiarIdiomaInterfaz

8.6.7. CU-7: AnalizarVariablesNoBinarias

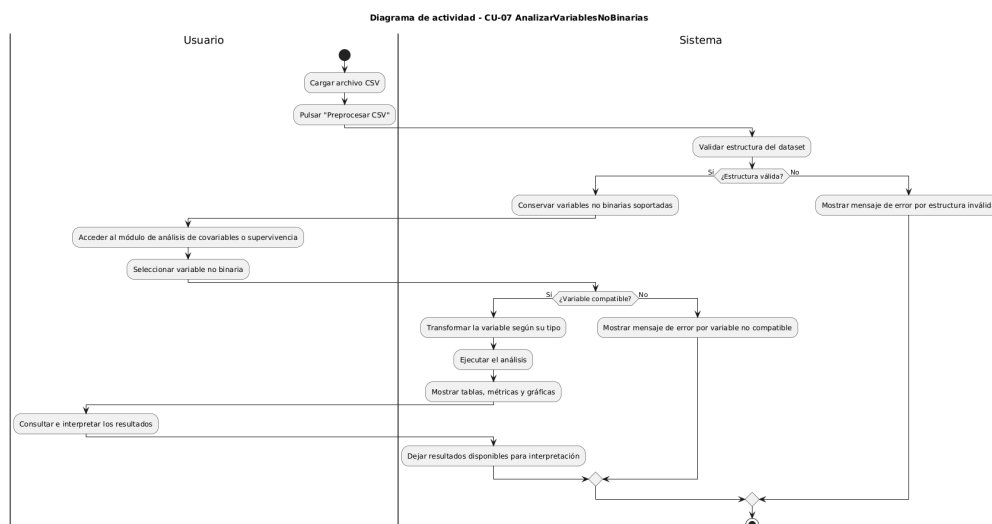


Figura 11: Diagrama de actividad del CU-7: AnalizarVariablesNoBinarias

CAPÍTULO 8. ANÁLISIS DEL PROBLEMA

8.6.8. CU-8: Mejorar Interpretación IA

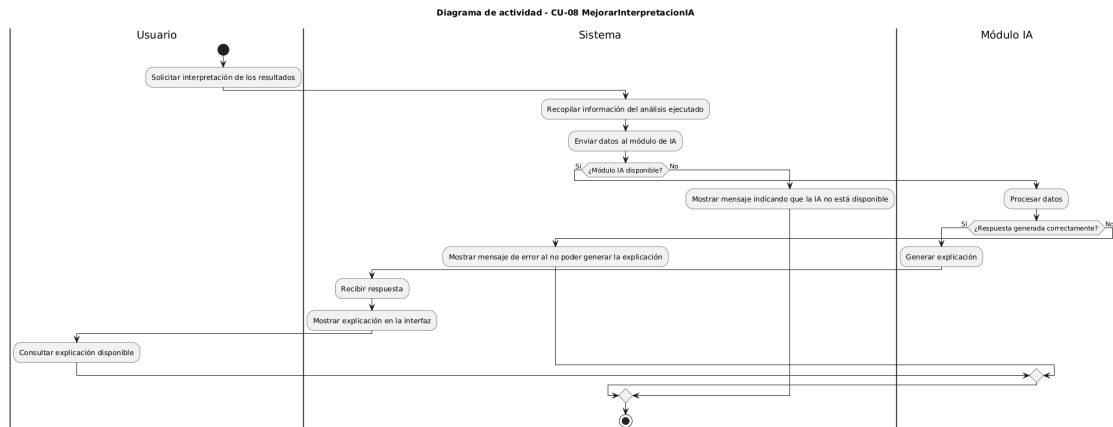


Figura 12: Diagrama de actividad del CU-8: Mejorar Interpretación IA

8.6.9. CU-9: Ampliar Visualizaciones Cox Test Log Rank

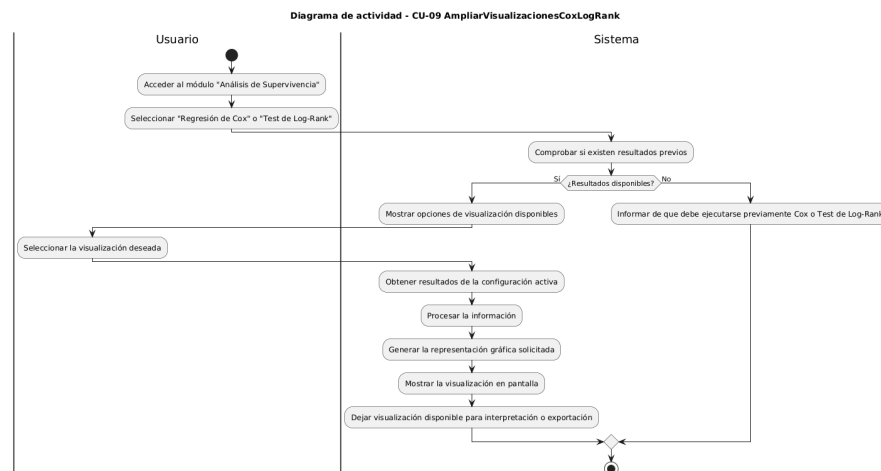


Figura 13: Diagrama de actividad del CU-9: Ampliar Visualizaciones Cox Test Log Rank

9 Diseño del sistema

La etapa de diseño es una fase clave dentro del desarrollo de la aplicación, ya que permite transformar los requisitos definidos anteriormente en una solución técnica concreta [9]. En este punto el proyecto deja de ser solo una lista de necesidades y empieza a tomar forma: se decide cómo se organiza el sistema, cómo se comunican sus partes y de qué manera el usuario va a interactuar con los resultados.

En este Trabajo Fin de Grado, además, el diseño tiene una particularidad importante. La aplicación no se construye desde cero, sino que parte de un dashboard previo que ya ofrecía funcionalidades de carga de datos, preprocesamiento y análisis de supervivencia. Por tanto, la solución diseñada se plantea como una ampliación ordenada del sistema existente. La idea no es romper la base anterior, sino aprovecharla y hacerla crecer con nuevos modelos, nuevas visualizaciones, soporte multilingüe, exportación de informes e interpretación asistida por inteligencia artificial.

9.1. Arquitectura del sistema

La arquitectura del sistema se organiza siguiendo una estructura modular. Esta decisión resulta bastante natural para una aplicación desarrollada en Python y Dash, ya que permite separar la interfaz, la lógica de análisis, la generación de informes y los servicios auxiliares. Gracias a esta división, cada parte del sistema tiene una responsabilidad concreta y es más sencillo mantener el proyecto, localizar errores o incorporar nuevas técnicas de análisis en el futuro.

Desde un punto de vista general, el sistema puede entenderse como una aplicación web local compuesta por varias capas. En primer lugar, se encuentra la capa de presentación, construida con Dash, HTML y componentes interactivos. Esta capa es la que ve el usuario: menús, formularios, botones, tablas, gráficas y modales de exportación. En segundo lugar, aparece la capa de control, formada principalmente por los callbacks de Dash, que conectan las acciones del usuario con la lógica interna de la aplicación. Por último, se encuentra la capa de procesamiento, donde se ejecutan los modelos estadísticos, las transformaciones de datos, la generación de visualizaciones, la creación de PDFs y las peticiones al módulo de inteligencia artificial.

La separación por capas se completa con una distribución por componentes. En lugar de concentrar toda la lógica en una única parte de la aplicación, el diseño distingue entre navegación, presentación visual, gestión de eventos, preparación de datos, ejecución

CAPÍTULO 9. DISEÑO DEL SISTEMA

de modelos, visualización de resultados, generación de informes, traducción de textos e interpretación mediante inteligencia artificial. Esta visión resulta más clara para entender el sistema en conjunto, mientras que los archivos concretos que materializan cada responsabilidad se detallan más adelante en el capítulo de implementación.

Esta organización también facilita la ampliación del dashboard. Si se incorpora una nueva técnica de supervivencia, no es necesario replantear toda la aplicación: basta con integrarla dentro del componente de análisis, conectarla con la interfaz y, si procede, añadir sus visualizaciones y su exportación. La verdad es que esta idea es importante, porque el proyecto parte de un sistema previo y necesita crecer sin perder estabilidad ni coherencia.

Componente de diseño	Responsabilidad principal
Interfaz de usuario	Presenta las pantallas, controles, tablas, gráficas, botones y modales con los que interactúa el usuario.
Gestor de navegación	Organiza el acceso a las distintas secciones del dashboard y mantiene un recorrido coherente entre carga, análisis, comparación y exportación.
Gestor de interacción	Recibe las acciones del usuario y coordina la respuesta del sistema, conectando la interfaz con los procesos internos.
Gestor de datos	Controla la carga, validación, limpieza y preparación del dataset antes de ejecutar los modelos.
Motor de análisis	Agrupar las técnicas estadísticas y predictivas utilizadas para estudiar la supervivencia académica.
Generador de visualizaciones	Transforma los resultados de los modelos en gráficas, curvas, tablas y comparaciones interpretables.
Generador de informes	Prepara la exportación de resultados en documentos reutilizables, incorporando tablas, gráficos e interpretaciones.
Módulo de inteligencia artificial	Produce explicaciones textuales de apoyo a partir de los resultados obtenidos en los análisis.
Sistema de internacionalización	Gestiona los textos de la interfaz y permite adaptar la aplicación a castellano e inglés.

Tabla 12: Componentes principales de la arquitectura del sistema

9.2. Diseño de los datos de análisis

La ampliación del sistema se apoya en un conjunto de variables que permiten estudiar el abandono académico desde una perspectiva temporal y, además, relacionarlo con características concretas del estudiante. Algunas de ellas son imprescindibles para cualquier modelo de supervivencia, mientras que otras actúan como covariables y permiten comparar perfiles o alimentar modelos más complejos, como la regresión de Cox, el Test de Log-Rank o Random Survival Forest.

La selección de estas variables no se ha realizado de forma arbitraria. Se han priorizado aquellas que cumplen tres criterios principales: que estén disponibles de forma estable en el conjunto de datos, que tengan una interpretación clara dentro del contexto académico y que puedan aportar información útil sobre la permanencia o el abandono del estudiante. También se ha buscado mantener un equilibrio razonable entre riqueza analítica y claridad, ya que incluir demasiadas variables puede hacer que los resultados sean más difíciles de explicar.

Variable o grupo	Qué representa	Motivo de selección
Identificador del estudiante <code>id_student</code>	Identifica de forma única a cada estudiante dentro del conjunto de datos.	Se utiliza para consolidar registros y evitar duplicidades durante el preprocesamiento. No es una covariable explicativa, pero resulta necesaria para construir un dataset limpio y coherente.
Tiempo observado <code>date</code>	Indica el tiempo de seguimiento utilizado por los modelos de supervivencia.	Es una variable imprescindible, ya que el objetivo del análisis no es solo saber si se produce el abandono, sino estudiar cuándo ocurre.
Evento de abandono <code>final_result</code>	Recoge el resultado final del estudiante y permite identificar si se produce el evento de abandono.	Es la variable que define el evento analizado. Se transforma en una variable binaria para diferenciar abandono y no abandono dentro de los modelos.
Género <code>gender_F</code>	Representa el género del estudiante mediante una codificación binaria.	Se incluye porque permite analizar si existen diferencias de permanencia entre grupos. Además, es una variable sencilla de interpretar y útil para segmentar resultados.

CAPÍTULO 9. DISEÑO DEL SISTEMA

Variable o grupo	Qué representa	Motivo de selección
Discapacidad disability_N	Indica si el estudiante aparece registrado con o sin discapacidad.	Se selecciona por su relevancia en el contexto educativo, ya que puede estar relacionada con necesidades de apoyo, accesibilidad o condiciones que influyen en la trayectoria académica.
Tramo de edad age_band	Agrupar al alumnado en rangos de edad: 0-35, 35-55 y 55<=.	Se incluye porque la edad puede reflejar perfiles académicos distintos: estudiantes jóvenes, personas que retoman estudios o alumnado con otras responsabilidades. Esto permite comparar curvas y riesgos entre grupos.
Nivel educativo previo highest_education	Describe la formación máxima alcanzada antes de acceder al curso o programa analizado.	Se considera relevante porque la preparación académica previa puede influir en la adaptación al entorno universitario y, por tanto, en la probabilidad de permanencia o abandono.
Créditos estudiados studied_credits	Refleja la carga académica asumida por el estudiante.	Se incorpora porque permite estudiar si la dedicación académica o el volumen de créditos se relaciona con el abandono. Además, es una variable no binaria, por lo que ayuda a cumplir el objetivo de ampliar el análisis más allá de variables simples.

Tabla 13: Variables principales utilizadas y justificación de su selección

Aunque el conjunto de datos puede contener más atributos procedentes del sistema original, como actividad en recursos, módulos, presentaciones, regiones o intentos previos, esta ampliación se centra especialmente en las variables anteriores. La razón es que permiten cubrir los objetivos del trabajo sin perder claridad: se conserva la información mínima necesaria para los modelos de supervivencia y se incorporan covariables personales y académicas que enriquecen la interpretación de los resultados. Otras variables podrían ser útiles en futuros análisis, pero en esta versión se ha optado por un conjunto más controlado, comprensible y directamente relacionado con los casos de uso definidos.

9.3. Diseño de la interfaz

El diseño de la interfaz mantiene la idea principal del dashboard original: ofrecer al usuario un entorno visual desde el que pueda cargar datos, ejecutar análisis y consultar resultados sin tener que trabajar directamente con código. La verdad es que este punto es importante, porque el sistema está pensado para facilitar el análisis de supervivencia en un contexto académico, donde no siempre se parte de un perfil técnico avanzado.

La navegación se estructura mediante diferentes páginas o secciones. Cada una agrupa funcionalidades relacionadas: carga y visualización del dataset, análisis de covariables, técnicas de supervivencia, comparación de modelos y exportación de resultados. Esta organización evita mezclar demasiada información en una sola pantalla y permite que el usuario avance poco a poco, como si siguiera un pequeño recorrido: primero carga los datos, después los prepara, luego ejecuta modelos y finalmente interpreta o exporta los resultados.

En las páginas de análisis se ha seguido un patrón común. Primero se presenta la técnica seleccionada, después se muestran los controles necesarios y, finalmente, se generan tablas, métricas, gráficas e interpretaciones. Este patrón se repite en modelos como Exponencial, Weibull y Random Survival Forest, lo que aporta coherencia al uso de la aplicación. Cuando el usuario aprende cómo funciona una sección, puede moverse por las demás con menos esfuerzo.

También se ha cuidado el diseño de los modales de exportación. Estos permiten seleccionar qué contenido se desea incluir en el informe, como tablas, gráficos o explicaciones generadas por IA. De esta forma, la exportación no se limita a descargar una captura rígida de la pantalla, sino que ofrece cierto control sobre el resultado final. Además, la interfaz incorpora soporte multilingüe, permitiendo alternar entre castellano e inglés y adaptando los textos visibles de la aplicación al idioma seleccionado.

9.3.1. Interfaz de usuario (UI)

A continuación se muestran las principales pantallas de la interfaz desarrollada. Estas capturas permiten ver el recorrido básico que sigue el usuario dentro del dashboard: desde la entrada inicial al sistema, pasando por la carga y limpieza del archivo de datos, hasta llegar a las secciones de análisis de covariables y análisis de supervivencia.

CAPÍTULO 9. DISEÑO DEL SISTEMA

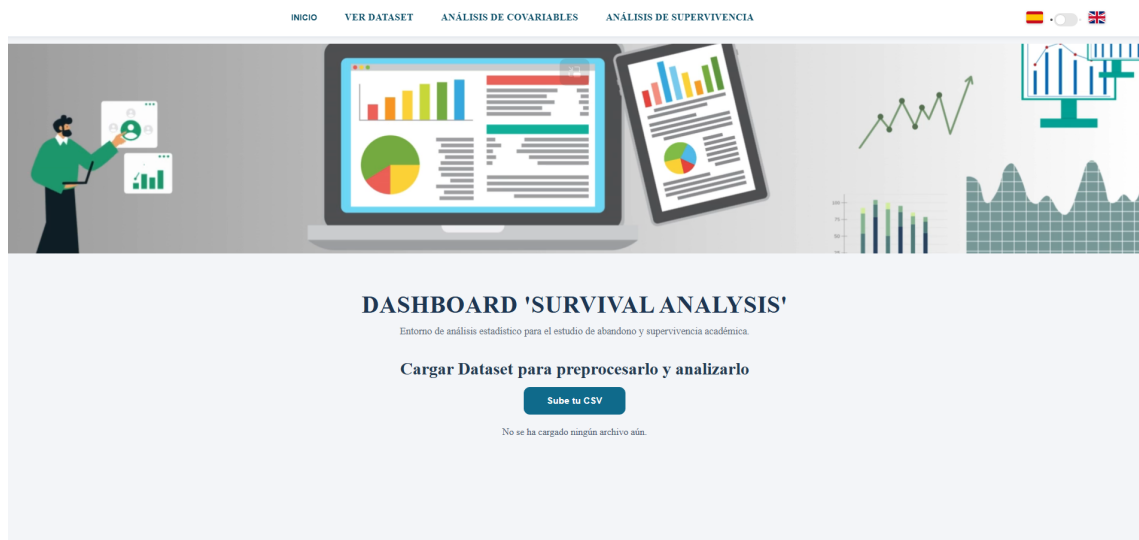


Figura 14: Pantalla principal del dashboard

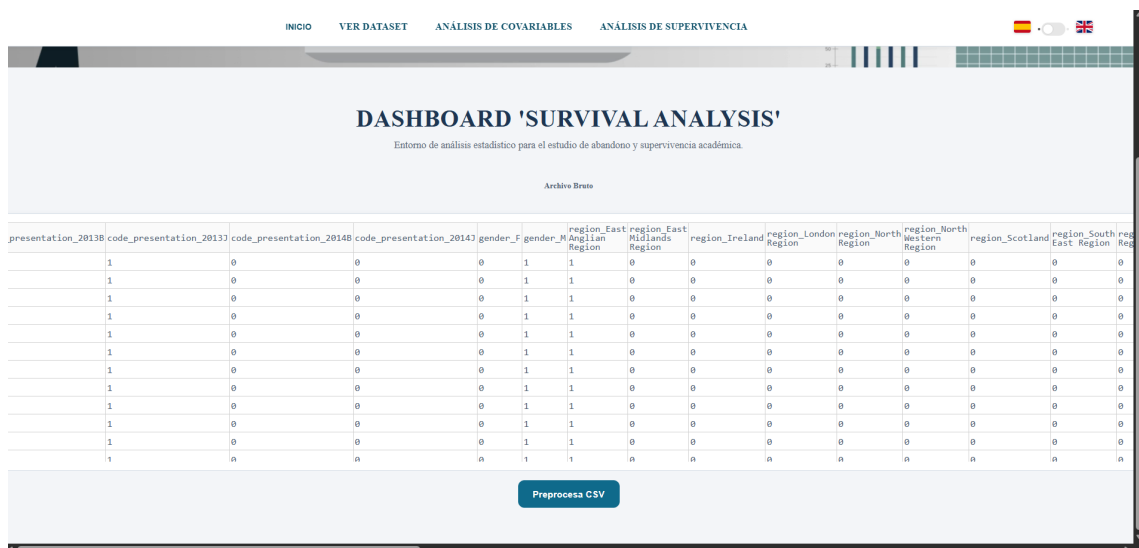


Figura 15: Visualización del archivo bruto cargado en el sistema

CAPÍTULO 9. DISEÑO DEL SISTEMA

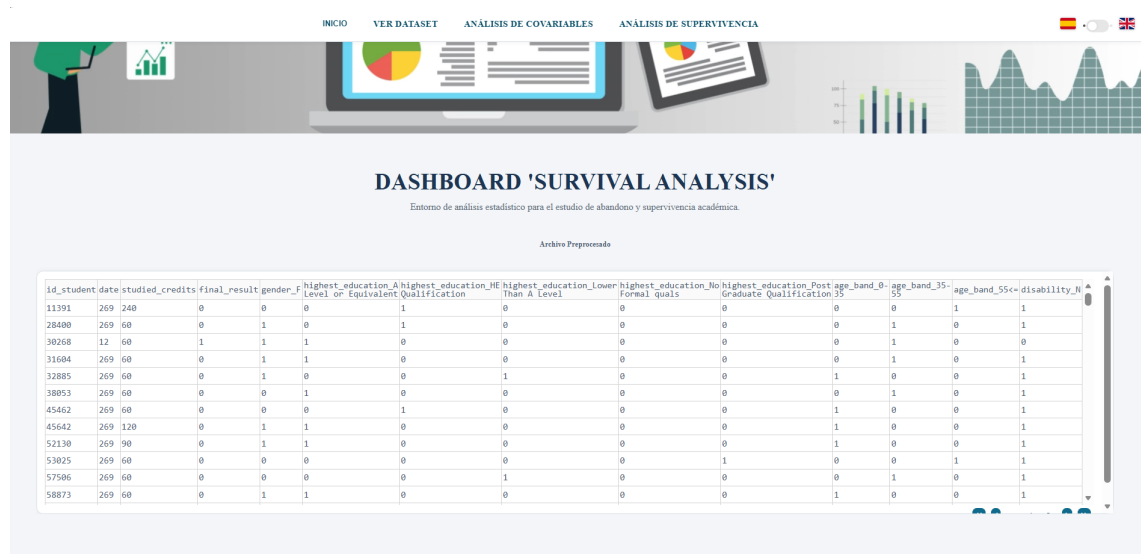


Figura 16: Visualización del archivo limpio tras el preprocesamiento

Ver dataset

La sección de visualización del dataset permite revisar los datos cargados antes y después del proceso de limpieza. Esta parte resulta útil para que el usuario pueda comprobar, de un vistazo, qué información se está utilizando realmente en los análisis posteriores.

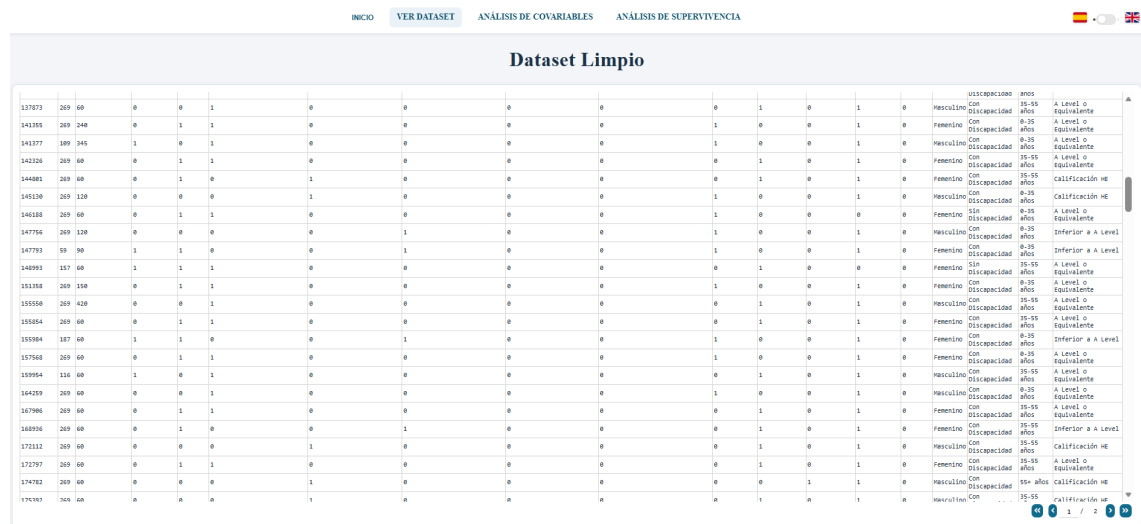


Figura 17: Vista inicial de la sección Ver dataset

CAPÍTULO 9. DISEÑO DEL SISTEMA

Variable	Valores	Significado
Evento (final_result)	0 o 1	Representa si ocurrió el evento en el análisis de supervivencia: 1 = abandono (evento), 0 = censura (no abandono durante el seguimiento).
id_student	Entero (ID)	Identificador único del estudiante.
date	Númeroico >= 0	Tiempo de seguimiento usado en supervivencia (por ejemplo, días).
final_result	0 o 1	Indicador de evento: 1 = abandono (evento), 0 = censurado sin abandono.
gender_F	0 o 1	Género codificado: 1 = femenino, 0 = masculino.
disability_N	0 o 1	Discapacidad codificada: 1 = con discapacidad, 0 = sin discapacidad.
age_band_0-35	0 o 1	Indicador one-hot del grupo de edad 0-35.
age_band_35-55	0 o 1	Indicador one-hot del grupo de edad 35-55.
age_band_55+	0 o 1	Indicador one-hot del grupo de edad 55+.
highest_education_A Level or Equivalent	0 o 1	Indicador one-hot de nivel educativo A Level o equivalente.
highest_education_HE Qualification	0 o 1	Indicador one-hot de titulación HE Qualification.
highest_education_Lower Than A Level	0 o 1	Indicador one-hot de nivel educativo inferior a A Level.
highest_education_No Formal qual	0 o 1	Indicador one-hot de ausencia de cualificaciones formales.
highest_education_Post Graduate Qualification	0 o 1	Indicador one-hot de estudios de posgrado.
studied_credits	Númeroico	Créditos cursados por el estudiante (covariable numérica).

Figura 18: Detalle de la visualización del dataset dentro del dashboard

Análisis de covariables

El análisis de covariables permite explorar la relación entre las variables disponibles y el evento de interés. Esta pantalla sirve como apoyo previo al análisis de supervivencia, ya que ayuda a detectar qué variables pueden aportar información relevante antes de aplicar los modelos estadísticos.

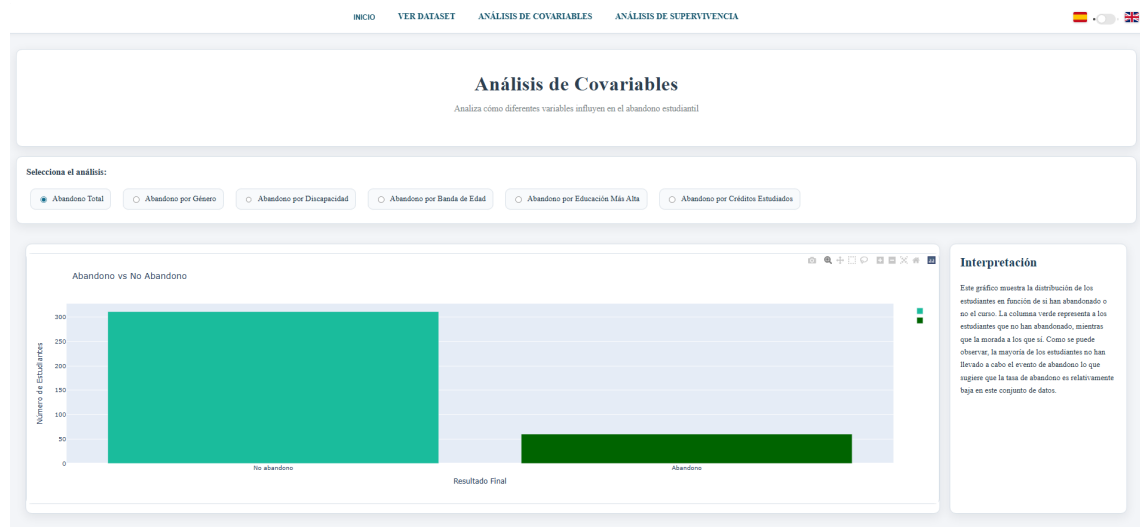


Figura 19: Sección de análisis de covariables

Análisis de supervivencia

La sección de análisis de supervivencia actúa como punto de entrada a las distintas técnicas disponibles en el sistema. Desde esta pantalla el usuario puede acceder a los métodos clásicos, a las nuevas técnicas incorporadas y a la comparación general

CAPÍTULO 9. DISEÑO DEL SISTEMA

entre métodos mediante un botón ancho situado bajo el conjunto de técnicas. De este modo, la navegación se mantiene centralizada y fácil de seguir.



Figura 20: Sección de análisis de supervivencia

Kaplan-Meier

Dentro del análisis de supervivencia, la sección de Kaplan-Meier permite consultar la curva de supervivencia general, aplicar filtros sobre los datos y exportar los resultados obtenidos. Es una de las vistas más importantes del sistema, ya que ofrece una primera lectura visual de la evolución de la supervivencia en el conjunto de datos.

CAPÍTULO 9. DISEÑO DEL SISTEMA



Figura 21: Vista general del análisis Kaplan-Meier



Figura 22: Curva de supervivencia generada mediante Kaplan-Meier

Generar Informe PDF



Haz clic y selecciona qué quieres incluir en el PDF:

Contenido a incluir:

- ☐ Resumen general
- ☐ Gráfica principal
- ☐ Interpretación automática IA

Nombre del archivo:

No incluir extensión .pdf

Cancelar

Descargar PDF

Figura 23: Filtros disponibles en el análisis Kaplan-Meier

 **Generar Informe PDF** ×

Haz clic y selecciona qué quieres incluir en el PDF:

 **Contenido a incluir:**

☐ Resumen general

☐ Gráfica principal

☐ Interpretación automática IA

 **Nombre del archivo:**

No incluir extensión .pdf

Cancelar

Descargar PDF

Figura 24: Exportación a PDF de los resultados de Kaplan-Meier

Regresión de Cox

La vista de regresión de Cox permite analizar la influencia de distintas covariables sobre el tiempo hasta el evento. Además de mostrar los resultados del modelo, la interfaz incorpora filtros, gráficas asociadas y la opción de exportar la información generada.

CAPÍTULO 9. DISEÑO DEL SISTEMA



Figura 25: Vista principal de la regresión de Cox

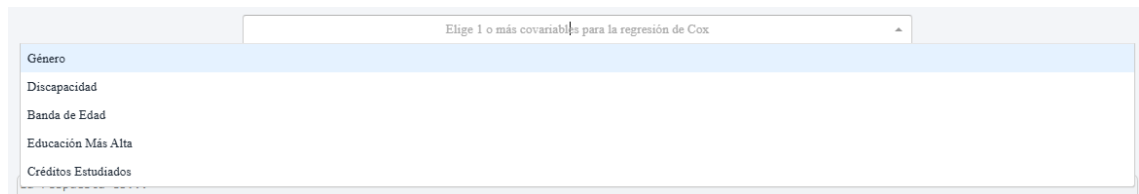


Figura 26: Filtros utilizados en la regresión de Cox



Figura 27: Gráfica generada a partir del modelo de regresión de Cox

CAPÍTULO 9. DISEÑO DEL SISTEMA

Generar Informe PDF X

Haz clic y selecciona qué quieres incluir en el PDF:

Contenido a incluir:

- ☐ Resumen general
- ☐ Gráfica principal
- ☐ Tabla de resultados
- ☐ Interpretación automática IA

Nombre del archivo:
No incluir extensión .pdf

Cancelar Descargar PDF

Figura 28: Botón de exportación a PDF utilizado en Cox y también en el Test de Log-Rank

Test de Log-Rank

El Test de Log-Rank se utiliza para comparar curvas de supervivencia entre grupos. En la aplicación comparte parte de la estructura visual con la regresión de Cox, especialmente en los filtros, pero orienta la salida a la comparación estadística entre categorías.

INICIO VER DATASET ANÁLISIS DE COVARIABLES ANÁLISIS DE SUPERVIVENCIA COMPARACIÓN DE TÉCNICAS 🇪🇸 🇬🇧

Análisis de Supervivencia: Test de Log-Rank

Aplicar Exportar a PDF

La respuesta es...

Figura 29: Vista principal del Test de Log-Rank

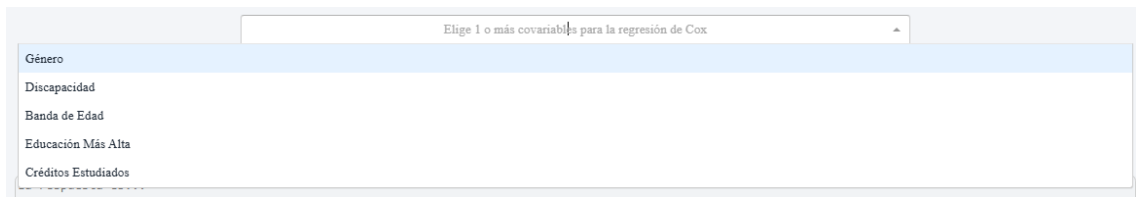


Figura 30: Filtros compartidos por Cox y el Test de Log-Rank

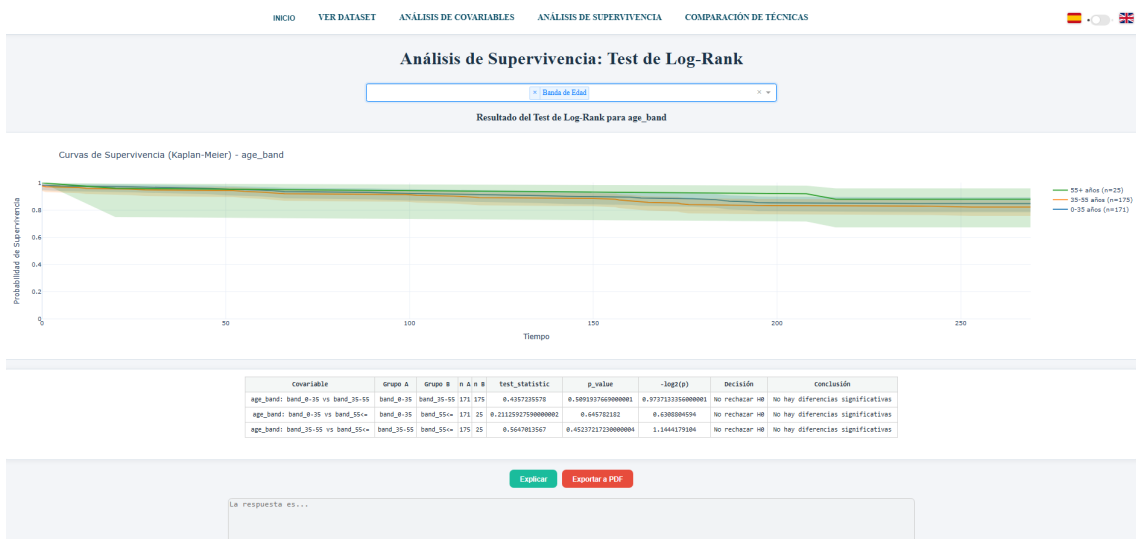


Figura 31: Ejemplo de resultado generado mediante el Test de Log-Rank

Weibull

La técnica de Weibull amplía el análisis de supervivencia con un modelo paramétrico capaz de representar distintos comportamientos del riesgo a lo largo del tiempo. En la interfaz se muestran los resultados principales del ajuste, las visualizaciones asociadas y la opción de exportación conjunta.

CAPÍTULO 9. DISEÑO DEL SISTEMA



Figura 32: Vista inicial del análisis mediante el modelo de Weibull

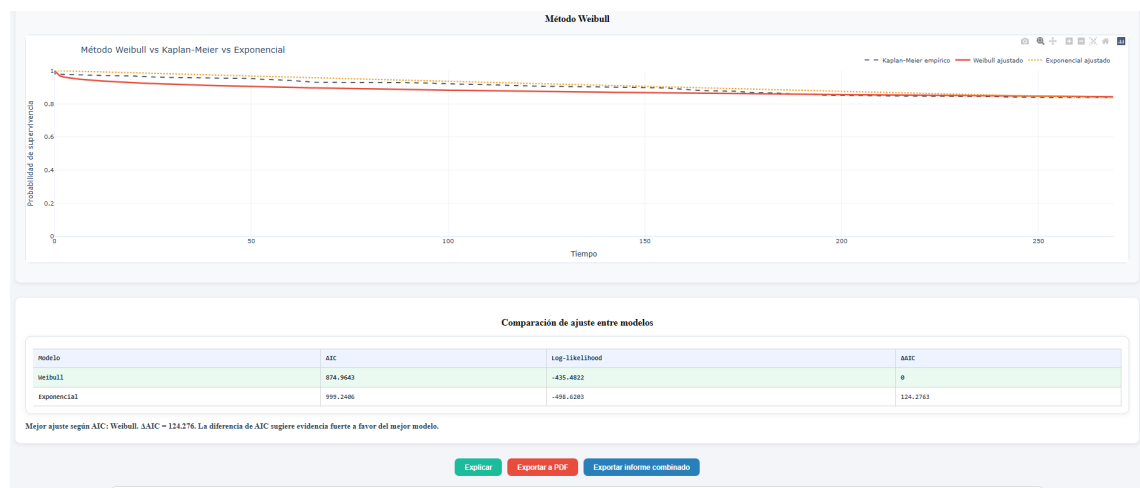


Figura 33: Resultados y visualizaciones del modelo de Weibull

Informe combinado: Weibull + Exponencial



Nombre del informe

Ejemplo: informe_weibull_exponencial

No incluir .pdf

Técnicas

- ☒ Weibull
- ☒ Exponencial

Contenido

- ☒ Resumen
- ☒ Tabla de resultados
- ☒ Gráfica
- ☐ Interpretación IA (opcional)

Nota: Exponencial se incluirá como caso particular del modelo Weibull.

Cancelar

Descargar PDF

Figura 34: Botón de exportación combinada utilizado en Weibull y Exponencial, ya que ambos modelos exportan el mismo tipo de resultados

CAPÍTULO 9. DISEÑO DEL SISTEMA

Exponencial

El modelo exponencial ofrece una aproximación paramétrica más sencilla, basada en una tasa de riesgo constante. Por eso, dentro de la aplicación resulta útil como técnica de referencia, especialmente para comparar su comportamiento frente a modelos más flexibles como Weibull o Random Survival Forest.



Figura 35: Vista inicial del análisis mediante el modelo exponencial

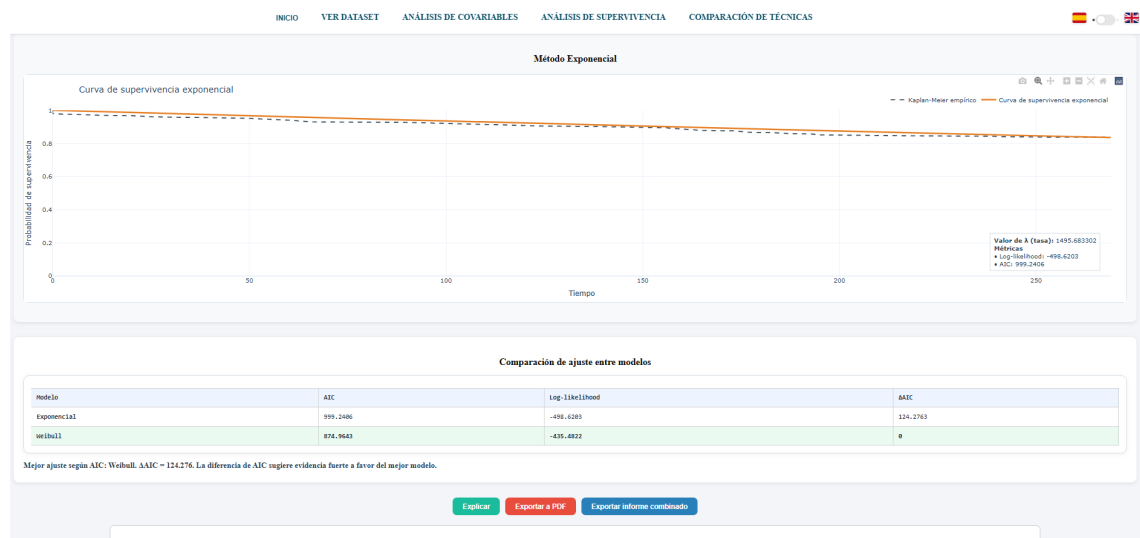


Figura 36: Resultados y visualizaciones del modelo exponencial

CAPÍTULO 9. DISEÑO DEL SISTEMA

Random Survival Forest

Random Survival Forest incorpora una técnica basada en aprendizaje automático para estudiar relaciones más complejas entre las variables del dataset y el tiempo hasta el evento. La interfaz recoge tanto la ejecución del modelo como los resultados gráficos y métricos necesarios para interpretar su comportamiento.

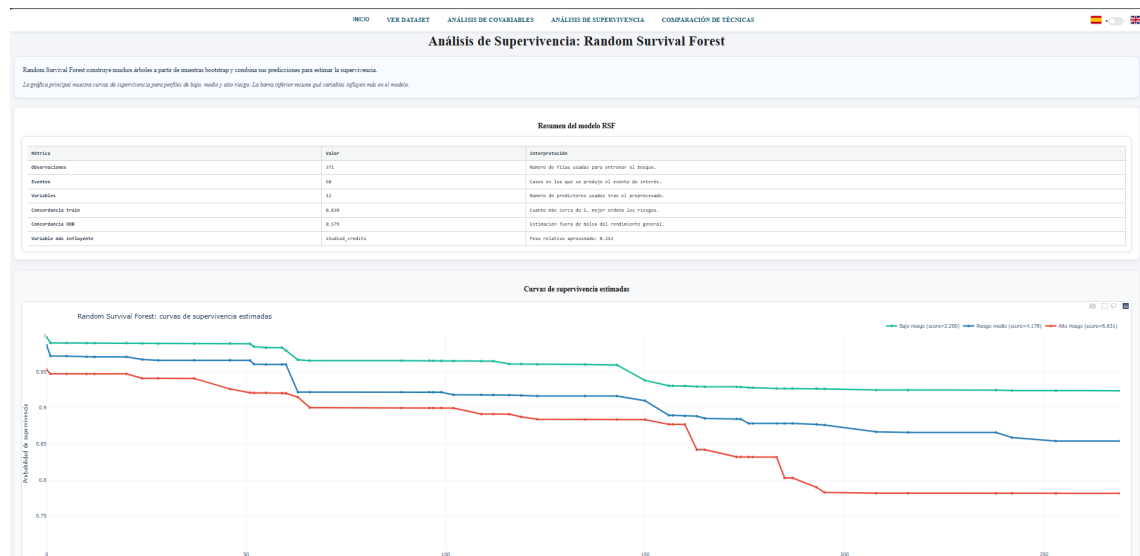


Figura 37: Vista inicial del análisis mediante Random Survival Forest

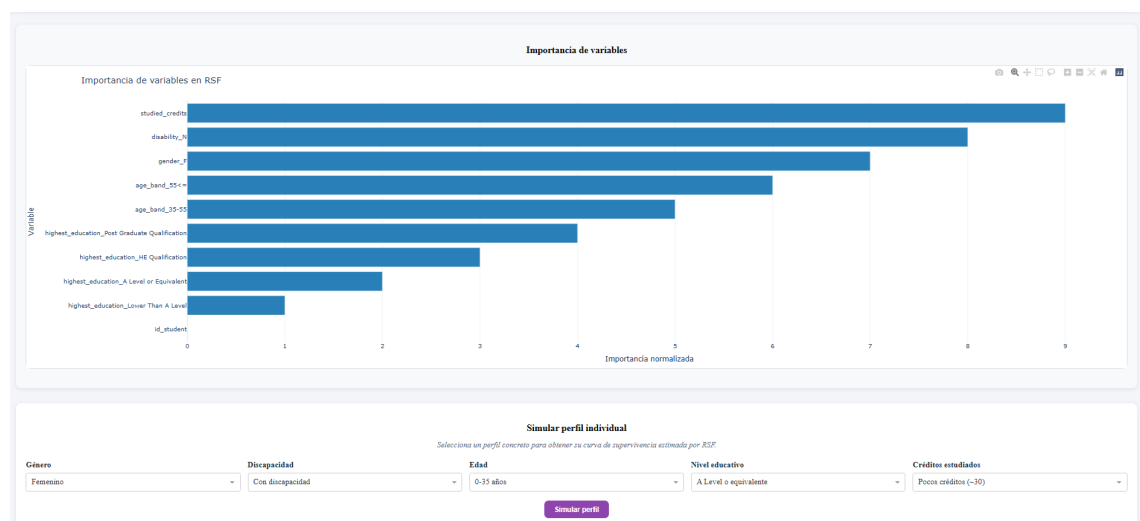


Figura 38: Resultados generales generados por Random Survival Forest

CAPÍTULO 9. DISEÑO DEL SISTEMA

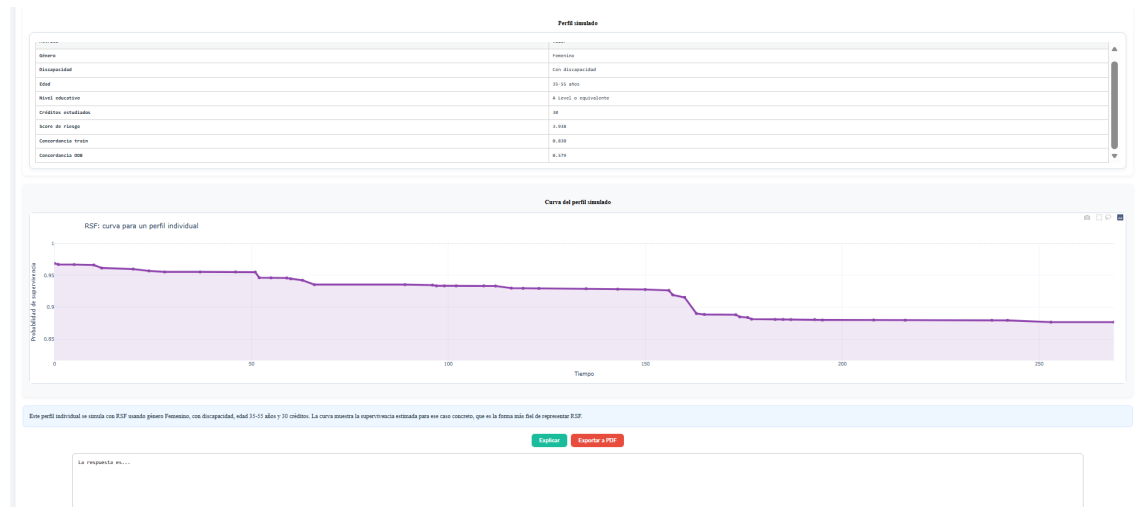


Figura 39: Visualización complementaria del análisis Random Survival Forest

Comparación de técnicas

La página de comparación de técnicas se abre desde el botón específico situado bajo las técnicas de la sección de análisis de supervivencia. A diferencia de las vistas anteriores, no se trata de un modelo ejecutable, sino de una página estática pensada para orientar al usuario. En ella se resume, de forma sencilla, qué hace cada método, qué tipo de información aporta y en qué situaciones puede resultar más útil. La idea es que el usuario no tenga que elegir una técnica a ciegas, sino que cuente con una pequeña guía dentro de la propia aplicación.

INICIO VER DATASET ANÁLISIS DE COVARIABLES ANÁLISIS DE SUPERVIVENCIA **COMPARACIÓN DE TÉCNICAS**

Comparación de técnicas de supervivencia

Esta página resume los principales métodos de supervivencia para facilitar la elección metodológica según objetivo analítico, supuestos y nivel de interpretabilidad.

Técnica	Tipo de modelo	Hipótesis	Ventajas	Limitaciones	Cuando usarlo
Kaplan-Meier	No paramétrico	Censura no informativa; estima supervivencia empírica sin forma funcional.	Muy interpretable; útil como línea base visual.	No ajuste covariables simultáneamente.	Comparación descriptiva inicial entre grupos.
Exponencial	Paramétrico	Riesgo constante en el tiempo (hazard constante).	Sencillo, estable y fácil de interpretar.	Puede infraajustar si el riesgo cambia con el tiempo.	Cuando se espera comportamiento aproximadamente constante del riesgo.
Weibull	Paramétrico	Riesgo monótono (creciente o decreciente) según parámetro de forma.	Flexible y con parámetros interpretables.	Si el riesgo no es monótono puede no capturar bien el patrón.	Cuando se requiere modelo paramétrico más flexible que Exponencial.
Regresión de Cox	Semiparamétrico	Proporcionalidad de riesgos entre grupos/covariables.	Ajusta múltiples covariables sin asumir forma base del hazard.	Sensibilidad a violaciones de proporcionalidad.	Para cuantificar efecto de covariables (hazard ratios).
Log-Rank Test	Test no paramétrico	Compara igualdad global de curvas de supervivencia entre grupos.	Sencillo para contraste de hipótesis entre grupos.	No estima tamaño de efecto multivariable por sí solo.	Para evaluar si las curvas difieren de forma estadísticamente significativa.
Random Survival Forest	Ensamble RL	No requiere proporcionalidad ni forma paramétrica explícita.	Captura relaciones no lineales e interacciones complejas.	Menor interpretabilidad que Cox/RM.	Cuando prima capacidad predictiva y complejidad de patrones.

Figura 40: Página de comparación de técnicas de supervivencia

CAPÍTULO 9. DISEÑO DEL SISTEMA



Figura 41: Detalle explicativo de la comparación de técnicas

Botones

La interfaz incluye distintos botones de acción que guían al usuario durante el uso de la aplicación. Algunos permiten cambiar entre vistas del dataset, otros activan funcionalidades de apoyo como la interpretación mediante inteligencia artificial, y otros se encargan de la exportación de resultados. Aunque son elementos pequeños dentro de la pantalla, la verdad es que tienen bastante peso en la experiencia de uso, porque marcan los pasos principales del flujo de trabajo.

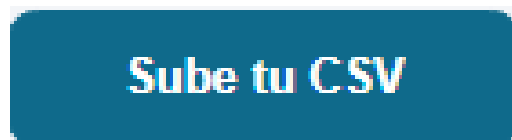


Figura 42: Botón para consultar el dataset bruto



Figura 43: Botón para consultar el dataset limpio

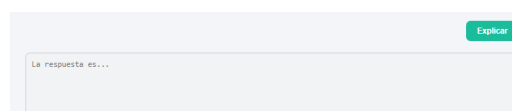


Figura 44: Botón para solicitar una interpretación mediante inteligencia artificial

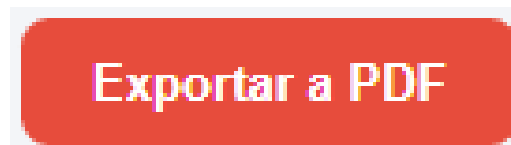


Figura 45: Botón de exportación de resultados

Al pulsar el botón de exportación se abre una ventana modal en la que el usuario puede seleccionar la opción de generar informe. Una vez confirmada la acción, el sistema prepara el documento y se descarga el PDF correspondiente.



Figura 46: Botón de confirmación para generar y descargar el informe PDF

En los modelos Weibull y Exponencial se incluye además un botón de exportación combinada, ya que ambos representan una relación de caso general y caso particular dentro de los modelos paramétricos. Por su parte, Random Survival Forest incorpora un botón específico para simular perfiles, lo que permite explorar el comportamiento del modelo ante distintas combinaciones de variables.

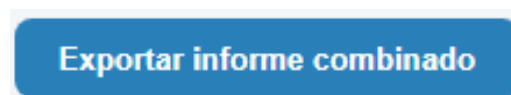


Figura 47: Botón de exportación combinada para Weibull y Exponencial

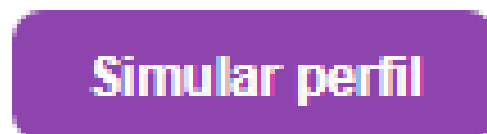


Figura 48: Botón de simulación de perfiles en Random Survival Forest

9.4. Diagrama de clases y componentes

Al tratarse de una aplicación desarrollada principalmente con Dash, funciones de Python y callbacks, el sistema no sigue una estructura clásica basada únicamente

CAPÍTULO 9. DISEÑO DEL SISTEMA

en clases. Por ese motivo, en este apartado se plantea una vista mixta de clases y componentes, donde se representan tanto la clase principal de exportación como los módulos funcionales que sostienen el comportamiento del dashboard. Esta visión resulta más fiel al proyecto que forzar un diagrama de clases tradicional con entidades que realmente no existen en el código.

El componente central es la aplicación Dash, que actúa como núcleo de ejecución y como punto de conexión entre la interfaz, la navegación y los callbacks. A partir de ella se coordinan los componentes visuales y las respuestas del sistema ante las acciones del usuario. Por ejemplo, cuando se selecciona una técnica de análisis, el sistema recupera el dataset preparado, ejecuta el modelo correspondiente y devuelve a la interfaz los resultados que deben mostrarse.

En cuanto a clases propiamente dichas, destaca la clase responsable de construir los informes PDF. Esta clase concentra la creación del documento final, añadiendo secciones, tablas, gráficas e interpretaciones. Su existencia tiene sentido porque la generación de informes requiere mantener una estructura interna y reutilizar operaciones comunes en distintos tipos de salida. Los detalles concretos de esta clase y de los archivos implicados se describen en el capítulo de implementación.

CAPÍTULO 9. DISEÑO DEL SISTEMA

Elemento de diseño	Descripción
Dash App	Núcleo de la aplicación web. Gestiona la ejecución del servidor local y la navegación entre páginas.
Layouts	Componentes visuales creados en <code>layout.py</code> . Definen qué ve el usuario en cada pantalla.
Callbacks de análisis	Funciones que responden a eventos de la interfaz y conectan la entrada del usuario con los modelos estadísticos.
Módulos de modelos	Funciones específicas para Kaplan-Meier, Cox, Test de Log-Rank, Exponencial, Weibull y Random Survival Forest.
Módulo IA	Componente encargado de enviar información del análisis al modelo local y recuperar una explicación textual.
SurvivalAnalysisPDF Exporter	Clase responsable de generar informes PDF reutilizando tablas, gráficos e interpretaciones.
Traducciones	Diccionario de textos que permite adaptar la interfaz al idioma seleccionado.

Tabla 14: Vista de diseño basada en clases y componentes del sistema

9.5. Diagramas de secuencia

Los diagramas de secuencia muestran cómo colaboran los principales componentes del sistema cuando el usuario ejecuta cada una de las funcionalidades definidas en los casos de uso. A diferencia de los diagramas de actividad, que se centran en el flujo de acciones, estos diagramas ayudan a entender qué partes internas intervienen en cada proceso: la interfaz, los callbacks, el dataset, los modelos de análisis, la generación de visualizaciones, la inteligencia artificial o la exportación de informes.

En los diagramas de secuencia se utiliza una nomenclatura descriptiva para facilitar la lectura de las interacciones. Por este motivo, algunos participantes aparecen con nombres funcionales, como “Interfaz Dash”, “Callbacks de análisis” o “Estado del dataset”, en lugar de emplear exactamente los nombres técnicos utilizados en la vista de clases y componentes. Estos participantes no deben interpretarse siempre como clases Python estrictas, sino como componentes o responsabilidades del sistema. No obstante, se corresponden con los elementos descritos anteriormente: la interfaz se relaciona con `DashApp` y `layout.py`, los

CAPÍTULO 9. DISEÑO DEL SISTEMA

callbacks con **AnalysisCallbacks**, el estado del dataset con el componente **Dataset**, y los módulos de análisis con sus correspondientes modelos.

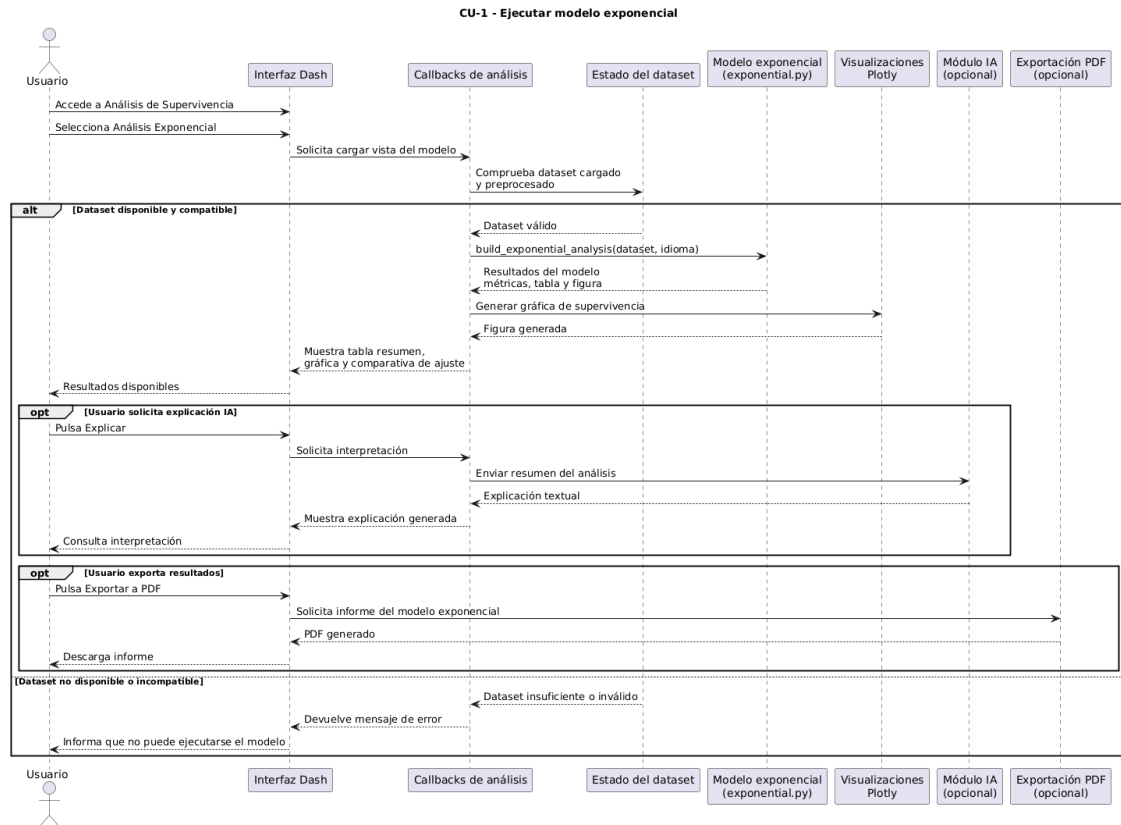


Figura 49: Diagrama de secuencia del CU-1: EjecutarModeloExponencial

CAPÍTULO 9. DISEÑO DEL SISTEMA

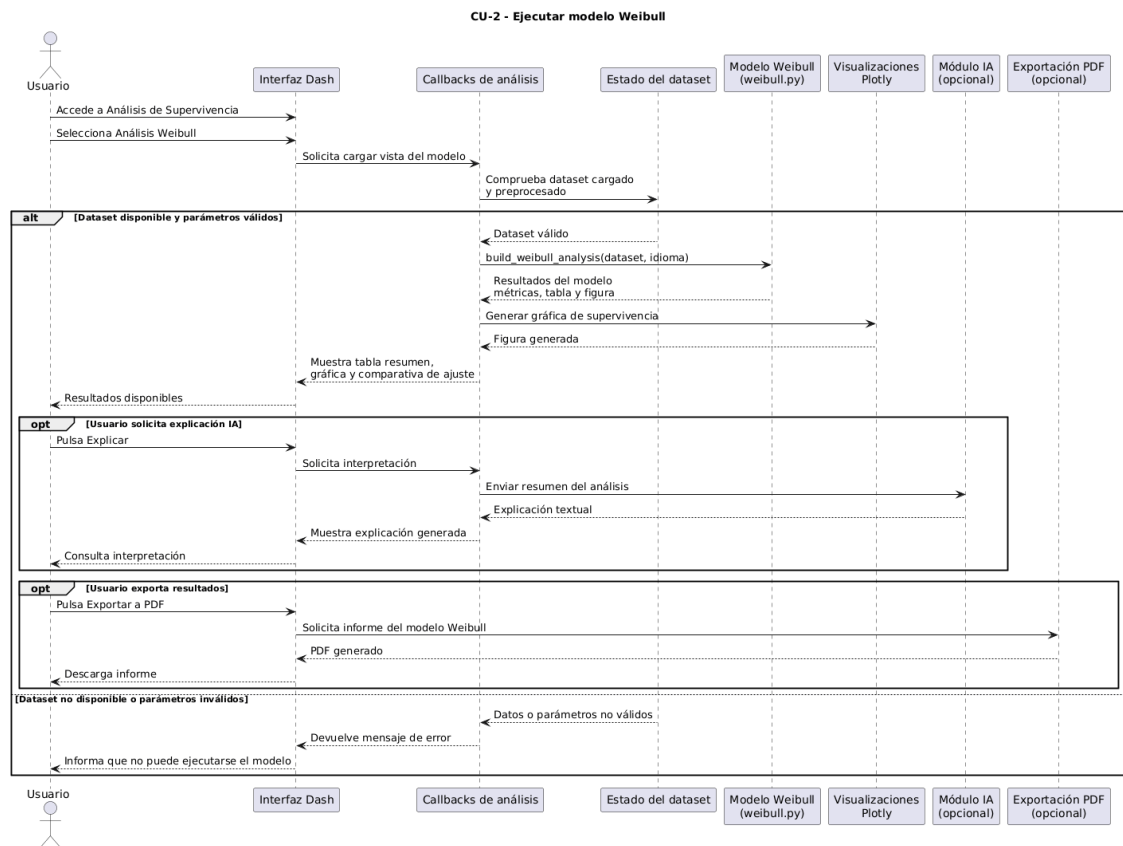


Figura 50: Diagrama de secuencia del CU-2: EjecutarModeloWeibull

CAPÍTULO 9. DISEÑO DEL SISTEMA

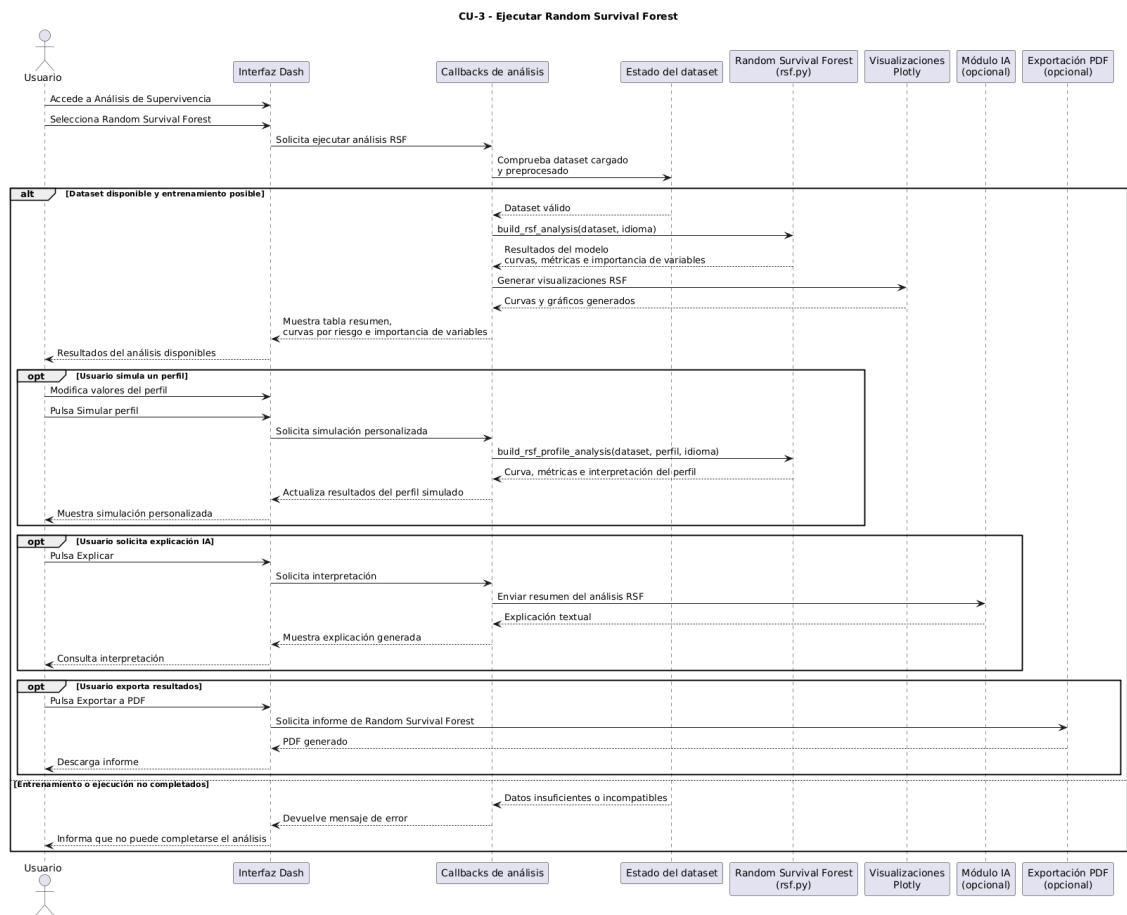


Figura 51: Diagrama de secuencia del CU-3: EjecutarRandomSurvivalForest

CAPÍTULO 9. DISEÑO DEL SISTEMA

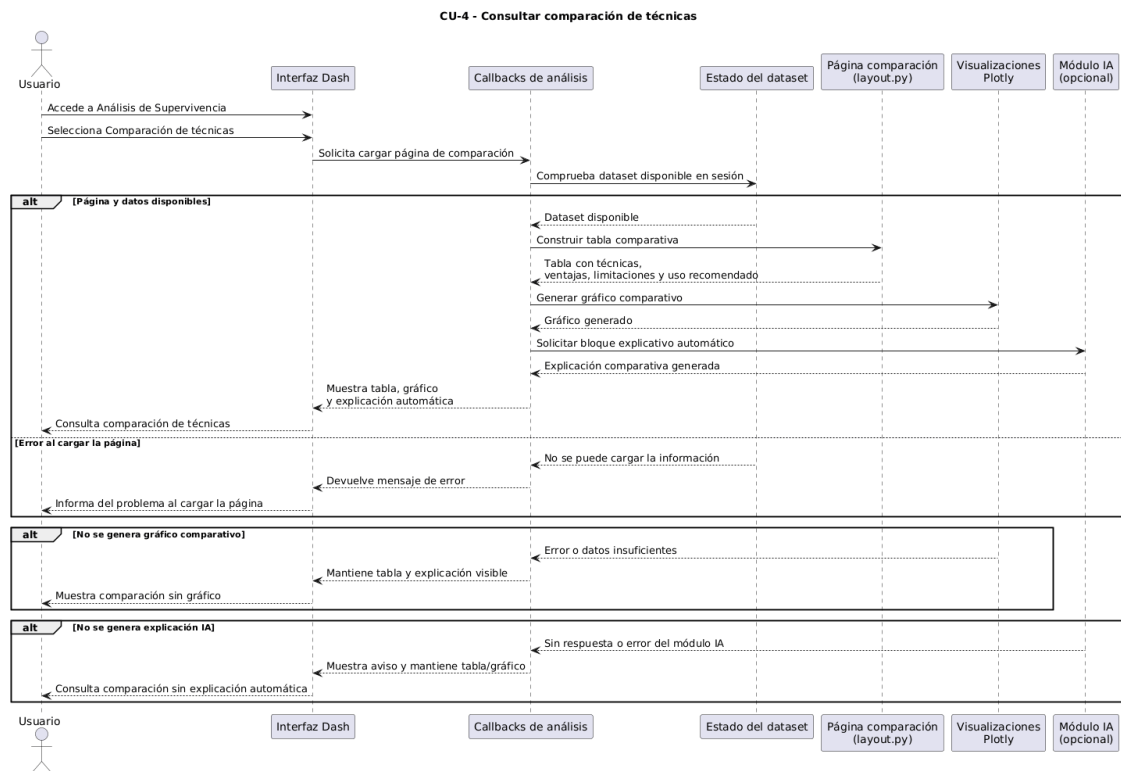


Figura 52: Diagrama de secuencia del CU-4: ConsultarComparacionDeTecnica

CAPÍTULO 9. DISEÑO DEL SISTEMA

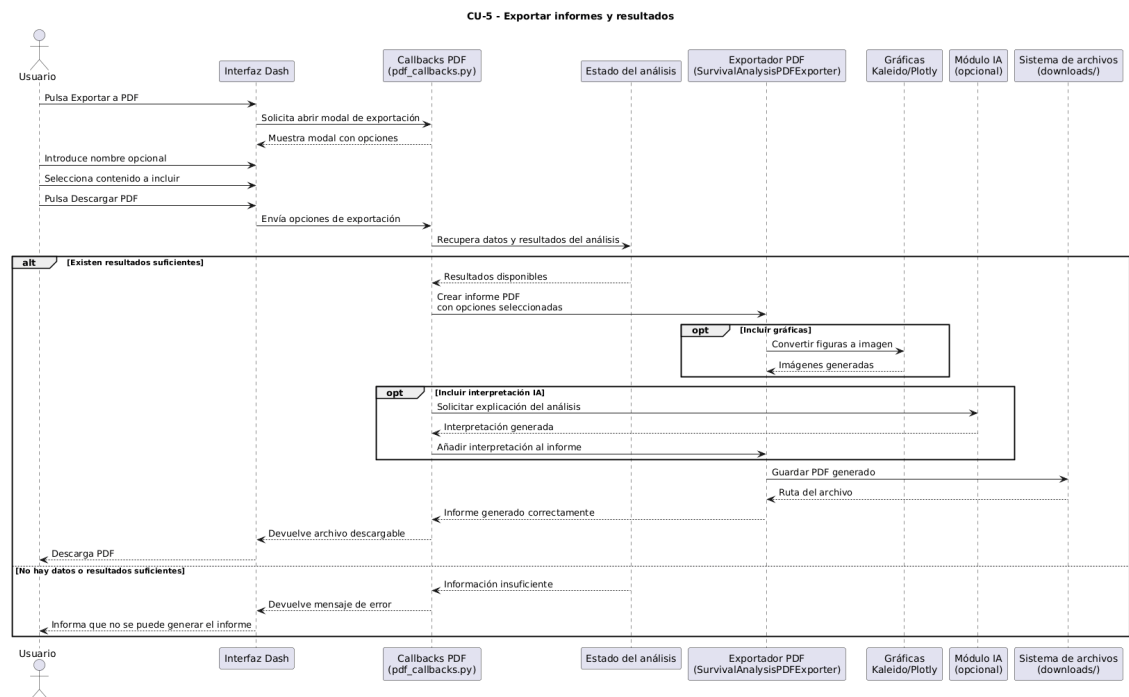


Figura 53: Diagrama de secuencia del CU-5: ExportarInformesYResultados

CAPÍTULO 9. DISEÑO DEL SISTEMA

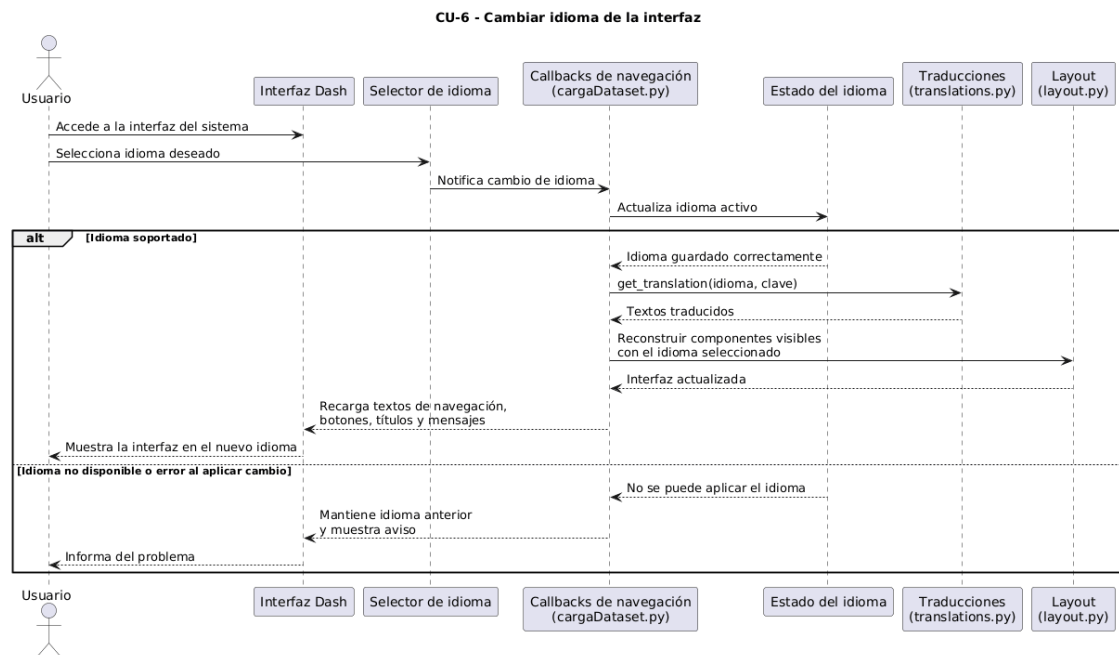


Figura 54: Diagrama de secuencia del CU-6: CambiarIdiomaInterfaz

CAPÍTULO 9. DISEÑO DEL SISTEMA

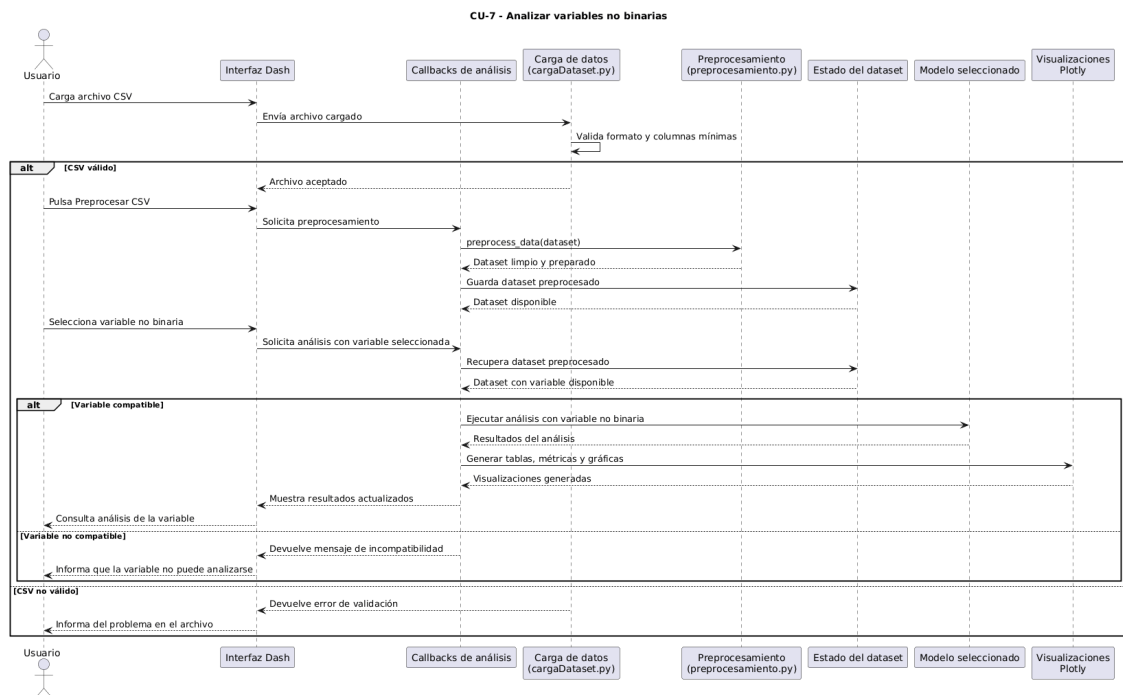


Figura 55: Diagrama de secuencia del CU-7: AnalizarVariablesNoBinarias

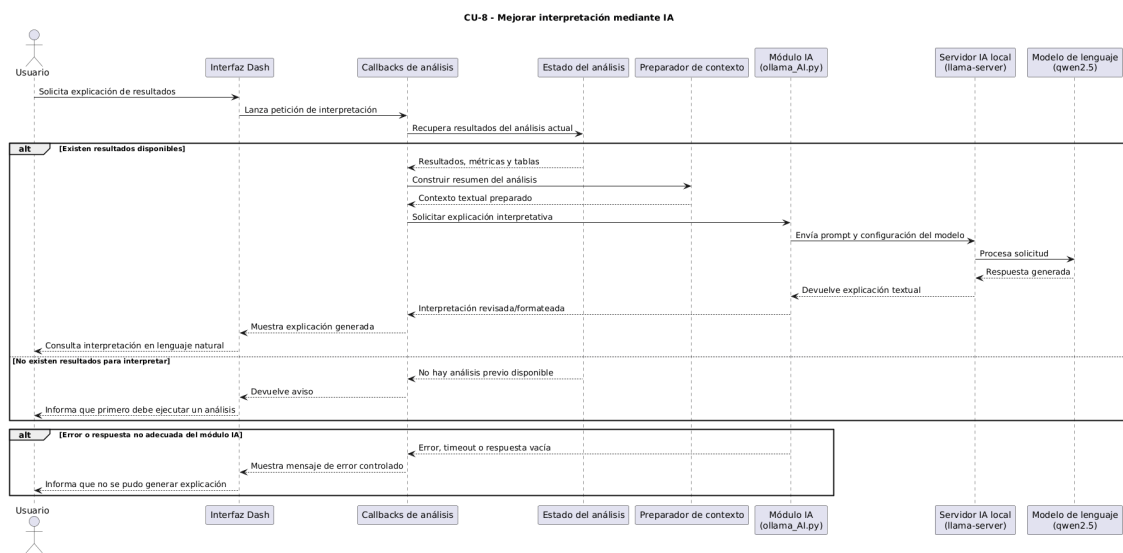


Figura 56: Diagrama de secuencia del CU-8: MejorarInterpretacionIA

CAPÍTULO 9. DISEÑO DEL SISTEMA

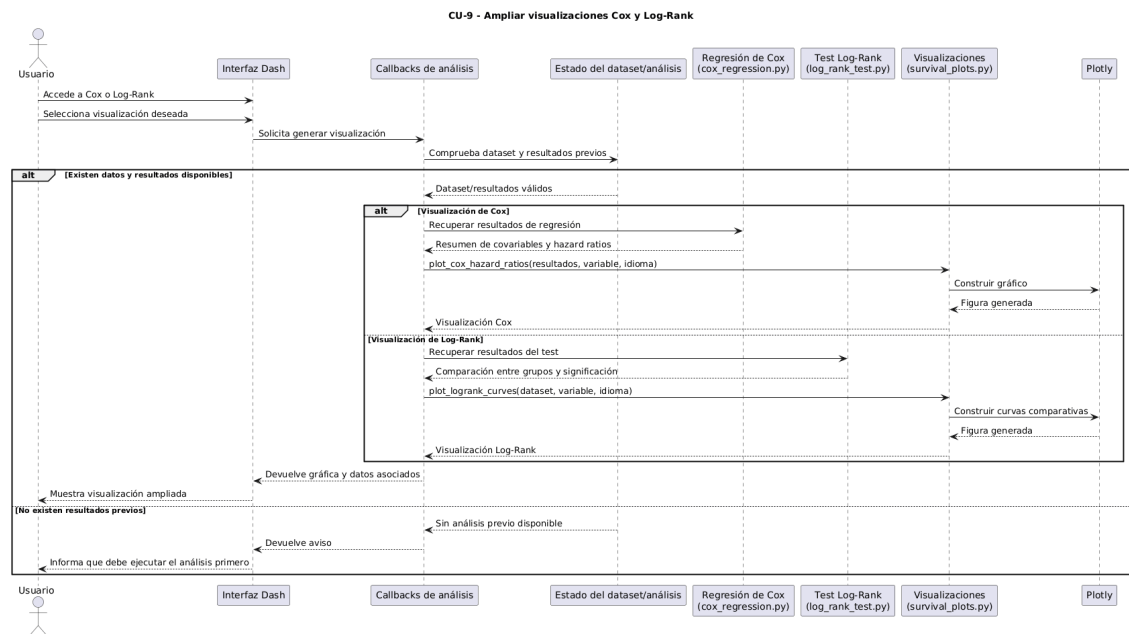


Figura 57: Diagrama de secuencia del CU-9: AmpliarVisualizacionesCoxTestLogRank

10 Implementación

Tras completar las fases de análisis y diseño, se aborda la implementación de la ampliación del dashboard. En este capítulo no se pretende repetir la arquitectura descrita anteriormente, sino explicar cómo se ha materializado en el código fuente: qué librerías se han utilizado, qué archivos concentran cada responsabilidad y cómo se han integrado los nuevos modelos, las visualizaciones, la interpretación mediante inteligencia artificial, la exportación de informes y el soporte multilingüe.

10.1. Tecnologías y librerías utilizadas

La aplicación se ha desarrollado en Python, partiendo de la base tecnológica del proyecto original. Por ese motivo, algunas librerías no se presentan como una aportación nueva, sino como parte del entorno heredado sobre el que se ha construido la ampliación. Es el caso de `Dash`, `Pandas`, `NumPy`, `Plotly`, `lifelines`, `Matplotlib` y la comunicación con un modelo local de inteligencia artificial. Todas ellas siguen siendo necesarias para que el sistema funcione, pero en este capítulo interesa distinguir entre la base ya existente y los elementos que se han reforzado o incorporado para cubrir los nuevos objetivos.

La verdad es que esta separación resulta importante, porque evita presentar como nuevo aquello que ya formaba parte del trabajo previo y permite centrar la explicación en la ampliación realizada. En concreto, se ha prestado especial atención a las librerías y componentes que hacen posible los nuevos modelos paramétricos, Random Survival Forest, las visualizaciones añadidas, la generación de informes PDF y la interpretación automática de resultados.

CAPÍTULO 10. IMPLEMENTACIÓN

Tecnología o librería	Papel dentro de la ampliación
<code>lifelines</code>	Se mantiene como librería estadística principal, pero se amplía su uso mediante <code>ExponentialFitter</code> y <code>WeibullFitter</code> , necesarios para implementar los modelos Exponencial y Weibull y compararlos con Kaplan-Meier [1].
<code>scikit-survival</code>	Se incorpora para implementar Random Survival Forest, construir el objeto de supervivencia requerido por el modelo y calcular métricas como el índice de concordancia [6].
<code>Plotly</code>	Ya formaba parte de la base del dashboard, pero en esta ampliación se utiliza para generar nuevas curvas de supervivencia, comparaciones entre modelos, gráficos de importancia de variables y visualizaciones asociadas a Cox y al Test de Log-Rank.
<code>ReportLab</code> y <code>Kaleido</code>	Se utilizan para construir informes PDF con tablas, textos, cabeceras e imágenes, y para convertir las gráficas interactivas de <code>Plotly</code> en figuras insertables en el documento final.
<code>requests</code>	Permite mantener la comunicación con el servidor local de inteligencia artificial, enviando los resultados estadísticos y recibiendo interpretaciones textuales para la interfaz o los informes.

Tabla 15: Tecnologías con mayor peso en la ampliación implementada

El resto de librerías, como `Dash`, `Pandas` o `NumPy`, se mantienen como soporte fundamental de la aplicación: permiten construir la interfaz, almacenar estados, transformar datos y preparar las entradas que necesitan los modelos. Sin embargo, su explicación se mantiene más breve porque pertenecen principalmente a la infraestructura ya existente. La ampliación se apoya en ellas, pero aporta valor sobre todo en la nueva lógica estadística, las nuevas salidas visuales, la exportación de resultados y la interpretación asistida.

10.2. Estructura del código fuente

El código se organiza en distintos archivos Python, cada uno centrado en una responsabilidad principal. Esta organización evita concentrar toda la lógica en un único fichero y permite que los nuevos modelos añadidos en este TFG se integren sin modificar de forma agresiva el funcionamiento ya existente.

Los nombres de archivo que aparecen en este apartado se incluyen únicamente como referencia textual para facilitar su localización dentro del proyecto. No se

CAPÍTULO 10. IMPLEMENTACIÓN

han configurado como enlaces clicables, ya que la memoria debe poder leerse correctamente aunque el tribunal no disponga de la misma estructura de carpetas local. Todos los archivos Python mencionados se encuentran en la raíz del repositorio de la aplicación, mientras que los recursos propios de la memoria, como imágenes, capítulos y referencias bibliográficas, se ubican dentro de la carpeta `doc/memoria_tfg`.

Archivo	Responsabilidad principal
<code>cargaDataset.py</code>	Inicialización de la aplicación Dash, navegación general entre páginas, cambio de idioma y registro de callbacks principales.
<code>layout.py</code>	Definición de las páginas de la interfaz, incluyendo las vistas de análisis, comparación de técnicas, visualización del dataset, modales de exportación y componentes visuales.
<code>analysis_callbacks.py</code>	Registro de callbacks asociados al análisis de datos, actualización de vistas, ejecución de modelos, interpretación de resultados y conexión entre interfaz y lógica interna.
<code>preprocesamiento.py</code>	Carga del dataset, limpieza de columnas, validación de datos críticos, selección de filas por estudiante y generación del dataset preparado para los modelos.
<code>exponential.py</code> , <code>weibull.py</code> y <code>rsf.py</code>	Implementación de las nuevas técnicas incorporadas: modelo exponencial, modelo de Weibull y Random Survival Forest.
<code>kaplan_meier.py</code> , <code>cox_regression.py</code> , <code>log_rank_test.py</code> y <code>survival_plots.py</code>	Implementación y ampliación de técnicas ya presentes en el sistema base, incluyendo nuevas visualizaciones para Cox y Test de Log-Rank.
<code>ollama_AI.py</code>	Generación de interpretaciones mediante un modelo de inteligencia artificial ejecutado en entorno local.
<code>pdf_callbacks.py</code> y <code>pdf_exporter.py</code>	Gestión de la exportación desde la interfaz y construcción de informes PDF con resultados, gráficas e interpretaciones.
<code>translations.py</code>	Diccionario de traducciones y función de acceso a los textos en castellano e inglés.
<code>config.py</code>	Definición de rutas de datos, configuración del servidor de inteligencia artificial y carga diferida de datasets.

Tabla 16: Distribución de responsabilidades en el código fuente

10.3. Carga y preprocesamiento de datos

La carga inicial del dataset se realiza mediante la función `load_dataset`, definida en `preprocesamiento.py`. Esta función lee el archivo en formato CSV con separador de punto y coma y elimina espacios sobrantes en los nombres de las columnas. Aunque es una operación sencilla, resulta importante porque evita errores posteriores derivados de nombres de columnas mal formados.

El procesamiento principal se concentra en `preprocess_data`. Esta función valida que el dataset no esté vacío y comprueba la existencia de columnas críticas como `id.student`, `date` y `final_result`. A partir de ahí, transforma `final_result` en una variable binaria, donde el abandono se representa como evento observado. Esta conversión es esencial, ya que los modelos de supervivencia necesitan una variable temporal y una variable indicadora del evento.

Una parte relevante del preprocesamiento consiste en seleccionar una única fila representativa por estudiante. Para ello, el sistema ordena los registros por identificador y fecha, y aplica una lógica distinta según el estudiante haya abandonado o no. Si no ha abandonado, se intenta conservar la observación final del periodo de estudio. Si ha abandonado, se priorizan filas con actividad registrada y, en caso contrario, se selecciona una fila representativa del inicio o del registro disponible. Esta decisión reduce duplicidades y deja un dataset más estable para los análisis posteriores.

Además, el preprocesamiento conserva tanto variables binarias como variables no binarias incorporadas en la ampliación. Entre ellas se encuentran `gender_F`, `disability_N`, los rangos de edad definidos mediante `age_band_*`, las categorías de nivel educativo previo recogidas en `highest_education_*` y la variable numérica `studied_credits`. Esta última resulta especialmente relevante porque no se trata de una variable binaria: según la técnica utilizada, el sistema puede agruparla en intervalos o cuantiles para compararla dentro de Kaplan-Meier, Cox o el Test de Log-Rank. También se eliminan columnas no necesarias y se registran advertencias cuando faltan columnas esperadas o aparecen valores nulos en campos críticos. De esta forma, el sistema no solo limpia los datos, sino que también ofrece cierto control sobre la calidad del dataset que se usará en los modelos.

10.4. Implementación de los modelos

10.4.1. Modelo exponencial

El modelo exponencial se implementa en `exponential.py` mediante la función `build_exponential_analysis`. Esta función recibe el dataset limpio, comprueba que existan las columnas `date` y `final_result`, convierte sus valores a formato numérico y elimina registros incompletos. Si los datos no son suficientes, la función devuelve `None`, evitando que la interfaz muestre resultados inconsistentes.

Para el ajuste se utiliza `ExponentialFitter` de la librería `lifelines`. Además, se ajustan también Kaplan-Meier y Weibull para poder comparar el modelo exponencial con una estimación empírica y con su generalización paramétrica. La función calcula métricas como el valor de λ , el log-likelihood y el AIC, y devuelve tanto la gráfica de supervivencia como una tabla resumen con interpretaciones breves.

Una decisión importante de esta implementación es presentar el modelo exponencial como caso particular de Weibull cuando el parámetro de forma es igual a 1. Por eso, la función incluye una comparación directa entre ambos modelos mediante AIC. Esto no solo aporta una salida estadística, sino que ayuda al usuario a entender si la hipótesis de riesgo constante resulta razonable para los datos analizados.

10.4.2. Modelo de Weibull

El modelo de Weibull se implementa en `weibull.py` a través de `build_weibull_analysis`. Su flujo de validación es similar al del modelo exponencial: se comprueba que el dataset tenga las columnas mínimas, se convierten los valores temporales y de evento, y se descartan registros incompletos antes de ajustar el modelo.

La implementación utiliza `WeibullFitter` para obtener los parámetros de forma y escala. El parámetro de forma permite interpretar el comportamiento del riesgo a lo largo del tiempo: si es mayor que 1, el riesgo aumenta; si es menor que 1, disminuye; y si se aproxima a 1, el comportamiento se parece al modelo exponencial. Esta interpretación se incorpora directamente a la tabla de resultados, junto con métricas como la mediana de supervivencia, el log-likelihood y el AIC.

CAPÍTULO 10. IMPLEMENTACIÓN

La gráfica generada compara tres curvas: Kaplan-Meier empírico, Weibull ajustado y Exponencial ajustado. Esta comparación visual permite observar si el modelo paramétrico se adapta de forma razonable al comportamiento real de los datos. Además, al igual que en el modelo exponencial, se genera una tabla comparativa entre Weibull y Exponencial para facilitar la lectura conjunta de ambos modelos.

```
def build_weibull_analysis(df, Language='es'):
    """Fit a Weibull model and build the figure and summary table."""
    if df is None or df.empty:
        return None

    required_columns = ["date", "final_result"]
    missing_columns = [column for column in required_columns if column not in df.columns]
    if missing_columns:
        return None

    working_df = df[required_columns].copy()
    working_df["date"] = pd.to_numeric(working_df["date"], errors="coerce")
    working_df["final_result"] = pd.to_numeric(working_df["final_result"], errors="coerce")
    working_df = working_df.dropna(subset=["date", "final_result"])

    if working_df.empty:
        return None

    durations = working_df["date"].astype(float).where(working_df["date"] > 0, 1e-6)
    events = working_df["final_result"].astype(int)

    if durations.nunique() < 2:
        return None

    wbf = WeibullFitter()
    wbf.fit(durations, event_observed=events, Label="Weibull")

    expf = ExponentialFitter()
    expf.fit(durations, event_observed=events, Label="Exponential")

    kmf = KaplanMeierFitter()
    kmf.fit(durations, event_observed=events, Label="Kaplan-Meier")

    max_time = float(durations.max())
    time_grid = np.linspace(0, max_time, 200)
    fitted_survival = wbf.survival_function_at_times(time_grid).values

    fitted_times = [float(value) for value in time_grid.tolist()]
    fitted_values = [float(value) for value in fitted_survival.tolist()]

    exponential_survival = expf.survival_function_at_times(time_grid).values
    exponential_values = [float(value) for value in exponential_survival.tolist()]

    is_en = Language == 'en'
```

Figura 58: Fragmento de la implementación del modelo de Weibull, donde se ajustan los modelos paramétricos y se obtienen métricas de comparación

10.4.3. Random Survival Forest

Random Survival Forest se implementa en `rsf.py`. A diferencia de los modelos paramétricos, este modelo necesita construir una matriz de características a partir de las variables disponibles en el dataset. Para ello, la función interna `_build_feature_matrix` elimina las columnas temporales y de evento, conserva variables numéricas, booleanas y categóricas, y aplica codificación mediante `get_dummies`. Así se obtiene una matriz preparada para ser procesada por `scikit-survival`.

La función `_fit_rsf_model` crea el objeto de supervivencia mediante `Surv.from_arrays`, ajusta un `RandomSurvivalForest` con 200 árboles y calcula puntuaciones de riesgo para los estudiantes. También obtiene el índice de concordancia sobre entrenamiento y, cuando está disponible, la puntuación *out-of-bag*. Estas métricas permiten valorar de forma general la capacidad predictiva del modelo.

La función principal `build_rsf_analysis` devuelve varias salidas reutilizables por el dashboard: curvas de supervivencia para perfiles representativos, tabla de métricas, importancia de variables y gráficos asociados. Como algunas implementaciones de Random Survival Forest no siempre proporcionan importancias directas de variables, el código incorpora un cálculo alternativo basado en la correlación entre cada característica y las puntuaciones de riesgo. La verdad es que este detalle es importante, porque evita que la vista quede incompleta cuando la librería no ofrece esa información de forma directa.

Además, se incluye `build_rsf_profile_analysis`, que permite simular perfiles de estudiantes. A partir de valores por defecto obtenidos del dataset y de las selecciones realizadas por el usuario, se construye una fila de características compatible con el modelo entrenado. Con ella se estima una curva de supervivencia personalizada, lo que añade una capa interpretativa más cercana al uso práctico del sistema.

CAPÍTULO 10. IMPLEMENTACIÓN

```
def _fit_rsf_model(df: pd.DataFrame):
    feature_df, durations, events = _build_feature_matrix(df)
    if feature_df.empty or durations.nunique() < 2:
        return None

    if len(feature_df) < 5 or feature_df.shape[1] < 1:
        return None

    y = Surv.from_arrays(event=events.astype(bool).to_numpy(), time=durations.to_numpy(dtype=float))

    rsf = RandomSurvivalForest(
        n_estimators=200,
        min_samples_split=10,
        min_samples_leaf=5,
        max_features="sqrt",
        n_jobs=-1,
        random_state=42,
        oob_score=True,
    )
    rsf.fit(feature_df, y)

    risk_scores = rsf.predict(feature_df)
    train_c_index = concordance_index_censored(events.astype(bool).to_numpy(), durations.to_numpy(dtype=float), risk_scores)[0]
    oob_score = getattr(rsf, "oob_score_", None)

    return {
        "model": rsf,
        "feature_df": feature_df,
        "durations": durations,
        "events": events,
        "risk_scores": risk_scores,
        "train_c_index": float(train_c_index),
        "oob_score": None if oob_score is None else float(oob_score),
    }
```

Figura 59: Fragmento de la implementación de Random Survival Forest, incluyendo la construcción del objeto de supervivencia y el ajuste del modelo

10.4.4. Regresión de Cox y Test de Log-Rank

Aunque Cox y el Test de Log-Rank ya formaban parte del sistema base, en esta ampliación se han reforzado sus visualizaciones e integración con el resto del dashboard. La regresión de Cox se ejecuta desde `cox_regression.py`, principalmente mediante `run_cox_regression`. Esta función prepara las covariables seleccionadas, ajusta el modelo con `CoxPHFitter` y genera una tabla de resultados con coeficientes, hazard ratios y valores de significación.

El Test de Log-Rank se implementa en `log_rank_test.py`. La función `perform_log_rank_test` permite comparar curvas de supervivencia entre grupos, incluyendo estrategias específicas para variables multicategoría o variables que requieren agrupación por cuantiles. Esto encaja con uno de los objetivos de la ampliación: permitir el análisis de nuevas variables, incluidas aquellas que no son estrictamente binarias.

Las funciones de visualización asociadas se concentran en `survival_plots.py`. En concreto, `plot_logrank_curves` genera curvas comparativas entre grupos y `plot_cox_hazard_ratios` representa gráficamente los efectos estimados por el

modelo de Cox. Estas gráficas se construyen con `Plotly`, lo que permite mantener el mismo estilo interactivo que el resto del dashboard.

10.5. Generación de gráficas

La generación de gráficas se ha desarrollado principalmente con `Plotly`. Esta librería se adapta bien al proyecto porque permite crear figuras interactivas que se integran directamente en componentes `dcc.Graph` de `Dash`. De esta manera, el usuario puede explorar curvas, valores y comparaciones sin salir de la propia aplicación.

En la práctica, `Plotly` actúa como el puente entre los resultados numéricos de los modelos y la parte visual del dashboard. Los módulos de análisis calculan tablas, métricas o predicciones, y a partir de esos datos se construyen objetos de tipo figura. Estas figuras no son imágenes estáticas desde el principio, sino estructuras reutilizables que contienen los datos representados, las trazas, los ejes, las leyendas y la configuración visual necesaria para mostrarlas en pantalla.

Una vez creada la figura, `Dash` la presenta mediante componentes `dcc.Graph`. Esto permite que las gráficas se actualicen de forma dinámica cuando el usuario cambia una covariable, selecciona un modelo o recalcula un análisis. La ventaja de este enfoque es que no hace falta generar archivos intermedios para visualizar los resultados: el callback devuelve la figura de `Plotly` y `Dash` se encarga de renderizarla en el navegador manteniendo la interactividad propia del dashboard.

En los modelos Exponencial y Weibull, las gráficas se construyen a partir de una rejilla temporal generada con `NumPy`. Sobre esa rejilla se calculan las probabilidades de supervivencia del modelo ajustado y se representan junto a la curva empírica de Kaplan-Meier. En `Random Survival Forest`, en cambio, las curvas se obtienen a partir de las funciones de supervivencia predichas por el modelo para perfiles representativos o simulados.

Además de las curvas de supervivencia, se generan visualizaciones complementarias como gráficos de importancia de variables, curvas comparativas por grupo y forest plots para la regresión de Cox. Todas estas figuras mantienen una estructura común: título, ejes, leyenda, plantilla visual clara y datos preparados para su posterior exportación a PDF.

Este último punto es importante porque la misma figura que se muestra en Dash puede reutilizarse después en otros contextos. Por ejemplo, las curvas de Kaplan-Meier, las curvas ajustadas de Weibull y Exponencial, las visualizaciones de Random Survival Forest o el forest plot de Cox se construyen como figuras de **Plotly** y posteriormente pueden convertirse en imágenes para incluirse en los informes PDF. Así se evita duplicar la lógica de representación y se mantiene una mayor coherencia entre lo que el usuario ve en la aplicación y lo que aparece en el documento exportado.

10.6. Integración con callbacks

La conexión entre interfaz y lógica interna se realiza mediante callbacks de Dash. El archivo `cargaDataset.py` inicializa la aplicación, configura la navegación y registra los callbacks principales mediante `register_analysis_callbacks` y `register_pdf_export_callbacks`. Por su parte, `analysis_callbacks.py` concentra gran parte de las interacciones relacionadas con la carga del dataset, la ejecución de análisis y la actualización de los resultados mostrados en pantalla.

El flujo habitual es el siguiente: el usuario interactúa con un botón, filtro o selector; Dash activa el callback correspondiente; el callback recupera el dataset almacenado, llama a la función de análisis necesaria y devuelve a la interfaz tablas, gráficas, mensajes o componentes actualizados. Este patrón se repite en los distintos modelos, lo que permite que la experiencia de uso sea coherente aunque internamente cada técnica requiera un tratamiento diferente.

La interfaz se define en `layout.py` mediante funciones específicas para cada página. Entre ellas se encuentran las vistas de análisis de supervivencia, Weibull, Exponencial, Random Survival Forest, análisis de covariables y el modal de exportación a PDF. Esta separación permite construir las pantallas de forma modular y reutilizar componentes comunes, como botones, modales o tablas.

10.7. Interpretación mediante inteligencia artificial

La interpretación automática de resultados se implementa en `ollama_AI.py`. El sistema se comunica con un servidor local mediante la librería `requests`, utilizando un endpoint HTTP compatible con la estructura de la API de chat completions. La configuración del servidor y el nombre del modelo se definen en `config.py`, donde se establece `LLAMA_SERVER_URL` y `MODEL_NAME`. En la versión actual se utiliza

CAPÍTULO 10. IMPLEMENTACIÓN

Qwen2.5-1.5B-Instruct en formato GGUF, una variante cuantizada preparada para ejecución local [10, 11].

El modelo no se ejecuta desde un servicio externo, sino mediante `llama.cpp`, una herramienta open-source de inferencia local para modelos de lenguaje en C/C++ [12]. En concreto, se emplea el componente `llama-server.exe`, que levanta un servidor HTTP local y permite consumir el modelo a través de rutas compatibles como `/v1/chat/completions` [13]. Esta decisión permite mantener la generación de explicaciones dentro del propio equipo de trabajo, sin enviar los resultados estadísticos a un servicio remoto.

Un aspecto práctico importante es que el servidor local debe estar activo antes de solicitar una interpretación. Si `llama-server.exe` no está en ejecución o el modelo no se ha cargado correctamente, el módulo de inteligencia artificial no puede devolver una respuesta. Por ello, el código contempla este escenario para evitar que la aplicación quede bloqueada y devolver una salida controlada cuando el servicio no está disponible.

Para mejorar la calidad de las respuestas, el módulo define un prompt de sistema orientado al contexto académico del TFG. Este prompt indica al modelo que debe redactar explicaciones claras, basadas únicamente en los resultados proporcionados y sin usar listas. Además, si la salida generada parece una enumeración, el sistema intenta reescribirla como prosa académica mediante `_rewrite_to_prose_if_needed`. Este pequeño control ayuda a que las interpretaciones encajen mejor con el tono del informe.

La función `generate_interpretation_for_pdf` construye prompts específicos según el tipo de análisis: Kaplan-Meier, Cox, Test de Log-Rank, Weibull o Exponencial. Para cada caso se recopilan datos resumen y tablas de resultados, y se envían al modelo para obtener una explicación textual. De esta manera, la IA actúa como una capa de apoyo interpretativo sobre resultados ya calculados por el sistema, no como sustituto de los modelos estadísticos.

CAPÍTULO 10. IMPLEMENTACIÓN

```
def _academic_system_prompt(Language='es'):
    if Language == 'en':
        return (
            "You are a survival-analysis assistant for an academic thesis. "
            "Write in formal, clear, evidence-based prose using only provided results. "
            "Return 2-3 short paragraphs with no bullet points and no numbered lists."
        )
    return (
        "Eres un asistente de análisis de supervivencia para un TFG. "
        "Redacta en prosa académica clara y basada en evidencias, usando solo resultados proporcionados. "
        "Devuelve 2-3 párrafos breves, sin viñetas ni listas numeradas."
    )
```

Figura 60: Definición del prompt académico utilizado para orientar las respuestas generadas por el módulo de inteligencia artificial

```
def _call_llm(prompt, max_tokens, temperature=DEFAULT_TEMPERATURE, timeout=REQUEST_TIMEOUT_S, Language='es'):
    """Invoca llama-server con parámetros homogéneos y salida académica consistente."""
    payload = {
        "model": MODEL_NAME,
        "messages": [
            {"role": "system", "content": _academic_system_prompt(Language)},
            {"role": "user", "content": prompt}
        ],
        "temperature": temperature,
        "max_tokens": max_tokens,
        "stream": False,
    }
    response = requests.post(LLAMA_SERVER_URL, json=payload, timeout=timeout)
    response.raise_for_status()
    result = response.json()
    content = result['choices'][0]['message']['content'].strip()
    return _rewrite_to_prose_if_needed(content, max_tokens=max_tokens, Language=Language, timeout=timeout)
```

Figura 61: Llamada al modelo local de inteligencia artificial mediante una petición HTTP y tratamiento posterior de la respuesta generada

10.8. Exportación de informes

La exportación de informes se divide en dos partes. Por un lado, `pdf_callbacks.py` gestiona la interacción con la interfaz: apertura de modales, selección de contenido, validación de interpretaciones y descarga del archivo generado. Por otro lado, `pdf_exporter.py` contiene la lógica encargada de construir el documento PDF.

El elemento central de la exportación es la clase `SurvivalAnalysisPDFExporter`. Esta clase configura los estilos del documento, añade cabeceras, textos, tablas, gráficas e interpretaciones, y genera el archivo final. Para ello utiliza `ReportLab`, que permite construir PDFs de forma programática, y `Plotly` junto con `Kaleido`, que facilita convertir las figuras en imágenes aptas para ser insertadas en el informe.

CAPÍTULO 10. IMPLEMENTACIÓN

El proceso seguido para las gráficas es distinto al de una captura de pantalla. Primero, el análisis genera una figura de **Plotly** y la interfaz la muestra en Dash mediante **dcc.Graph**. Si el usuario decide exportar el informe e incluye la opción de gráfica, esa figura se recupera o se regenera con los mismos datos del análisis y se transforma en una imagen mediante **Kaleido**. Finalmente, **ReportLab** inserta esa imagen dentro del PDF junto con el resto de secciones del informe.

Este flujo permite mantener una relación directa entre visualización interactiva y documento exportado. En pantalla, **Plotly** aporta exploración e interacción; en el PDF, la misma representación se conserva como imagen fija para que pueda consultarse, archivarse o compartirse. De esta forma, la exportación no depende de una imagen tomada manualmente, sino de la propia estructura generada por el sistema.

El sistema contempla una exportación general para los análisis de supervivencia y una exportación combinada para Weibull y Exponencial. Esta última tiene sentido porque ambos modelos están relacionados: el exponencial puede entenderse como un caso particular del Weibull. Por ello, el informe combinado permite presentar sus métricas y comparaciones en un mismo documento.

Como resultado de este proceso, el sistema no genera una simple captura de pantalla, sino un informe estructurado. El documento incluye una cabecera identificativa, el tipo de análisis realizado, tablas resumen con los valores obtenidos, métricas relevantes y, cuando procede, gráficas exportadas desde **Plotly** mediante **Kaleido**. Además, si se ha solicitado una interpretación mediante inteligencia artificial, esta también puede incorporarse al informe para acompañar los resultados con una explicación textual más cercana.

CAPÍTULO 10. IMPLEMENTACIÓN



Informe del modelo exponencial

Informe combinado Weibull-Exponencial

Figura 62: Ejemplos de informes PDF generados por el sistema a partir de distintos análisis

10.9. Internacionalización del Dash

El soporte multilingüe se implementa mediante el archivo `translations.py`. Este módulo contiene un diccionario de textos en castellano e inglés y expone la función `get_translation`, que recibe el idioma actual y la clave del texto solicitado. Con este mecanismo, la interfaz puede cambiar de idioma sin duplicar la lógica de las páginas.

El idioma se gestiona desde `cargaDataset.py`, donde se actualizan los textos de navegación y se pasa el idioma activo a las funciones de layout y análisis. A su vez, los modelos y las exportaciones reciben el parámetro `language`, lo que permite adaptar títulos, etiquetas, captions, tablas e interpretaciones. Esta decisión evita que el cambio de idioma sea solo superficial y hace que afecte también a los resultados generados por el sistema.

CAPÍTULO 10. IMPLEMENTACIÓN

```
def get_translation(Lang, key):  
    """  
    Obtiene una traducción basada en el idioma y la clave.  
  
    ✓ ERROR #5: Manejo seguro de traducciones faltantes  
    - Valida que el idioma exista  
    - Retorna clave como fallback si traducción no existe  
    - Loguea advertencias para debugging  
    """  
    # Validar idioma  
    if Lang not in translations:  
        print(f"⚠ [TRANSLATION] Idioma '{Lang}' no soportado, usando 'es'")  
        Lang = 'es'  
  
    # Obtener traducción  
    translation = translations.get(Lang, {}).get(key, None)  
  
    # Si no existe, retornar la clave como fallback  
    if translation is None:  
        print(f"⚠ [TRANSLATION] Clave '{key}' no encontrada para idioma '{Lang}'")  
        return key  
  
    return translation
```

Figura 63: Implementación del sistema de traducciones empleado en la internacionalización del dashboard

10.10. Aspectos destacables de la implementación

Uno de los aspectos más relevantes de la implementación es que las nuevas técnicas se han añadido como módulos independientes. Exponencial, Weibull y Random Survival Forest cuentan con sus propios archivos y funciones de construcción de resultados, pero devuelven estructuras parecidas: figura, tabla resumen, métricas e interpretación. Esto facilita que los callbacks puedan integrarlas sin tener que tratar cada técnica como un caso completamente aislado.

También destaca la separación entre análisis, visualización, IA y exportación. Los modelos no generan directamente el PDF, ni la interfaz contiene la lógica estadística completa. Cada parte tiene su lugar, y eso hace que el sistema sea más mantenible. Si en el futuro se quisiera añadir otra técnica de supervivencia, el camino natural sería crear un nuevo módulo de análisis, una página en `layout.py`, los callbacks correspondientes y, si procede, una rutina de exportación.

Por último, la implementación mantiene compatibilidad con el trabajo previo, pero añade funcionalidades que amplían claramente su alcance: nuevas variables, nuevos modelos, nuevas visualizaciones, interpretación más controlada mediante IA,

CAPÍTULO 10. IMPLEMENTACIÓN

exportación mejorada y soporte multilingüe. En conjunto, el código resultante no sustituye el dashboard original, sino que lo extiende de forma progresiva y coherente con los objetivos del TFG.

11 Pruebas

Una vez finalizada la implementación de la aplicación, resulta fundamental verificar que las funcionalidades desarrolladas producen los resultados esperados. El objetivo de este capítulo es comprobar, caso por caso, que el sistema responde correctamente tanto ante situaciones válidas como ante escenarios erróneos o incompletos. De esta forma, no solo se valida que la funcionalidad exista, sino también que se comporte de manera controlada cuando las condiciones de ejecución no son adecuadas.

11.1. CU-1: EjecutarModeloExponencial

Prueba válida

Caso de prueba	Ejecutar el modelo exponencial con un dataset previamente cargado, válido y preprocesado.
Objetivo de la prueba	Verificar que el sistema permite aplicar el modelo exponencial como técnica de análisis de supervivencia y que muestra los resultados generados de forma comprensible para el usuario.
Resultado esperado	El sistema carga la vista del modelo, valida la existencia del dataset, ajusta el modelo exponencial y muestra la curva de supervivencia, la tabla de métricas y la comparación con otros ajustes, dejando los resultados disponibles para su interpretación o exportación.
Resultado obtenido	El sistema ejecuta correctamente el análisis exponencial y muestra los resultados asociados, incluyendo la representación gráfica de la supervivencia y las métricas principales del modelo, como el parámetro λ , el AIC y el log-likelihood.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 17: Prueba válida del CU-1: EjecutarModeloExponencial

CAPÍTULO 11. PRUEBAS



Figura 64: Evidencia de ejecución correcta del CU-1: EjecutarModeloExponencial

Prueba errónea

Caso de prueba	Intentar ejecutar el modelo exponencial sin disponer de un dataset cargado o con datos insuficientes para ajustar el análisis.
Objetivo de la prueba	Comprobar que el sistema no genera resultados estadísticos inválidos cuando no se cumplen las condiciones mínimas necesarias para ejecutar el modelo.
Resultado esperado	El sistema debe impedir la ejecución del análisis o devolver una salida controlada, evitando mostrar gráficas, métricas o interpretaciones que puedan inducir a error.
Resultado obtenido	El sistema no completa el análisis cuando los datos no son válidos o no contienen información suficiente, manteniendo un comportamiento controlado y evitando presentar resultados inconsistentes.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con la validación implementada en el modelo.
Solución	No se requiere ninguna corrección. La acción adecuada es cargar y preprocesar un dataset válido antes de ejecutar el modelo exponencial.

Tabla 18: Prueba errónea del CU-1: EjecutarModeloExponencial

11.2. CU-2: EjecutarModeloWeibull

Prueba válida

Caso de prueba	Ejecutar el modelo de Weibull con un dataset previamente cargado, válido y preprocesado.
Objetivo de la prueba	Verificar que el sistema permite aplicar el modelo de Weibull como técnica paramétrica de análisis de supervivencia, mostrando los resultados necesarios para estudiar el comportamiento del riesgo a lo largo del tiempo.
Resultado esperado	El sistema carga la vista del modelo, comprueba que existe un dataset disponible, ajusta el modelo de Weibull y muestra la tabla resumen, la gráfica de supervivencia y la comparativa de ajuste. Además, los resultados deben quedar disponibles para su interpretación mediante IA o para su exportación a PDF.
Resultado obtenido	El sistema ejecuta correctamente el análisis de Weibull y presenta los resultados asociados al modelo, incluyendo la curva de supervivencia, los parámetros principales del ajuste y las métricas de comparación necesarias para valorar su comportamiento frente a otros modelos.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 19: Prueba válida del CU-2: EjecutarModeloWeibull



Figura 65: Evidencia de ejecución correcta del CU-2: EjecutarModeloWeibull

CAPÍTULO 11. PRUEBAS

Prueba errónea

Caso de prueba	Intentar ejecutar el modelo de Weibull sin disponer de un dataset cargado, con datos insuficientes o con información no compatible con el ajuste del modelo.
Objetivo de la prueba	Comprobar que el sistema controla los errores de entrada y evita generar resultados estadísticos cuando no se cumplen las condiciones mínimas para ajustar correctamente el modelo de Weibull.
Resultado esperado	El sistema debe impedir la ejecución completa del análisis o devolver una respuesta controlada, informando de que no es posible calcular el modelo con los datos disponibles y evitando mostrar métricas o gráficas incorrectas.
Resultado obtenido	El sistema no genera resultados cuando el dataset no está disponible o no contiene información suficiente para realizar el ajuste, manteniendo un comportamiento coherente con las validaciones definidas en el flujo alternativo del caso de uso.
Problemas encontrados	No se han encontrado fallos críticos. El sistema responde de forma controlada ante la ausencia de datos válidos.
Solución	No se requiere ninguna corrección. La acción adecuada es cargar y preprocesar un dataset válido antes de ejecutar el modelo de Weibull.

Tabla 20: Prueba errónea del CU-2: EjecutarModeloWeibull

11.3. CU-3: EjecutarRandomSurvivalForest

Prueba válida

CAPÍTULO 11. PRUEBAS

Caso de prueba	Ejecutar el modelo Random Survival Forest con un dataset previamente cargado, válido y preprocesado.
Objetivo de la prueba	Verificar que el sistema permite aplicar Random Survival Forest como técnica de aprendizaje automático para análisis de supervivencia, mostrando tanto los resultados generales del modelo como la información asociada a la simulación de perfiles.
Resultado esperado	El sistema debe cargar la vista del modelo, entrenar Random Survival Forest con los datos disponibles y mostrar la tabla resumen, las curvas de supervivencia por niveles de riesgo, la importancia de variables y el panel de simulación de perfil con valores iniciales. Además, debe permitir recalcular el perfil y dejar disponibles las acciones de explicación mediante IA y exportación a PDF.
Resultado obtenido	El sistema ejecuta correctamente el análisis con Random Survival Forest y presenta los resultados esperados, incluyendo las estimaciones de supervivencia, la información sobre variables relevantes y el bloque de simulación de perfil para explorar escenarios concretos de forma interactiva.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 21: Prueba válida del CU-3: EjecutarRandomSurvivalForest

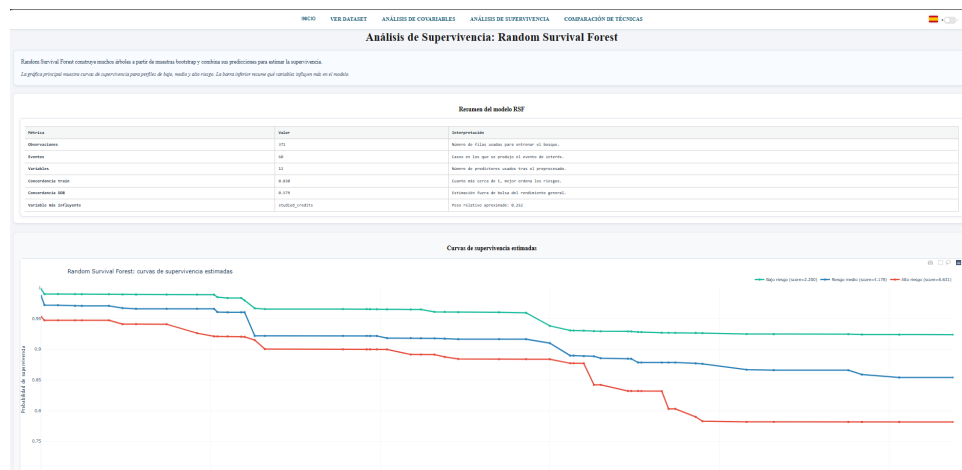


Figura 66: Evidencia de ejecución correcta del CU-3: EjecutarRandomSurvivalForest

CAPÍTULO 11. PRUEBAS

Prueba errónea

Caso de prueba	Intentar ejecutar Random Survival Forest sin disponer de un dataset cargado, con datos insuficientes o con variables no válidas para entrenar el modelo.
Objetivo de la prueba	Comprobar que el sistema controla los errores que pueden impedir el entrenamiento o la ejecución del modelo, evitando mostrar resultados incompletos, métricas inconsistentes o simulaciones de perfil no fiables.
Resultado esperado	El sistema debe detener la ejecución del análisis o devolver una respuesta controlada cuando no sea posible entrenar Random Survival Forest con los datos disponibles, informando al usuario de la imposibilidad de completar el proceso.
Resultado obtenido	El sistema no completa el entrenamiento cuando los datos no cumplen las condiciones mínimas necesarias y mantiene un comportamiento controlado, sin presentar curvas, tablas o simulaciones que puedan interpretarse como resultados válidos.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con el flujo alternativo definido para el caso de uso.
Solución	No se requiere ninguna corrección. La acción adecuada es cargar y preprocesar un dataset válido, con las variables necesarias, antes de ejecutar Random Survival Forest.

Tabla 22: Prueba errónea del CU-3: EjecutarRandomSurvivalForest

11.4. CU-4: ConsultarComparacionDeTecnicas

Prueba válida

CAPÍTULO 11. PRUEBAS

Caso de prueba	Acceder a la página de comparación de técnicas desde el botón situado bajo las técnicas de la sección de Análisis de supervivencia, con un dataset disponible en sesión.
Objetivo de la prueba	Verificar que el sistema permite consultar la página comparativa de técnicas y que muestra correctamente la información de apoyo para interpretar las diferencias entre los métodos implementados.
Resultado esperado	Al pulsar el botón “Comparación de técnicas”, el sistema debe cargar la página de comparación, mostrar la tabla comparativa con las características principales de cada técnica, generar la visualización de puntuaciones orientativas y presentar un bloque explicativo que resuma las diferencias entre modelos y su contexto de uso recomendado.
Resultado obtenido	El sistema carga correctamente la página de comparación de técnicas y muestra la información prevista, incluyendo la tabla de métodos, la representación comparativa y el contenido explicativo asociado, facilitando al usuario una visión conjunta de los modelos disponibles.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 23: Prueba válida del CU-4: ConsultarComparacionDeTecnica



Técnica	Tipo de modelo	Hipótesis	Ventajas	Limitaciones	Código fuente
Kaplan-Meier	No paramétrico	Censura no informativa; estimé supervivencia empírica sin forma funcional.	No interpretable; útil como línea base visual.	No ajusta covariables simultáneamente.	Comparación descriptiva inicial entre grupos.
Exponencial	Paramétrico	Riesgo constante en el tiempo (hazard constante).	Simples, estable y fácil de interpretar.	Puede infraajustar si el riesgo cambia con el tiempo.	Cuando se espera comportamiento aproximadamente constante del riesgo.
Weibull	Paramétrico	Riesgo monótono (creciente o decreciente) según parámetro de forma.	Flexibles y con parámetros interpretables.	Si el riesgo no es monótono puede no capturar bien el patrón.	Cuando se requiere modelo paramétrico más flexible que exponencial.
Regresión de Cox	Semiparamétrico	Proporcionalidad de riesgos entre grupos/covariables.	Ajusta múltiples covariables sin asumir forma base del hazard.	Sensibilidad a violaciones de proporcionalidad.	Para cuantificar efecto de covariables (hazard ratios).
Log-rank test	Test no paramétrico	Compara igualdad global de curvas de supervivencia entre grupos.	Simples para contraste de hipótesis entre grupos.	No estima tamaño de efecto multivariable por sí solo.	Para evaluar si las curvas difieren de forma estadísticamente significativa.
Random Survival Forest	Ensemble ML	No requiere proporcionalidad ni forma paramétrica explícita.	Captura relaciones no lineales e interacciones complejas.	Baja interpretabilidad que COX.	Cuando prima capacidad predictiva y complejidad de patrones.

Figura 67: Evidencia de ejecución correcta del CU-4: ConsultarComparacionDeTecnica

Prueba errónea

CAPÍTULO 11. PRUEBAS

Caso de prueba	Intentar acceder a la página de comparación de técnicas cuando no existe un dataset disponible en sesión o cuando alguno de los elementos comparativos no puede generarse correctamente.
Objetivo de la prueba	Comprobar que el sistema mantiene un comportamiento controlado si no puede cargar la página completa, generar la visualización comparativa o producir el bloque explicativo automático.
Resultado esperado	El sistema debe informar al usuario si la página no puede cargarse. Si el problema afecta solo a una parte concreta, debe mantener visible el resto de la información disponible, como la tabla comparativa, y omitir únicamente el elemento que no haya podido generarse.
Resultado obtenido	El sistema responde de forma controlada ante situaciones incompletas, evitando dejar la vista en un estado inconsistente y conservando la información comparativa disponible cuando no es posible generar alguno de los elementos auxiliares.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con los flujos alternativos definidos para este caso de uso.
Solución	No se requiere ninguna corrección. La acción adecuada es acceder a la funcionalidad desde la sección correspondiente y disponer de los datos necesarios para completar la comparación.

Tabla 24: Prueba errónea del CU-4: ConsultarComparacionDeTecnica

11.5. CU-5: ExportarInformesYResultados

Prueba válida

CAPÍTULO 11. PRUEBAS

Caso de prueba	Exportar a PDF los resultados generados previamente por uno de los módulos de análisis del sistema.
Objetivo de la prueba	Verificar que el sistema permite generar un informe reutilizable a partir de los resultados, tablas, gráficas e interpretaciones disponibles en la aplicación.
Resultado esperado	El sistema debe abrir el modal de exportación al pulsar el botón correspondiente, permitir introducir opcionalmente el nombre del archivo y seleccionar el contenido que se desea incluir. Tras confirmar la operación, debe validar que existen resultados suficientes, generar el informe en formato PDF y entregar el archivo para su descarga.
Resultado obtenido	El sistema muestra correctamente la opción de exportación, permite confirmar la generación del informe y produce el archivo PDF con los bloques seleccionados, dejando los resultados disponibles para documentación o análisis posteriores.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 25: Prueba válida del CU-5: ExportarInformesYResultados

Exportar a PDF

⇒


exp: 15/04/2026 11:31


EXPONENTIAL METHOD REPORT: GENERAL

I. RESUMEN GENERAL

Métrica	Valor
Número de pacientes	371
Número de eventos	60
Tasa de eventos (%)	16.2%
Follow-up medio (meses)	241.89
Follow-up mediano (meses)	209.00

Informe PDF generado

Figura 68: Evidencia de exportación correcta del CU-5: ExportarInformesYResultados

Prueba errónea

CAPÍTULO 11. PRUEBAS

Caso de prueba	Intentar generar un informe PDF sin disponer de resultados previos o cuando el sistema no puede construir el archivo solicitado.
Objetivo de la prueba	Comprobar que el sistema valida la existencia de información suficiente antes de generar el documento y que responde de forma controlada si la exportación no puede completarse.
Resultado esperado	El sistema debe impedir la generación del PDF cuando no existan datos o resultados exportables, mostrando un mensaje de error o evitando entregar un archivo incompleto.
Resultado obtenido	El sistema no genera el informe cuando no se cumplen las condiciones necesarias para la exportación y mantiene un comportamiento controlado, evitando la descarga de documentos vacíos o inconsistentes.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con la validación previa a la generación del archivo.
Solución	No se requiere ninguna corrección. La acción adecuada es ejecutar previamente un análisis válido y disponer de resultados antes de utilizar la funcionalidad de exportación.

Tabla 26: Prueba errónea del CU-5: ExportarInformesYResultados

11.6. CU-6: CambiarIdiomaInterfaz

Prueba válida

CAPÍTULO 11. PRUEBAS

Caso de prueba	Cambiar el idioma de la interfaz desde el selector disponible en la barra de navegación.
Objetivo de la prueba	Verificar que el sistema permite seleccionar otro idioma disponible y que actualiza los textos visibles de la interfaz sin alterar el funcionamiento general de la aplicación.
Resultado esperado	El sistema debe mostrar los idiomas disponibles, procesar la selección realizada por el usuario y recargar los textos visibles de la interfaz en el idioma elegido, manteniendo accesibles las mismas secciones y funcionalidades del dashboard.
Resultado obtenido	El sistema aplica correctamente el cambio de idioma y actualiza los textos de navegación, botones, títulos y mensajes principales, conservando el estado funcional de la aplicación y permitiendo continuar el uso normal del sistema.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 27: Prueba válida del CU-6: CambiarIdiomaInterfaz

Prueba errónea

CAPÍTULO 11. PRUEBAS

Caso de prueba	Intentar aplicar un idioma no disponible o provocar una situación en la que el sistema no pueda cargar correctamente los textos traducidos.
Objetivo de la prueba	Comprobar que el sistema mantiene un comportamiento estable cuando no puede aplicar el idioma seleccionado, evitando dejar la interfaz con textos incompletos o mezclados de forma incoherente.
Resultado esperado	El sistema debe conservar el idioma anterior y, en caso necesario, mostrar un aviso al usuario indicando que no ha sido posible aplicar el cambio solicitado.
Resultado obtenido	El sistema mantiene la interfaz en un estado coherente cuando no puede aplicar una traducción, conservando el idioma previo y evitando que la navegación o las funcionalidades principales queden afectadas.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con el flujo alternativo definido para este caso de uso.
Solución	No se requiere ninguna corrección. La acción adecuada es seleccionar uno de los idiomas disponibles y mantener correctamente definidas las traducciones del sistema.

Tabla 28: Prueba errónea del CU-6: CambiarIdiomaInterfaz

11.7. CU-7: AnalizarVariablesNoBinarias

Prueba válida

CAPÍTULO 11. PRUEBAS

Caso de prueba	Cargar un dataset que contiene variables adicionales o no binarias y analizarlas desde los módulos de covariables o supervivencia.
Objetivo de la prueba	Verificar que el sistema conserva las variables no binarias soportadas durante el preprocesamiento y permite utilizarlas en los análisis correspondientes, aplicando las transformaciones necesarias según la técnica seleccionada.
Resultado esperado	El sistema debe validar la estructura del dataset, mantener las variables compatibles, permitir su selección en la interfaz y ejecutar el análisis utilizando dichas variables, mostrando tablas, métricas y gráficas actualizadas.
Resultado obtenido	El sistema permite trabajar con variables no binarias compatibles, las integra en el flujo de análisis y muestra los resultados generados de forma coherente, facilitando la interpretación del efecto de dichas variables sobre el estudio realizado.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 29: Prueba válida del CU-7: AnalizarVariablesNoBinarias



Figura 69: Evidencia de análisis de variables en el CU-7: AnalizarVariablesNoBinarias

Prueba errónea

CAPÍTULO 11. PRUEBAS

Caso de prueba	Intentar analizar una variable no compatible con la técnica seleccionada o un dataset que no contiene variables adicionales válidas para el análisis.
Objetivo de la prueba	Comprobar que el sistema valida las variables seleccionadas antes de ejecutar el análisis y evita generar resultados cuando la variable no puede transformarse o utilizarse correctamente.
Resultado esperado	El sistema debe informar al usuario de que la variable no es compatible con el análisis seleccionado o impedir la ejecución del proceso, evitando mostrar tablas, métricas o gráficas basadas en datos no válidos.
Resultado obtenido	El sistema responde de forma controlada cuando una variable no es apta para el análisis, evitando resultados inconsistentes y manteniendo el flujo de trabajo en un estado estable.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con la validación de variables definida en el flujo alternativo.
Solución	No se requiere ninguna corrección. La acción adecuada es seleccionar variables compatibles o cargar un dataset que contenga variables válidas para el análisis.

Tabla 30: Prueba errónea del CU-7: AnalizarVariablesNoBinarias

11.8. CU-8: MejorarInterpretacionIA

Prueba válida

CAPÍTULO 11. PRUEBAS

Caso de prueba	Solicitar una interpretación mediante inteligencia artificial a partir de resultados generados previamente por un análisis del sistema.
Objetivo de la prueba	Verificar que el sistema recopila correctamente la información relevante del análisis, la envía al módulo de inteligencia artificial y muestra una explicación comprensible para el usuario.
Resultado esperado	El sistema debe permitir solicitar la explicación, construir el resumen de entrada con los resultados disponibles, enviarlo al módulo de IA y presentar en la interfaz una respuesta textual coherente, útil y relacionada con el análisis realizado.
Resultado obtenido	El sistema genera la interpretación solicitada y la muestra en pantalla, ofreciendo una explicación más clara de los resultados obtenidos y facilitando su lectura por parte del usuario.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 31: Prueba válida del CU-8: MejorarInterpretacionIA

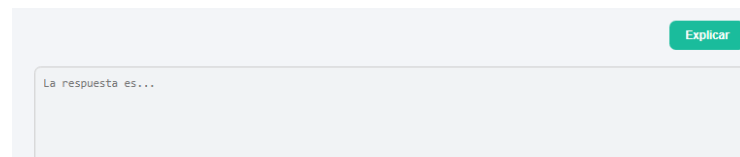


Figura 70: Evidencia de acceso a la interpretación mediante IA del CU-8: MejorarInterpretacionIA

Prueba errónea

CAPÍTULO 11. PRUEBAS

Caso de prueba	Solicitar una interpretación cuando no existen resultados previos suficientes o cuando el módulo de inteligencia artificial no está disponible.
Objetivo de la prueba	Comprobar que el sistema controla los errores asociados a la generación de respuestas y evita mostrar explicaciones vacías, incompletas o no relacionadas con el análisis.
Resultado esperado	El sistema debe informar al usuario de que no ha sido posible generar una respuesta adecuada, manteniendo la interfaz estable y sin bloquear el resto de funcionalidades.
Resultado obtenido	El sistema responde de forma controlada cuando la IA no puede generar una interpretación válida, evitando presentar una explicación incorrecta y conservando el funcionamiento normal de la aplicación.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con el flujo alternativo definido para este caso de uso.
Solución	No se requiere ninguna corrección. La acción adecuada es disponer de resultados válidos y comprobar que el módulo de inteligencia artificial se encuentra disponible antes de solicitar la interpretación.

Tabla 32: Prueba errónea del CU-8: MejorarInterpretacionIA

Medición de rendimiento

Además de comprobar que el módulo de inteligencia artificial genera respuestas válidas, se realizó una medición específica de rendimiento. La verdad es que esta prueba resulta importante porque la IA no debe convertirse en una espera incómoda dentro del flujo de análisis; su función es acompañar la interpretación de los resultados, no interrumpir el trabajo del usuario. Para ello, se ejecutaron cinco consultas por cada técnica de análisis disponible, utilizando el servidor local `llama-server` y el modelo `Qwen2.5-1.5B-Instruct` en formato `GGUF`. Los resultados se registraron automáticamente mediante el script `benchmark_ia.py`, quedando almacenados en el archivo `benchmarks/ia_benchmark_20260506_175802.csv`.

CAPÍTULO 11. PRUEBAS

Elemento	Valor utilizado en la prueba
Modelo evaluado	qwen2.5-1.5b-instruct mediante llama-server.
Modelo del sistema previo	Llama 3 mediante Ollama, tomado como referencia tecnológica del TFG anterior.
Equipo de pruebas	Ordenador personal descrito en el apartado de recursos hardware: Lenovo 82KU, AMD Ryzen 7 5700U with Radeon Graphics, 8 GB de memoria RAM, Windows 11 Home de 64 bits y tarjeta gráfica AMD Radeon Graphics integrada.
Modo de ejecución	Inferencia local en CPU, con servidor HTTP en http://127.0.0.1:8000 .
Número de consultas	30 consultas medidas en total: 5 por cada una de las 6 técnicas de análisis.

Tabla 33: Configuración utilizada para medir el rendimiento del módulo de inteligencia artificial

Técnica	N	Tiempo medio (s)	Tiempo máximo (s)	RAM máx. aprox. (MB)
Kaplan-Meier	5	3.877	5.340	1013.32
Regresión de Cox	5	8.553	12.468	1049.43
Test Log-Rank	5	4.566	4.927	1052.55
Weibull	5	6.361	8.591	1073.43
Exponencial	5	5.352	7.558	1097.62
Random Survival Forest	5	3.976	4.526	1098.16
Media global	30	5.448	12.468	1098.16

Tabla 34: Resultados de rendimiento del modelo de inteligencia artificial por técnica de análisis

Los resultados muestran que el modelo actual ofrece tiempos de respuesta asumibles para un uso interactivo. La media global se sitúa en 5.448 segundos, con un máximo de 12.468 segundos en una consulta asociada a la regresión de Cox, que es precisamente una de las técnicas que genera entradas más densas para la explicación. El consumo máximo aproximado de memoria se mantiene alrededor de 1.1 GB, una cifra razonable para una ejecución local en un equipo portátil con recursos limitados.

En comparación con el enfoque del sistema previo, basado en Llama 3 mediante Ollama, esta nueva configuración apuesta por un modelo más ligero y por una ejecución local controlada mediante llama-server. Aunque la comparación exacta con el TFG anterior depende del equipo y de la configuración concreta usada en aquel entorno, la medición realizada confirma que la solución actual responde de forma

CAPÍTULO 11. PRUEBAS

estable en todas las técnicas evaluadas y que puede integrarse en el dashboard sin romper la continuidad del análisis.

11.9. CU-9:AmpliarVisualizacionesCoxTestLogRank

Prueba válida

Caso de prueba	Generar nuevas visualizaciones asociadas a los análisis de regresión de Cox y Test de Log-Rank con resultados previamente disponibles.
Objetivo de la prueba	Verificar que el sistema permite acceder a las opciones de visualización, procesar los resultados activos y mostrar en pantalla las representaciones gráficas ampliadas para facilitar la interpretación del análisis.
Resultado esperado	El sistema debe mostrar las opciones de visualización disponibles, obtener los resultados correspondientes a la configuración activa y generar la gráfica solicitada, dejándola visible para su consulta, interpretación o exportación posterior.
Resultado obtenido	El sistema genera correctamente las visualizaciones asociadas a Cox y al Test de Log-Rank, mostrando las gráficas en la interfaz y permitiendo al usuario analizar de forma más clara los resultados obtenidos.
Problemas encontrados	No se han encontrado problemas durante la ejecución de la prueba válida.
Solución	No es necesario aplicar ninguna acción correctiva.

Tabla 35: Prueba válida del CU-9: AmpliarVisualizacionesCoxTestLogRank

CAPÍTULO 11. PRUEBAS

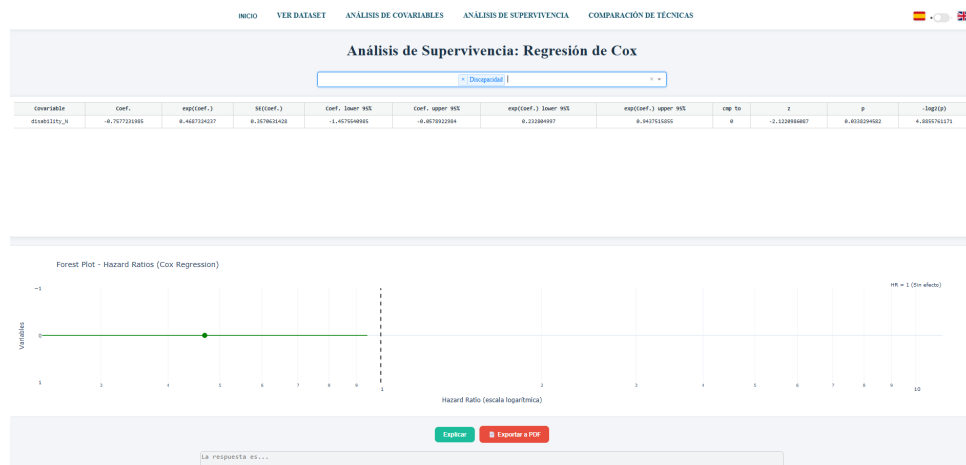


Figura 71: Evidencia de visualización ampliada del CU-9:
AmpliarVisualizacionesCoxTestLogRank

Prueba errónea

CAPÍTULO 11. PRUEBAS

Caso de prueba	Intentar generar una visualización de Cox o Test de Log-Rank sin haber ejecutado previamente el análisis correspondiente o sin resultados disponibles.
Objetivo de la prueba	Comprobar que el sistema valida la existencia de resultados previos antes de construir la gráfica y que informa al usuario cuando no es posible generar la visualización solicitada.
Resultado esperado	El sistema debe impedir la generación de la gráfica cuando no existan datos suficientes y mostrar un aviso indicando que es necesario ejecutar previamente el análisis de Cox o el Test de Log-Rank.
Resultado obtenido	El sistema mantiene un comportamiento controlado cuando no existen resultados previos, evitando mostrar visualizaciones vacías o incoherentes y guiando al usuario hacia la ejecución previa del análisis necesario.
Problemas encontrados	No se han encontrado fallos críticos. El comportamiento observado es coherente con el flujo alternativo definido para este caso de uso.
Solución	No se requiere ninguna corrección. La acción adecuada es ejecutar primero el análisis correspondiente y, una vez generados los resultados, solicitar la visualización deseada.

Tabla 36: Prueba errónea del CU-9: AmpliarVisualizacionesCoxTestLogRank

12 Discusión y conclusiones finales

Este último capítulo recoge una visión global del trabajo realizado. Se valora el grado de cumplimiento de los objetivos planteados inicialmente, las dificultades encontradas durante el desarrollo y las posibles líneas de mejora que podrían abordarse en el futuro. La verdad es que este cierre resulta especialmente importante en un proyecto de ampliación, porque permite distinguir con más claridad qué aportaba ya el sistema original y qué se ha incorporado para hacerlo más completo, flexible y útil.

El trabajo desarrollado ha partido de una aplicación previa orientada al análisis de supervivencia aplicado al abandono académico universitario. Sobre esa base se han añadido nuevos modelos, nuevas visualizaciones, soporte multilingüe, exportación de informes, análisis de variables no binarias y una integración más cuidada con la interpretación mediante inteligencia artificial. En conjunto, la aplicación resultante no sustituye el dashboard anterior, sino que lo amplía de forma progresiva y coherente con los objetivos definidos en el anteproyecto.

12.1. Análisis de los objetivos iniciales

El objetivo general era ampliar y mejorar una aplicación interactiva para el análisis de supervivencia aplicado al abandono académico universitario. Ese objetivo puede considerarse cumplido, aunque conviene ir más allá de la afirmación genérica y revisar qué ha aportado cada pieza.

El módulo de inteligencia artificial es probablemente la mejora más visible para el usuario. La aplicación permite solicitar interpretaciones asociadas a los resultados y presentarlas dentro de la propia interfaz, lo que añade una capa explicativa muy útil para quienes no dominan todos los detalles estadísticos de los modelos. Aun así, se ha mantenido un criterio claro desde el principio: la IA apoya la interpretación, no la sustituye. El análisis estadístico sigue siendo el núcleo, y la IA lo acompaña.

El soporte multilingüe en castellano e inglés puede parecer un detalle menor, pero en la práctica mejora la accesibilidad de la herramienta de forma real. Una

CAPÍTULO 12. DISCUSIÓN Y CONCLUSIONES FINALES

aplicación que solo funciona en un idioma tiene un alcance limitado. Con este cambio, el dashboard puede usarse en contextos académicos más amplios sin fricciones innecesarias.

Ampliar el conjunto de variables fue otro objetivo relevante, y probablemente uno de los que más han enriquecido el análisis. Se han incorporado covariables como el género, la discapacidad, el tramo de edad, el nivel educativo previo y los créditos cursados. Esta última, `studied_credits`, merece mención especial: no es binaria, lo que obligó a aplicar estrategias de agrupación y transformación según la técnica utilizada. Ese esfuerzo adicional se nota en los resultados, que ahora reflejan una realidad más matizada.

En cuanto a las técnicas estadísticas, la incorporación de los modelos Exponencial y Weibull junto con Random Survival Forest completa el panorama de forma coherente. El usuario puede ahora comparar una aproximación simple, un modelo paramétrico flexible y un enfoque basado en aprendizaje automático, todo dentro de la misma herramienta. Esa capacidad de comparación es, en sí misma, uno de los valores añadidos más claros del proyecto.

Por último, la exportación de informes PDF cierra el ciclo de forma práctica. No se trata solo de que los resultados sean visibles en pantalla, sino de que puedan llevarse fuera de la aplicación, compartirse, documentarse y usarse en contextos académicos o de gestión. Las pruebas realizadas confirman que la exportación funciona de forma controlada, evitando generar documentos vacíos cuando no se cumplen las condiciones necesarias.

12.2. Problemas encontrados en el desarrollo

No todo ha ido sobre ruedas, y sería poco honesto no reconocerlo. Algunos problemas eran esperables; otros aparecieron donde menos se anticipaban.

El primero y más transversal fue el de ampliar sin romper. Trabajar sobre una aplicación ya existente implica respetar lo que había: los callbacks de Dash, las páginas, el estado del dataset, las salidas ya definidas. Cada nueva funcionalidad debía encajar en esa estructura sin desestabilizarla. Eso obligó a revisar con calma la arquitectura del sistema antes de tocar nada, y a mantener una separación clara entre interfaz, análisis, visualización, exportación e interpretación.

CAPÍTULO 12. DISCUSIÓN Y CONCLUSIONES FINALES

La preparación de los datos también tuvo más peso del previsto. Las nuevas técnicas no podían aplicarse directamente sobre cualquier tabla: antes era necesario validar columnas, transformar tipos, eliminar registros incompletos y gestionar los casos en los que no había información suficiente para ajustar un modelo. Y cada variable tenía su particularidad. `studied_credits` fue la más exigente, precisamente porque rompía el esquema binario en el que estaban pensadas muchas partes del sistema original.

Integrar librerías con requisitos distintos fue otro punto de fricción. `lifelines` para los modelos paramétricos y `scikit-survival` para Random Survival Forest no siempre se comportan igual ni devuelven resultados en el mismo formato. En el caso de RSF, construir la matriz de características y trabajar con estructuras específicas para datos censurados requirió más tiempo del esperado. La regresión de Cox también dio algún susto puntual con problemas de convergencia según las covariables seleccionadas, algo que se resolvió incorporando regularización y mecanismos de control.

La exportación PDF fue más compleja de lo que parecía a primera vista. Generar documentos con tablas, gráficos, textos e interpretaciones de forma estable no es trivial. Combinar figuras de Plotly con Kaleido y ReportLab, controlar tamaños y orientación de páginas, gestionar el contenido seleccionado por el usuario: cada detalle sumaba. Y el sistema tenía que comportarse bien tanto con resultados completos como con salidas parciales.

Por último, la integración de la IA introdujo una dependencia práctica inevitable. El modelo Qwen se ejecuta de forma local mediante `llama-server`, lo que tiene la ventaja de no depender de servicios externos, pero también implica que el servidor debe estar activo antes de solicitar una explicación. Gestionar esos errores de conexión, tiempos de espera y respuestas vacías sin que la aplicación se bloqueara fue una parte del desarrollo que no estaba del todo prevista al inicio.

12.3. Conclusiones y posibles mejoras

Mirando el resultado final, lo que más satisface no es la lista de funcionalidades añadidas, sino el hecho de que el proyecto tiene sentido como conjunto. La ampliación es coherente con el sistema original, las nuevas piezas encajan con las antiguas y el resultado es una herramienta más completa sin haber perdido lo que ya funcionaba.

Dicho eso, quedan cosas por hacer. Siempre quedan.

CAPÍTULO 12. DISCUSIÓN Y CONCLUSIONES FINALES

Una línea natural de mejora sería profundizar en el conjunto de variables analizadas, incorporando información sobre actividad en la plataforma, intentos previos, módulos cursados o indicadores socioeconómicos. Están en los datos, y podrían aportar mucho si se trabajan con cuidado.

También tendría sentido añadir validaciones más avanzadas de los modelos: métricas comparativas adicionales, validación cruzada, análisis de estabilidad. Eso permitiría valorar con más rigor qué técnica funciona mejor según el tipo de datos y el objetivo del usuario, en lugar de dejar esa decisión completamente en manos de quien usa la herramienta.

La interpretación mediante IA tiene también recorrido por delante. En una futura versión podría ser más contextualizada: no solo explicar qué resultado se ha obtenido, sino advertir sobre las precauciones que hay que tener al interpretarlo según el modelo y las variables elegidas. Una IA que no solo responde, sino que también avisa.

Y por último, el despliegue. Ahora mismo el sistema funciona en local, lo que limita su alcance real. Evolucionar hacia una aplicación en servidor, con gestión de usuarios, persistencia de análisis y almacenamiento de informes, abriría la puerta a un uso institucional genuino. Ese salto no es trivial, pero tampoco está tan lejos.

En definitiva, este trabajo ha conseguido lo que se proponía: acercar modelos estadísticos complejos a una interfaz comprensible y útil para explorar un problema educativo real. Eso es lo que queda cuando se apaga el ordenador, no las líneas de código ni las librerías instaladas, sino una herramienta que alguien podría usar de verdad para entender mejor por qué los estudiantes abandonan y, quizás, para hacer algo al respecto.

Bibliografía

- [1] Cameron Davidson-Pilon. lifelines: survival analysis in Python. *Journal of Open Source Software*, 4(40):1317, 2019. doi: 10.21105/joss.01317. URL <https://doi.org/10.21105/joss.01317>.
- [2] Christopher Jackson. flexsurv: A platform for parametric survival modeling in R. *Journal of Statistical Software*, 70(8):1–33, 2016. doi: 10.18637/jss.v070.i08. URL <https://www.jstatsoft.org/article/view/v070i08>.
- [3] Michaelg2015. Exponential survival functions. https://commons.wikimedia.org/wiki/File:Exponential_survival_functions.svg, 2016. Imagen publicada en Wikimedia Commons bajo licencia Creative Commons Attribution-Share Alike 4.0 International. Último acceso: 5 de mayo de 2026.
- [4] Calimo. Weibull probability density function. https://commons.wikimedia.org/wiki/File:Weibull_PDF.svg, 2010. Imagen publicada en Wikimedia Commons. Último acceso: 5 de mayo de 2026.
- [5] Hemant Ishwaran, Udaya B. Kogalur, Eugene H. Blackstone, and Michael S. Lauer. Random survival forests. *The Annals of Applied Statistics*, 2(3):841–860, 2008. doi: 10.1214/08-AOAS169. URL <https://projecteuclid.org/journals/annals-of-applied-statistics/volume-2/issue-3/Random-survival-forests/10.1214/08-AOAS169.full>.
- [6] Sebastian Pölsterl. scikit-survival: A library for time-to-event analysis built on top of scikit-learn. *Journal of Machine Learning Research*, 21(212):1–6, 2020. URL <https://jmlr.org/papers/v21/20-729.html>.
- [7] scikit-survival developers. Using Random Survival Forests. https://scikit-survival.readthedocs.io/en/v0.14.0/user_guide/random-survival-forest.html, s. f. Documentación oficial de scikit-survival. Último acceso: 5 de mayo de 2026.
- [8] Alfonso Muñoz Cuesta. Repositorio del Trabajo Fin de Grado. <https://github.com/alfonsoMunozCuesta/TFG.git>, 2026. Último acceso: 5 de mayo de 2026.
- [9] Roger S. Pressman and Bruce R. Maxim. *Ingeniería del Software: Un enfoque práctico*. McGraw-Hill, 8 edition, 2015.

BIBLIOGRAFÍA

- [10] An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Yang, Keqin Chen, Kexin Yang, Mingfeng Xue, Na Li, Pei Zhang, Peng Wang, Rui Men, Ruize Gao, Runji Lin, Shijie Wang, Shuai Bai, Sinan Tan, Tianhang Zhu, Tianhao Li, Tianyu Liu, Wenbin Ge, Xiaodong Deng, Xiaohuan Zhou, Xingzhang Ren, Xinyu Ren, Xipin Wei, Xuancheng Ren, Yang Liu, Yang Fan, Yang Yao, Yichang Zhang, Yu Wan, Yunfei Chu, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zhihao Fan. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024. URL <https://arxiv.org/abs/2412.15115>.
- [11] Qwen Team. Qwen2.5-1.5B-Instruct-GGUF. <https://huggingface.co/Qwen/Qwen2.5-1.5B-Instruct-GGUF>, 2024. Repositorio oficial del modelo en formato GGUF. Último acceso: 5 de mayo de 2026.
- [12] ggml-org. llama.cpp: LLM inference in C/C++. <https://github.com/ggml-org/llama.cpp>, 2026. Repositorio oficial. Último acceso: 5 de mayo de 2026.
- [13] ggml-org. llama.cpp server documentation. <https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md>, 2026. Documentación oficial de llama-server. Último acceso: 5 de mayo de 2026.

ANEXO A
MANUAL DE USUARIO

A Anexo del Manual de Usuario

A.1. Introducción

Este manual recoge, paso a paso, el proceso de instalación, puesta en marcha y uso de la aplicación desarrollada para el análisis de supervivencia aplicado al abandono académico universitario. La idea es que cualquier usuario pueda arrancarla sin perderse entre archivos, comandos o pantallas intermedias.

La aplicación se ejecuta como un dashboard web local construido con Dash. Esto significa que no es necesario desplegarla en un servidor externo para utilizarla: el usuario inicia el programa en su propio equipo y accede a la interfaz desde un navegador mediante una dirección local. Desde ahí puede cargar el dataset, preprocesarlo, consultar los datos, ejecutar diferentes técnicas de análisis de supervivencia, generar interpretaciones y exportar informes en formato PDF. En la práctica, todo el flujo de trabajo queda reunido en una misma herramienta.

El módulo de inteligencia artificial es opcional. La aplicación puede funcionar sin él, manteniendo disponibles la carga de datos, los análisis estadísticos, las gráficas y la exportación de informes. Ahora bien, si se desea utilizar la generación automática de explicaciones, es necesario arrancar previamente el servidor local de IA mediante el script correspondiente.

A.2. Instalación

A.2.1. Requisitos previos

Antes de ejecutar la aplicación, conviene comprobar con calma que el equipo dispone de los siguientes elementos. Es un paso sencillo, pero evita muchos errores posteriores:

- Sistema operativo Windows. Es el entorno principal para el que se han preparado los scripts de arranque.
- Python instalado. El proyecto se ha desarrollado y probado con Python 3.13.2, por lo que se recomienda utilizar esa misma versión para reproducir el entorno con mayor seguridad.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

- Gestor de paquetes `pip`, incluido normalmente con Python. En Windows, la forma más fiable de utilizarlo es mediante `python -m pip`.
- Navegador web actualizado, por ejemplo Google Chrome, Microsoft Edge o Firefox.
- Código fuente del proyecto descargado en una carpeta local.
- Conexión a Internet durante la instalación de dependencias, salvo que se disponga previamente de los paquetes descargados.

Las librerías necesarias se encuentran recogidas en el archivo `requirements.txt`. Entre las dependencias principales se incluyen:

- `dash==3.1.0` y `dash-daq==0.6.0`, utilizadas para construir la interfaz web.
- `pandas==2.2.3` y `numpy==2.2.3`, utilizadas para la carga y transformación de datos.
- `plotly==6.2.0` y `matplotlib==3.10.1`, utilizadas para la generación de gráficas.
- `lifelines==0.30.0`, utilizada para Kaplan-Meier, Cox, Weibull y Exponencial.
- `scikit-survival==0.27.0`, utilizada para Random Survival Forest.
- `reportlab==4.0.8` y `kaleido==0.2.1`, utilizadas para la exportación de informes PDF.
- `requests==2.32.3`, utilizada para la comunicación con el servidor local de inteligencia artificial.
- `Pillow==11.1.0`, utilizada como soporte para el tratamiento de imágenes.

La versión recomendada de Dash para una instalación nueva es `dash==3.1.0`. Aun así, el arranque de la aplicación se ha preparado de forma compatible con instalaciones que todavía tengan Dash 2, utilizando el método de ejecución disponible en cada caso. Esto no cambia la instalación recomendada, pero da un pequeño margen si el usuario está probando el proyecto en un equipo donde ya existía un entorno virtual anterior.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

Para utilizar el módulo de inteligencia artificial se requiere, además, disponer de `llama-server.exe` y de un modelo en formato GGUF. En la configuración actual se utiliza un modelo Qwen2.5 Instruct cuantizado. La verdad es que esta parte puede parecer la más delicada de la instalación, pero solo afecta a las explicaciones automáticas: el resto del dashboard puede usarse sin tener activa la IA.

A.2.2. Descarga del código

El código fuente del proyecto se obtiene desde el repositorio de GitHub correspondiente. Para ello existen dos caminos habituales: descargar el proyecto como archivo comprimido ZIP o clonar el repositorio mediante Git. La primera opción es más directa; la segunda resulta más cómoda si se quiere mantener el proyecto conectado con el repositorio.

Opción 1: descarga como archivo comprimido ZIP. La primera opción consiste en acceder al repositorio desde el navegador, pulsar sobre el botón **Code** y seleccionar la opción **Download ZIP**. Tras la descarga, se debe descomprimir el archivo en una carpeta local. Esta alternativa es la más sencilla si el usuario solo quiere ejecutar la aplicación, probar el dashboard y no necesita modificar el repositorio.

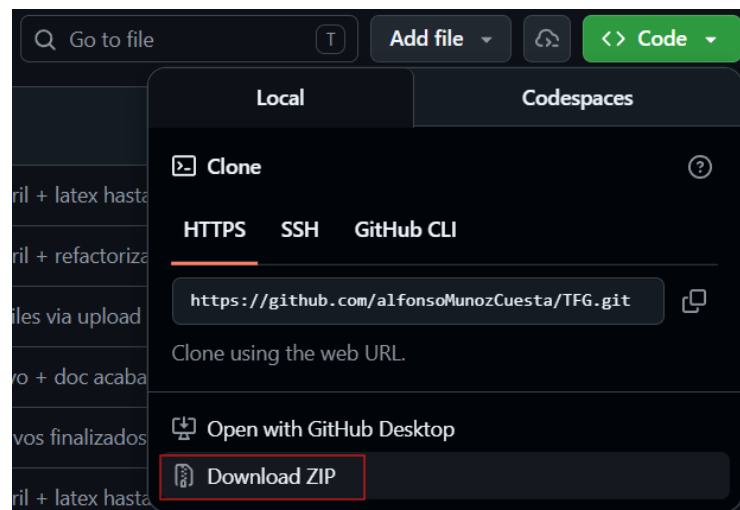


Figura 72: Descarga del proyecto desde GitHub como archivo ZIP

Opción 2: clonación del repositorio mediante Git. La segunda opción consiste en clonar el repositorio mediante Git. Para ello, el usuario debe tener Git instalado y ejecutar desde una terminal el siguiente comando:

```
git clone https://github.com/alfonsoMunozCuesta/TFG.git
```

Esta alternativa es recomendable si se desea mantener el proyecto vinculado a GitHub, actualizarlo posteriormente o trabajar con control de versiones. Además, resulta especialmente útil si el proyecto va a revisarse o ampliarse más adelante.

Una vez descargado, se debe abrir una terminal en la carpeta raíz del proyecto, es decir, en la carpeta donde se encuentran archivos como `cargaDataset.py`, `requirements.txt`, `START_DASH_APP.bat` y `START_LLAMA_SERVER.bat`.

A.2.3. Instalación de las librerías de Python requeridas

Se recomienda instalar las dependencias dentro de un entorno virtual para no mezclar las librerías del proyecto con las del sistema. Es una pequeña precaución, pero ayuda bastante a mantener el entorno limpio. Desde la carpeta raíz del proyecto, se pueden ejecutar los siguientes comandos en PowerShell:

```
cd TFG
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

El último comando instala todas las librerías declaradas en `requirements.txt`. La verdad es que esta parte conviene hacerla con un poco de cuidado, porque `scikit-survival` es una dependencia más sensible que otras librerías habituales. En este proyecto se ha fijado la versión 0.27.0, que es la utilizada durante las pruebas con Python 3.13.2.

A.2.4. Ejecución de la aplicación

Una vez instaladas las dependencias, la aplicación puede iniciarse desde la carpeta raíz del proyecto mediante:

```
python .\cargaDataset.py
```

ANEXO A. ANEXO DEL MANUAL DE USUARIO

También puede utilizarse el script de arranque preparado para Windows:

```
.\START_DASH_APP.bat
```

Ambas opciones arrancan el servidor local de Dash. Cuando el proceso se inicia correctamente, la aplicación queda disponible en el navegador a través de la siguiente dirección. Es, por así decirlo, la puerta de entrada al dashboard:

```
http://127.0.0.1:8050
```

Para detener la aplicación basta con volver a la terminal donde se está ejecutando y pulsar **Ctrl+C**. Si se utiliza el archivo **START_DASH_APP.bat**, la ventana mostrará también un mensaje final antes de cerrarse.

A.2.5. Ejecución del módulo de inteligencia artificial

El módulo de inteligencia artificial se ejecuta de forma independiente al dashboard. Para utilizarlo, debe abrirse una terminal adicional en la carpeta raíz del proyecto y ejecutar:

```
.\START_LLAMA_SERVER.bat
```

Este script inicia **llama-server** en la dirección local **http://127.0.0.1:8000**. La aplicación Dash se comunica con ese servidor cuando el usuario solicita una interpretación automática desde la interfaz o al generar un informe PDF con explicación de IA. Por eso, si se quiere usar esta parte, ambas ventanas deben permanecer abiertas.

Por defecto, el script espera encontrar el modelo y el ejecutable en las siguientes rutas relativas al proyecto:

- **models/qwen2.5-1.5b-instruct-q4_k_m.gguf**
- **tools/llama.cpp/llama-server.exe**

ANEXO A. ANEXO DEL MANUAL DE USUARIO

Estos dos archivos no se incluyen directamente en el repositorio por su tamaño y por tratarse de recursos externos al código fuente. Por ese motivo, deben entregarse aparte, por ejemplo mediante una carpeta compartida en Drive, y copiarse manualmente en las rutas indicadas antes de arrancar el servidor de inteligencia artificial. Es un paso pequeño, pero importante: si alguno de los dos archivos no está en su sitio, el dashboard seguirá funcionando, aunque la generación de explicaciones mediante IA no estará disponible.

Los enlaces de descarga empleados para esta instalación son los siguientes:

- Modelo GGUF: enlace de descarga del modelo.
- Ejecutable `llama-server.exe`: enlace de descarga del ejecutable.

Si el usuario tiene el modelo o el ejecutable en otra ubicación, puede modificar las variables `MODEL_PATH` y `LLAMA_SERVER` antes de ejecutar el script. Si el servidor de IA no está activo, la aplicación seguirá funcionando con normalidad, aunque no se generarán explicaciones automáticas.

A.2.6. Desinstalación del proyecto

La aplicación no requiere un proceso de desinstalación complejo, ya que se ejecuta desde la carpeta del proyecto. Para retirarla del equipo basta con cerrar la aplicación, borrar la carpeta del proyecto y, si se creó un entorno virtual, eliminar también la carpeta `.venv`. No deja una instalación repartida por el sistema.

Si se descargaron por separado el modelo GGUF o `llama-server.exe`, estos archivos también pueden eliminarse manualmente si ya no van a utilizarse. En caso de haber instalado dependencias dentro de un entorno virtual, no es necesario desinstalar cada librería una por una, ya que al borrar dicho entorno se eliminan de forma conjunta.

A.3. Interfaz de usuario

Una vez iniciada la aplicación y abierta la dirección local en el navegador, el usuario accede a la interfaz principal del dashboard. La aplicación está organizada en distintas páginas conectadas mediante una barra de navegación superior. Desde ella se puede volver al inicio, consultar el dataset, acceder al análisis de covariables y entrar en los módulos de análisis de supervivencia. Dentro de esta última sección se incluye también el acceso a la comparación de técnicas. La navegación está pensada

ANEXO A. ANEXO DEL MANUAL DE USUARIO

para que el recorrido sea bastante natural: primero se cargan los datos, después se revisan y, finalmente, se analizan.

Las capturas incluidas en este manual tienen una función práctica: muestran al usuario qué pantalla debe localizar durante el flujo de uso. El capítulo de diseño recoge la distribución visual y los componentes de la interfaz con mayor detalle, mientras que este anexo se centra en acompañar el uso real de la aplicación, casi como una guía de trabajo.

A.3.1. Pantalla principal

La pantalla principal es el punto de entrada de la aplicación. En ella se muestra el área de carga del archivo CSV y el botón que permite iniciar el preprocesamiento del dataset. Esta vista concentra el primer paso importante del flujo de trabajo: seleccionar el archivo de datos que después utilizarán todos los módulos del sistema.

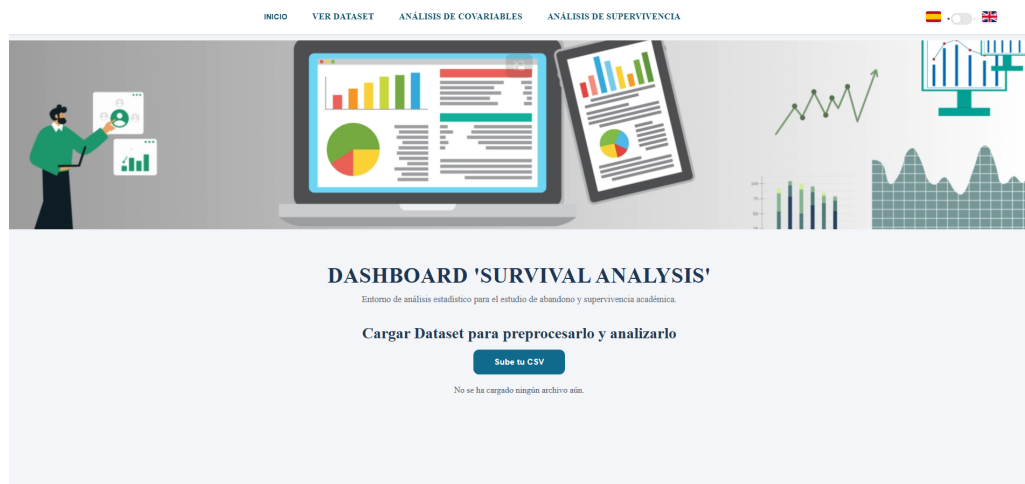


Figura 73: Pantalla principal de la aplicación

Antes de acceder a los módulos de análisis, es necesario cargar y preprocesar un dataset válido. Si el usuario intenta entrar en una sección que requiere datos sin haber completado este paso, la aplicación muestra un mensaje informativo indicando que primero debe cargarse el dataset académico. Es una protección sencilla, pero evita trabajar sobre datos vacíos o incompletos.

A.3.2. Selector de idioma

La interfaz incorpora un selector de idioma situado en la barra superior. Este control permite alternar entre español e inglés sin reiniciar la aplicación. Al modificar el idioma, se actualizan los textos principales de navegación, botones, mensajes y módulos de análisis.

El idioma seleccionado también se tiene en cuenta en varias salidas generadas por el sistema, como interpretaciones textuales o informes PDF. De esta forma, el cambio de idioma no afecta únicamente a los menús, sino también a parte del contenido producido por la aplicación, algo importante cuando los resultados van a compartirse con otras personas.



Figura 74: Botón que cambia el idioma de la aplicación

A.3.3. Carga del dataset

Para comenzar el análisis, el usuario debe cargar un archivo CSV desde la pantalla principal. La aplicación está preparada para trabajar con un dataset académico que contenga las columnas necesarias para el análisis de supervivencia. Entre las variables fundamentales se encuentran el identificador del estudiante, el tiempo observado y el resultado final que permite determinar si se ha producido el evento de abandono.

La carga se realiza seleccionando el archivo desde el explorador del sistema o arrastrándolo sobre el área correspondiente. Una vez cargado, la aplicación valida que el archivo tenga un formato compatible y que contenga los campos mínimos requeridos. Si el archivo es válido, se muestra una confirmación y el usuario puede continuar con el preprocesamiento sin tener que revisar manualmente toda la estructura del CSV.

[illegible]

Figura 75: Archivo CSV cargado antes de iniciar el preprocesamiento

Si el archivo no es correcto, está vacío o no contiene las columnas esperadas, el sistema muestra un mensaje de error. En ese caso, el usuario debe revisar el archivo y repetir la carga con un dataset compatible.

A.3.4. Preprocesamiento del dataset

Tras cargar un archivo válido, el usuario debe pulsar el botón de preprocesamiento. Este paso prepara el dataset para que pueda ser utilizado por las técnicas de análisis de supervivencia. Durante el proceso se validan columnas, se transforman variables, se eliminan registros no válidos y se genera una versión limpia de los datos. Es una especie de filtro previo: antes de analizar, el sistema deja los datos en una forma más estable.



Figura 76: Confirmación del dataset preprocesado

Cuando el preprocesamiento finaliza correctamente, la aplicación permite acceder al resto de módulos. A partir de ese momento, el dataset limpio queda disponible para consultar los datos, ejecutar análisis de covariables, aplicar técnicas de supervivencia y generar informes. En otras palabras, es el punto en el que el dashboard queda realmente preparado para trabajar.

A.3.5. Navegación entre módulos

La navegación principal se realiza desde la barra superior de la aplicación. Los accesos disponibles permiten desplazarse entre las partes principales del dashboard:

- **INICIO:** vuelve a la pantalla principal de carga del dataset.
- **VER DATASET:** muestra el dataset limpio y la descripción de las variables.
- **ANÁLISIS DE COVARIABLES:** permite explorar variables académicas y personales antes de aplicar técnicas de supervivencia.
- **ANÁLISIS DE SUPERVIVENCIA:** da acceso a Kaplan-Meier, Regresión de Cox, Test Log-Rank, Weibull, Exponencial, Random Survival Forest y, desde un botón interno situado bajo las técnicas, a la comparación general entre métodos.

Al volver a la pantalla inicial, la aplicación puede solicitar confirmación para evitar perder el dataset cargado y los análisis realizados. Esta medida reduce el riesgo de abandonar accidentalmente el flujo de trabajo activo, algo que puede pasar con facilidad cuando se está explorando varias secciones seguidas.

A.4. Visualización del dataset

Después de completar el preprocesamiento, el usuario puede acceder a la sección de visualización del dataset. Esta parte permite revisar los datos cargados y comprobar la estructura del dataset limpio que utilizarán los modelos de supervivencia. Es un paso recomendable antes de lanzarse al análisis, porque ayuda a detectar si algo no encaja.

A.4.1. Consulta del dataset cargado

La consulta del dataset cargado permite comprobar que el archivo original se ha incorporado correctamente a la aplicación. Esta revisión es útil para detectar posibles problemas iniciales, como columnas ausentes, separadores incorrectos o valores inesperados, antes de continuar con el análisis.

En esta fase el usuario debe verificar que el dataset corresponde al archivo esperado y que contiene las variables necesarias para el análisis. Si el archivo no cumple la estructura requerida, el sistema no permitirá continuar correctamente con el flujo de análisis.

A.4.2. Consulta del dataset preprocesado

Una vez aplicado el preprocesamiento, el usuario puede consultar el dataset limpio. Esta versión es la que se utiliza internamente para ejecutar las técnicas de análisis de supervivencia. En ella se han tratado las variables necesarias y se han preparado los datos para su uso en los modelos estadísticos.

La vista de dataset muestra una tabla con los registros disponibles y permite revisar el resultado del proceso de limpieza. Además, la aplicación incluye una descripción de las variables principales para facilitar la interpretación de los campos utilizados por los modelos. Esto ayuda a no tratar las columnas como nombres sueltos, sino como información con significado dentro del problema académico.

INGO

VER DATASET

ANÁLISIS DE COVARIABLES

ANÁLISIS DE SUPERVIVENCIA

Dataset Limpio

--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--	--

Figura 77: Consulta del dataset limpio desde la sección de visualización

Variable	Valores	Significado
Event (final_result)	0 o 1	Representa si ocurrió el evento en el análisis de supervivencia: 1 = abandono (evento), 0 = censura (no abandono durante el seguimiento).
id_student	Entero (ID)	Identificador único del estudiante.
date	Número >= 0	Tiempo de seguimiento usado en supervivencia (por ejemplo, días).
final_result	0 o 1	Indicador de evento: 1 = abandono (evento), 0 = censurado sin abandono.
gender_F	0 o 1	Otros codificado: 1 = femenino, 0 = masculino.
disability_N	0 o 1	Discapacidad codificada: 1 = con discapacidad, 0 = sin discapacidad.
age_band_0-35	0 o 1	Indicador one-hot del grupo de edad 0-35.
age_band_35-55	0 o 1	Indicador one-hot del grupo de edad 35-55.
age_band_55+	0 o 1	Indicador one-hot del grupo de edad 55+.
highest_education_A_Level or Equivalent	0 o 1	Indicador one-hot de nivel educativo A-Level o equivalente.
highest_education_HH_Qualification	0 o 1	Indicador one-hot de titulación HH Qualification.
highest_education_Lower Than A Level	0 o 1	Indicador one-hot de nivel educativo inferior a A Level.
highest_education_No Formal qual	0 o 1	Indicador one-hot de ausencia de calificaciones formales.
highest_education_Post Graduate Qualification	0 o 1	Indicador one-hot de estudios de posgrado.
studied_credits	Número	Créditos cursados por el estudiante (covariable numérica).

Figura 78: Descripción de variables disponible para interpretar el dataset

A.5. Análisis de covariables

El análisis de covariables permite explorar de forma inicial la relación entre distintas variables del dataset y el abandono académico. Esta sección sirve como paso previo a los modelos de supervivencia, ya que ayuda a identificar patrones generales en función de características personales o académicas del alumnado. No ofrece todavía una conclusión definitiva, pero sí una primera lectura muy útil.

A.5.1. Selección de covariables

Para utilizar esta sección, el usuario debe seleccionar el tipo de análisis de covariable que desea visualizar. La aplicación permite revisar variables como género, discapacidad, tramo de edad, nivel educativo previo y créditos estudiados. También puede mostrarse una visión general del abandono total.

La selección se realiza mediante los controles disponibles en la página. Al cambiar la opción seleccionada, la aplicación actualiza automáticamente la información mostrada para reflejar el análisis correspondiente.



Figura 79: Selección de covariables para la exploración descriptiva

A.5.2. Interpretación de tablas y gráficas

Los resultados del análisis de covariables se presentan mediante tablas, gráficos e interpretaciones textuales. Las gráficas permiten comparar de forma visual la distribución del abandono entre grupos, mientras que las tablas resumen ofrecen valores concretos que ayudan a interpretar cada variable.

La interpretación debe realizarse como una exploración descriptiva inicial. Es decir, esta sección permite observar diferencias entre grupos, pero no sustituye a los modelos de supervivencia posteriores. Para estudiar el comportamiento temporal del abandono y comparar curvas o riesgos, el usuario debe acudir a las técnicas específicas de análisis de supervivencia. La idea es mirar primero el terreno antes de entrar en el análisis más profundo.

A.6. Técnicas de análisis de supervivencia

La sección de análisis de supervivencia agrupa los métodos estadísticos disponibles en la aplicación. Para utilizar cualquiera de ellos es necesario haber cargado y preprocesado previamente el dataset. Desde esta página, el usuario puede acceder a los distintos modelos y pruebas mediante los botones o enlaces internos de navegación. Bajo el conjunto de técnicas se muestra además un botón ancho de “Comparación de técnicas”, que abre la vista comparativa general.



Figura 80: Página de acceso a las técnicas de análisis de supervivencia

A.6.1. Kaplan-Meier

El módulo Kaplan-Meier permite estimar la función de supervivencia del alumnado a lo largo del tiempo. Al acceder a esta sección se muestra una curva general de supervivencia, que representa la probabilidad estimada de permanencia en función del tiempo observado. Es una de las formas más directas de ver cómo evoluciona el abandono conforme avanza el periodo estudiado.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

El usuario puede seleccionar distintas covariables para comparar curvas entre grupos. Entre las opciones disponibles se encuentran género, discapacidad, tramo de edad, nivel educativo previo y créditos estudiados. Al elegir una covariable, la gráfica se actualiza para mostrar las curvas correspondientes a cada grupo.

La curva generada permite observar diferencias en la permanencia estimada entre grupos. Una caída más pronunciada de la curva indica una mayor concentración de eventos de abandono en menor tiempo. El usuario también puede solicitar una explicación mediante IA y exportar el análisis a PDF cuando existan resultados disponibles. De este modo, la gráfica no se queda solo como una visualización puntual, sino que puede convertirse en parte de un informe.



Figura 81: Resultado esperado al generar una curva Kaplan-Meier estratificada

A.6.2. Regresión de Cox

La regresión de Cox permite analizar el efecto de una o varias covariables sobre el riesgo de abandono. En esta pantalla, el usuario selecciona las variables que desea incluir en el modelo y ejecuta el análisis. El resultado principal se expresa mediante el Hazard Ratio, que ayuda a interpretar si una covariable se asocia con un aumento o una reducción del riesgo.

Tras ejecutar el modelo, la aplicación muestra una tabla con los coeficientes, valores de significación y medidas asociadas. Además, puede generarse una visualización tipo forest plot que facilita comparar el efecto relativo de las covariables seleccionadas.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

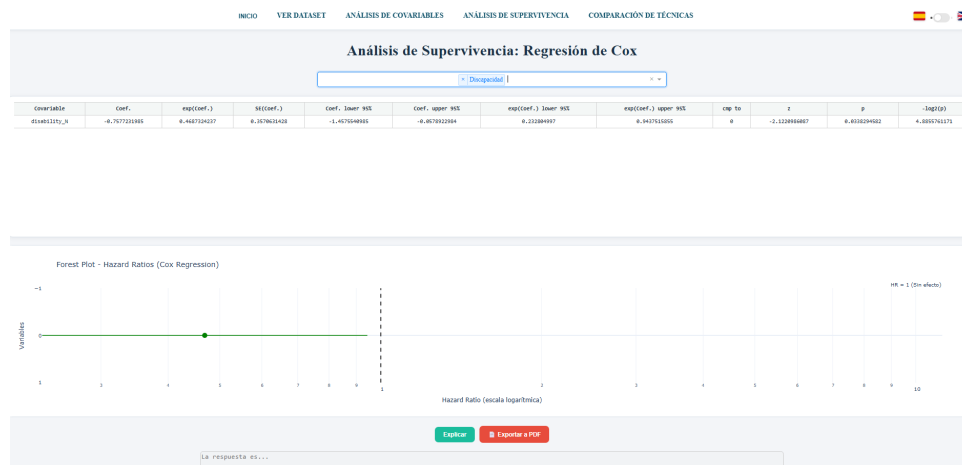


Figura 82: Visualización de los efectos obtenidos tras ejecutar la regresión de Cox

Para interpretar los resultados, se debe prestar atención al Hazard Ratio y al p-valor. Un Hazard Ratio superior a 1 indica mayor riesgo relativo de abandono, mientras que un valor inferior a 1 apunta a un posible efecto protector. El p-valor ayuda a valorar si la asociación observada es estadísticamente significativa. La lectura conjunta de ambos valores es importante, porque una cifra aislada puede resultar engañosa si no se mira dentro del contexto del modelo.

A.6.3. Test Log-Rank

El Test Log-Rank permite comparar curvas de supervivencia entre grupos y determinar si existen diferencias estadísticamente significativas entre ellas. Para utilizarlo, el usuario selecciona la variable o covariable que desea comparar y ejecuta la prueba. Resulta especialmente útil cuando una diferencia visual parece clara, pero se quiere comprobar con un contraste estadístico.

El sistema muestra los resultados de la comparación, incluyendo el estadístico de prueba y el p-valor. Si el p-valor es inferior al umbral habitual de 0.05, puede considerarse que existen diferencias estadísticamente significativas entre las curvas comparadas.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

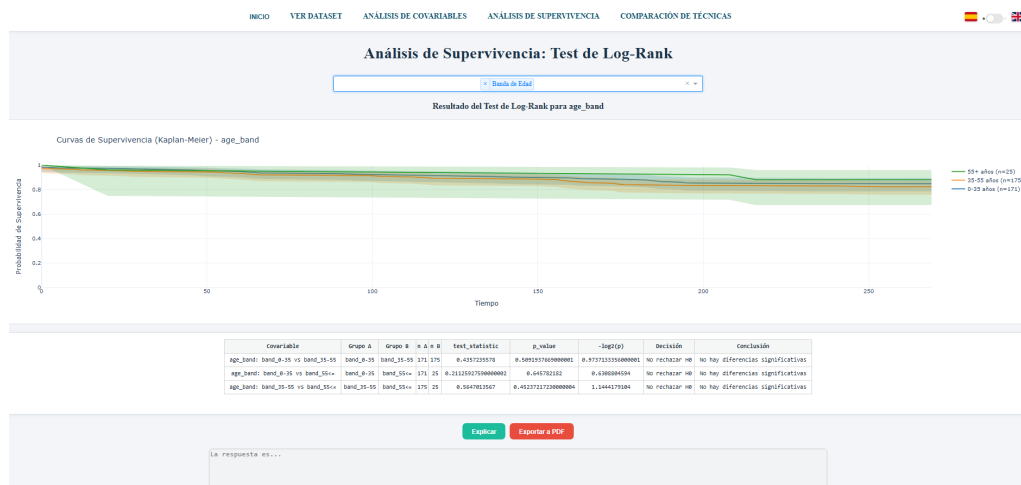


Figura 83: Resultado mostrado al comparar grupos mediante el Test Log-Rank

Este análisis funciona muy bien como complemento de Kaplan-Meier, ya que no solo permite visualizar diferencias entre curvas, sino también contrastarlas mediante una prueba estadística. Primero se observa la curva; después, se comprueba si esa diferencia tiene respaldo numérico.

A.6.4. Modelo Weibull

El modelo Weibull es una técnica paramétrica de análisis de supervivencia. A diferencia de Kaplan-Meier, que estima la curva de forma empírica, Weibull ajusta una distribución al comportamiento temporal de los datos. Esta aproximación permite interpretar parámetros del modelo y comparar su ajuste con otros métodos, algo útil cuando se busca una lectura más estructurada del fenómeno.

Al ejecutar el análisis, la aplicación muestra la curva ajustada y una tabla resumen con métricas del modelo. Entre los elementos relevantes se encuentran los parámetros de forma y escala, la mediana de supervivencia, el log-likelihood y el AIC. El parámetro de forma permite valorar si el riesgo aumenta, disminuye o se mantiene aproximadamente constante a lo largo del tiempo.

ANEXO A. ANEXO DEL MANUAL DE USUARIO



Figura 84: Resultados que debe revisar el usuario tras ejecutar el modelo Weibull

A.6.5. Modelo Exponencial

El modelo Exponencial es otro modelo paramétrico de supervivencia. Su característica principal es que asume un riesgo constante a lo largo del tiempo. Por este motivo, resulta útil como modelo sencillo de referencia frente a otros enfoques más flexibles, como Weibull o Kaplan-Meier. La verdad es que su interés está precisamente en esa sencillez: permite comprobar si una hipótesis básica encaja razonablemente con los datos.

Al ejecutar el análisis, la aplicación presenta la curva del modelo exponencial y una tabla resumen con sus métricas principales. El usuario puede comparar visualmente el ajuste exponencial con la estimación empírica de Kaplan-Meier para valorar si la hipótesis de riesgo constante resulta razonable.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

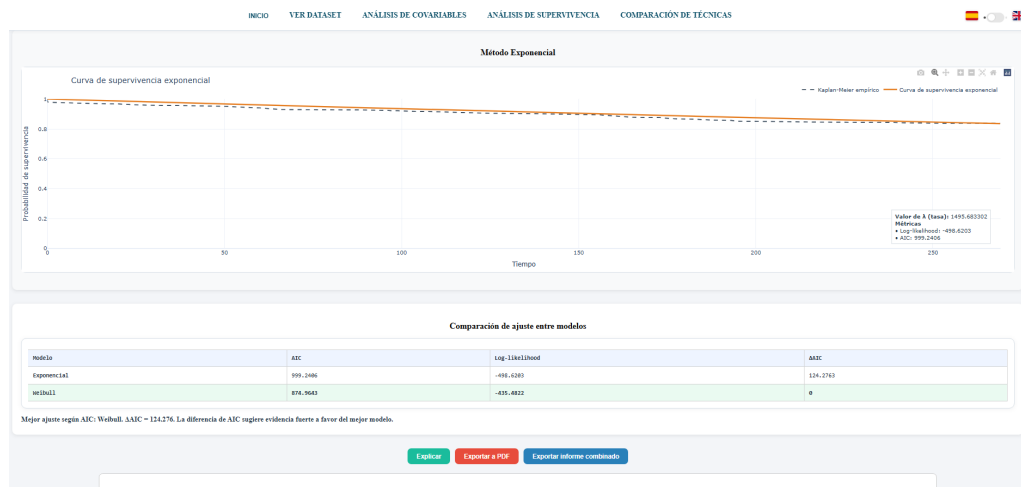


Figura 85: Resultados que debe revisar el usuario tras ejecutar el modelo Exponencial

A.6.6. Random Survival Forest

Random Survival Forest es un modelo de aprendizaje automático adaptado a datos de supervivencia. Su objetivo es estimar el riesgo y la supervivencia teniendo en cuenta múltiples variables de entrada. En la aplicación, este módulo permite entrenar el modelo, revisar métricas generales, consultar la importancia de variables y simular perfiles. Es el enfoque más flexible de los incluidos en el dashboard, aunque también exige una interpretación más cuidadosa.

Tras ejecutar el modelo, se muestran curvas de supervivencia para distintos perfiles de riesgo, junto con métricas como el índice de concordancia. Estas salidas permiten valorar la capacidad del modelo para diferenciar perfiles con mayor o menor riesgo de abandono.

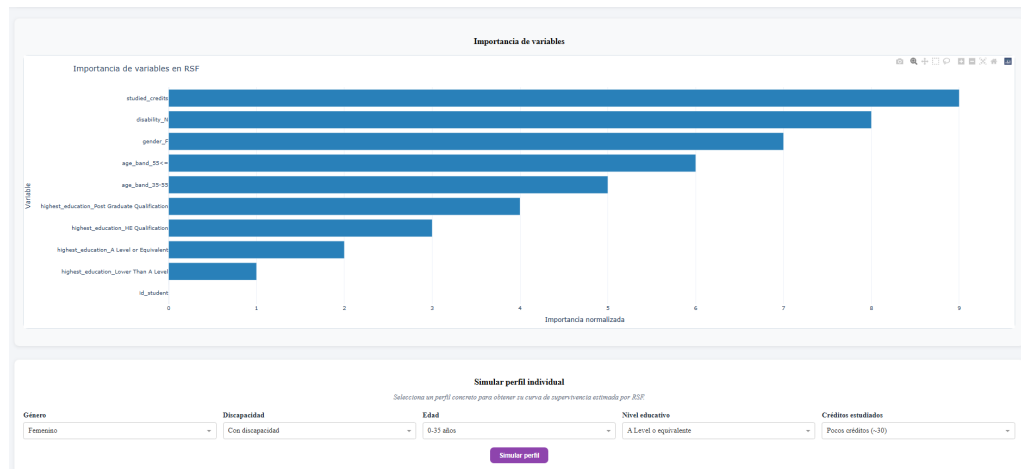


Figura 86: Curvas y métricas generadas por Random Survival Forest

El módulo también permite revisar la importancia de variables y simular un perfil específico. Esta funcionalidad ayuda a interpretar qué características tienen mayor influencia en la predicción y cómo varía la supervivencia estimada ante un caso concreto. Por ejemplo, permite observar cómo cambia la curva si se modifican determinados rasgos académicos del estudiante.

A.6.7. Comparación de técnicas

La comparación de técnicas ofrece una vista conjunta de los métodos incluidos en la aplicación. Se accede a ella desde el botón situado bajo las técnicas de la página de análisis de supervivencia, de manera que queda integrada en el mismo flujo de selección de modelos. Esta sección ayuda al usuario a distinguir el propósito de cada técnica, sus ventajas y sus limitaciones. Es especialmente útil cuando se desea decidir qué resultado utilizar como referencia principal en un análisis, porque no todos los métodos responden exactamente a la misma pregunta.

La aplicación permite revisar de forma comparativa los modelos clásicos, paramétricos y de aprendizaje automático. Esta visión conjunta facilita entender que cada método responde a una pregunta distinta: Kaplan-Meier estima curvas, Log-Rank compara grupos, Cox analiza riesgos relativos, Weibull y Exponencial ajustan modelos paramétricos, y Random Survival Forest incorpora capacidad predictiva con múltiples variables.

ANEXO A. ANEXO DEL MANUAL DE USUARIO

INICIO VER DATASET ANÁLISIS DE COVARIABLES ANÁLISIS DE SUPERVIVENCIA COMPARACIÓN DE TÉCNICAS

Esta página resume los principales métodos de supervivencia para facilitar la elección metodológica según objetivo analítico, supuestos y nivel de interpretabilidad.

Técnica	Tipo de modelo	Hipótesis	Ventajas	Limitaciones	Cuándo usarlo
Kaplan-Meier	No paramétrico	Censura no informativa; estima supervivencia empírica sin forma funcional.	Muy interpretable; útil como línea base visual.	No ajusta covariables simultáneamente.	Comparación descriptiva inicial entre grupos.
Exponencial	Paramétrico	Riesgo constante en el tiempo (hazard constante).	Simple, estable y fácil de interpretar.	Puede infraajustar si el riesgo cambia con el tiempo.	Cuando se espera comportamiento aproximadamente constante del riesgo.
Weibull	Paramétrico	Riesgo monótono (creciente o decreciente) según parámetro de forma.	Flexible y con parámetros interpretables.	Si el riesgo no es monótono puede no capturar bien el patrón.	Cuando se requiere modelo paramétrico más flexible que exponencial.
Regresión de cox	Semiparamétrico	Proporcionalidad de riesgos entre grupos/covariables.	Ajusta múltiples covariables sin asumir forma base del hazard.	Sensibilidad a violaciones de proporcionalidad.	Para cuantificar efecto de covariables (hazard ratios).
Log-rank test	Test no paramétrico	Compara igualdad global de curvas de supervivencia entre grupos.	Simple para contraste de hipótesis entre grupos.	No estima tamaño de efecto multivariable por sí solo.	Para evaluar si las curvas difieren de forma estadísticamente significativa.
Random Survival Forest	Ensamble ML	No requiere proporcionalidad ni forma paramétrica explícita.	Captura relaciones no lineales e interacciones complejas.	Menor interpretabilidad que COX.	Cuando prima capacidad predictiva y complejidad de patrones.

Figura 87: Vista utilizada para consultar la comparación entre técnicas disponibles

A.7. Interpretación con inteligencia artificial

La aplicación incorpora un módulo opcional de inteligencia artificial para generar explicaciones textuales de los resultados estadísticos. Esta funcionalidad actúa como apoyo interpretativo y no sustituye a las tablas, curvas ni métricas calculadas por los modelos. Su papel es ayudar a leer los resultados con más claridad, no decidir por el usuario.

A.7.1. Arranque del servidor local de IA

Para utilizar la interpretación automática, el servidor local de IA debe estar iniciado antes de solicitar una explicación desde el dashboard. Para ello, se abre una terminal en la carpeta raíz del proyecto y se ejecuta:

```
.\START_LLAMA_SERVER.bat
```

Este servidor se ejecuta de forma independiente a la aplicación Dash. Por tanto, si se desea usar la IA, deben mantenerse abiertas dos terminales: una con `llama-server` y otra con la aplicación Dash iniciada mediante `python cargaDataset.py` o mediante `START_DASH_APP.bat`. Puede parecer un detalle menor, pero si una de las dos se cierra, la parte de IA dejará de responder.

Si el servidor no está activo, la aplicación sigue funcionando. En ese caso, las técnicas estadísticas y las gráficas continúan disponibles, pero no se generarán respuestas automáticas de IA.

A.7.2. Generación de explicaciones automáticas

Las explicaciones automáticas pueden solicitarse desde los módulos de análisis mediante los botones de interpretación disponibles en la interfaz. Al pulsar el botón correspondiente, la aplicación recopila los resultados calculados y los envía al modelo local para obtener una explicación redactada en lenguaje natural. Así, el usuario no parte solo de una tabla o una curva, sino también de un texto que ayuda a ordenar la lectura.

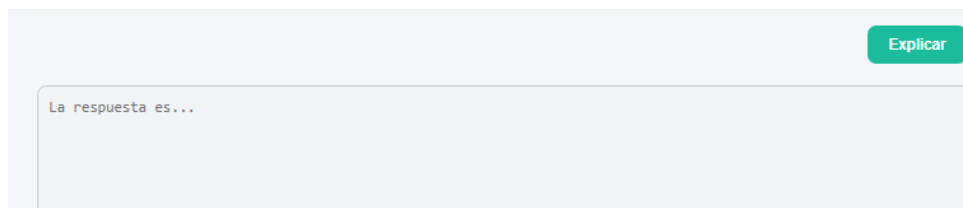


Figura 88: Botón de interpretación mediante inteligencia artificial

La respuesta generada se muestra en la propia página del análisis. Además, si el usuario selecciona la opción correspondiente al exportar un informe PDF, la interpretación puede incluirse dentro del documento generado.

```
def _academic_system_prompt(Language='es'):  
    if Language == 'en':  
        return (  
            "You are a survival-analysis assistant for an academic thesis. "  
            "Write in formal, clear, evidence-based prose using only provided results. "  
            "Return 2-3 short paragraphs with no bullet points and no numbered lists."  
        )  
    return (  
        "Eres un asistente de análisis de supervivencia para un TFG. "  
        "Redacta en prosa académica clara y basada en evidencias, usando solo resultados proporcionados. "  
        "Devuelve 2-3 párrafos breves, sin viñetas ni listas numeradas."  
    )
```

Figura 89: Ejemplo de explicación generada para apoyar la interpretación de resultados

A.7.3. Limitaciones de las interpretaciones generadas

Las respuestas generadas por inteligencia artificial deben entenderse como una ayuda para redactar e interpretar los resultados, no como una conclusión automática definitiva. El usuario debe revisar siempre que el texto sea coherente con las tablas, curvas y métricas mostradas en pantalla. Esta revisión es importante, porque la interpretación final sigue dependiendo del análisis estadístico y del criterio de quien utiliza la herramienta.

El modelo trabaja con la información que recibe desde la aplicación. Si los resultados son incompletos, si no se ha ejecutado correctamente el análisis o si el servidor local no está disponible, la explicación puede no generarse. Además, aunque el sistema intenta restringir la respuesta a los datos proporcionados, es recomendable contrastar cualquier interpretación con los valores estadísticos originales.

A.8. Exportación de informes PDF

La aplicación permite exportar informes PDF desde los distintos módulos de análisis. Esta funcionalidad facilita conservar los resultados principales de cada técnica, incluyendo tablas, gráficas, resúmenes e interpretaciones generadas mediante IA cuando el usuario así lo selecciona. Es especialmente útil cuando se quiere guardar una evidencia del análisis o compartir los resultados sin depender de la interfaz.

A.8.1. Selección de contenido del informe

Para generar un informe, el usuario debe pulsar el botón de exportación disponible en el módulo correspondiente. A continuación se abre una ventana modal donde se puede indicar el nombre del informe y seleccionar qué elementos se desean incluir.

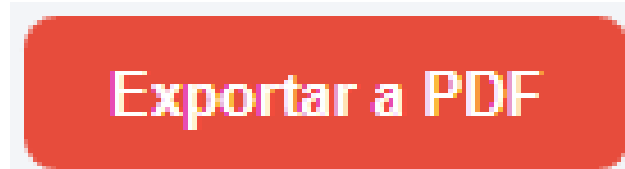


Figura 90: Botón que inicia la exportación desde un módulo de análisis

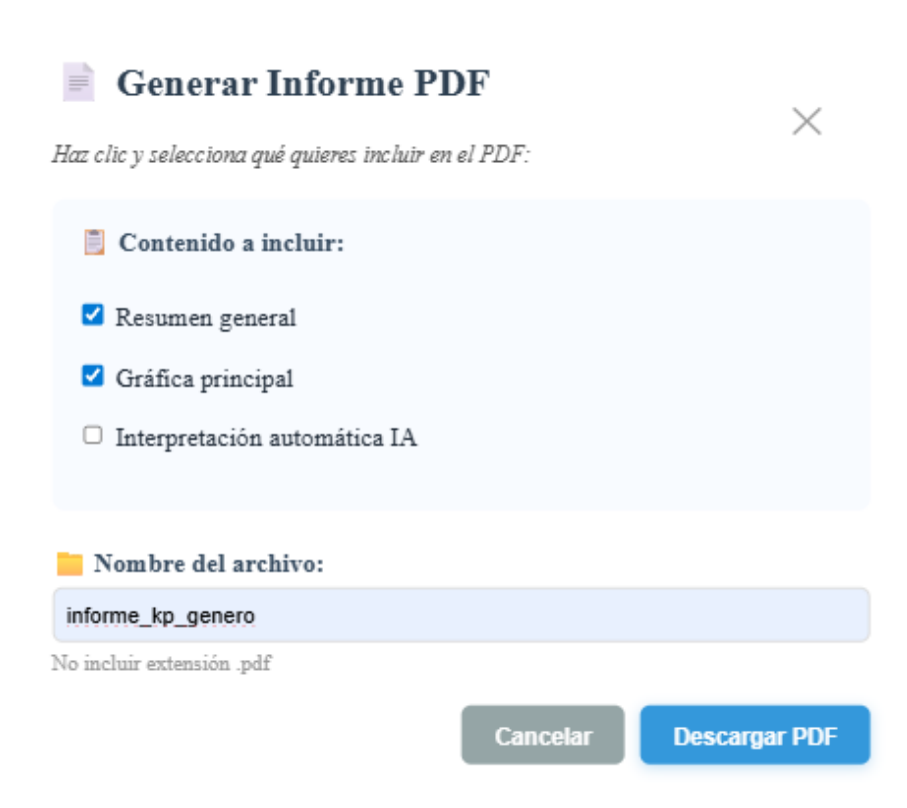


Figura 91: Opciones de contenido para el informe PDF

Las opciones disponibles pueden variar según el módulo, pero normalmente permiten incluir resumen, tabla de resultados, gráfica e interpretación automática. En algunos casos, como los modelos Weibull y Exponencial, también puede generarse un informe combinado.

A.8.2. Generación y descarga del PDF

Una vez seleccionadas las opciones deseadas, el usuario debe confirmar la exportación. El sistema valida que existan resultados suficientes y genera el informe PDF. Si no se han calculado resultados o si falta algún elemento necesario, la aplicación muestra un mensaje de aviso. De esta forma se evita descargar documentos vacíos o poco útiles.

El documento generado incluye los elementos seleccionados y mantiene una estructura orientada a la lectura del análisis: título del informe, información general, resultados, gráficas e interpretación cuando procede.



EXPONENTIAL METHOD REPORT: GENERAL

1. RESUMEN GENERAL

Métrica	Valor
Número de pacientes	371
Número de eventos	60
Tasa de eventos (%)	16.2%
Follow-up medio (meses)	241.89
Follow-up mediano (meses)	269.00

Figura 92: Ejemplo de informe PDF generado a partir de los resultados seleccionados

A.8.3. Ubicación de los informes generados

Los informes generados se almacenan en la carpeta **downloads/** del proyecto. Desde esa ubicación pueden abrirse, copiarse o adjuntarse como cualquier otro documento PDF.

Se recomienda asignar nombres descriptivos a los informes para identificar fácilmente la técnica utilizada y el contenido exportado. Por ejemplo, pueden emplearse nombres como `informe.kaplan.genero`, `cox_covariables` o `comparacion.weibull.exponencial`. Este pequeño hábito ahorra tiempo cuando se generan varios PDFs durante una misma sesión de análisis.

A.9. Errores frecuentes

Durante el uso de la aplicación pueden aparecer errores relacionados con la carga del dataset, la instalación de dependencias, el arranque del servidor o la

ANEXO A. ANEXO DEL MANUAL DE USUARIO

generación de informes. Esta sección resume las incidencias más habituales y la forma recomendada de actuar. No todos los avisos indican un fallo grave; muchas veces solo señalan que falta completar un paso previo.

A.9.1. Problemas al cargar el dataset

Si el sistema rechaza el archivo cargado, se debe comprobar que se trata de un archivo CSV válido y que contiene las columnas obligatorias. También conviene revisar el separador utilizado, la codificación del archivo y que las columnas críticas no estén vacías. A veces el problema no está en los datos en sí, sino en pequeños detalles de formato que impiden que la aplicación los lea correctamente.



Figura 93: Mensaje mostrado cuando el archivo cargado no corresponde al dataset esperado

Si el archivo se ha modificado manualmente, es recomendable compararlo con el formato esperado por la aplicación antes de volver a cargarlo. Un cambio mínimo en el nombre de una columna puede ser suficiente para que el sistema no la reconozca.

A.9.2. Problemas al acceder a módulos sin preprocesar

Algunas secciones del dashboard necesitan que el dataset haya sido cargado y preprocesado previamente. Si el usuario intenta acceder a un módulo de análisis sin haber completado ese paso, la aplicación muestra un aviso indicando que primero debe cargarse el dataset académico y pulsarse la opción de preprocesamiento.

ANEXO A. ANEXO DEL MANUAL DE USUARIO



Figura 94: Aviso mostrado al acceder a una sección que requiere el dataset preprocesado

Para resolver este caso, el usuario debe volver a la pantalla de inicio, cargar el archivo CSV correcto y ejecutar el preprocesamiento. Una vez finalizado correctamente, las secciones de análisis quedarán habilitadas para trabajar con el dataset limpio. Es el mismo flujo inicial, simplemente repetido con los datos preparados de forma adecuada.

A.9.3. Problemas durante la instalación de dependencias

Si aparece un error durante la instalación de librerías, el primer paso es comprobar la versión de Python utilizada. Para este proyecto se ha empleado Python 3.13.2 junto con `scikit-survival==0.27.0`. La dependencia que puede requerir más atención es precisamente `scikit-survival`, ya que trabaja con componentes científicos especializados y puede ser más sensible al entorno.

También se recomienda actualizar `pip` antes de instalar las dependencias y utilizar un entorno virtual limpio. Muchas instalaciones fallidas se solucionan simplemente empezando desde un entorno sin librerías mezcladas.

Si el entorno virtual se ha creado sobre una instalación previa o se han actualizado paquetes manualmente, pueden aparecer conflictos entre dependencias. En ese caso, una solución bastante fiable consiste en recrear el entorno desde cero y volver a instalar el archivo `requirements.txt`:

```
Remove-Item -Recurse -Force .venv
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
```

```
python -m pip install -r requirements.txt
```

A.9.4. Problemas al arrancar la aplicación

Si la aplicación no arranca, se debe comprobar que la terminal está situada en la carpeta raíz del proyecto y que las dependencias se han instalado correctamente. También debe verificarse que el archivo `cargaDataset.py` existe en esa carpeta. Es un error frecuente abrir la terminal en una ruta distinta y ejecutar el comando desde ahí.

Un error que puede aparecer en algunos equipos está relacionado con la importación de `get_current_traceback` desde `werkzeug.debug.tbtools`. Este problema no suele venir del código de la aplicación, sino de una instalación antigua de **Dash** combinada con versiones más recientes de **Flask** o **Werkzeug**. Es una de esas incompatibilidades algo frustrantes: el proyecto está bien, pero el entorno mezcla piezas que ya no encajan entre sí. El mensaje suele tener una forma parecida a:

```
ImportError: cannot import name 'get_current_traceback'  
from 'werkzeug.debug.tbtools'
```

La solución recomendada es instalar la versión de **Dash** indicada para este proyecto y volver a ejecutar el programa:

```
python -m pip install "dash==3.1.0"  
python .\cargaDataset.py
```

El código de arranque contempla tanto instalaciones actuales de **Dash** como entornos antiguos donde todavía exista **Dash 2**. En concreto, si la versión instalada dispone de `app.run`, se utiliza ese método; si no, se recurre a `app.run_server`. La verdad es que esto ayuda bastante cuando el proyecto se prueba en varios equipos, porque evita que una diferencia menor de versión impida abrir el dashboard.

Si el problema persiste porque el entorno virtual arrastra librerías de una instalación anterior, lo más limpio es reinstalar las dependencias desde el archivo `requirements.txt`. Así se evita ir corrigiendo paquetes uno a uno:

```
python -m pip install --upgrade pip  
python -m pip install -r requirements.txt
```

```
python .\cargaDataset.py
```

Si el puerto 8050 estuviera ocupado por otro proceso, puede ser necesario cerrar la ejecución anterior de la aplicación o reiniciar la terminal.

A.9.5. Problemas con el módulo de inteligencia artificial

Si no se genera una interpretación automática, lo más habitual es que el servidor local de IA no esté iniciado. En ese caso, debe ejecutarse `START_LLAMA_SERVER.bat` y comprobar que `llama-server` queda escuchando en `http://127.0.0.1:8000`. Mientras ese servidor no esté activo, el dashboard seguirá funcionando, pero no tendrá a quién pedirle la explicación.

También debe comprobarse que existen el modelo GGUF y el ejecutable `llama-server.exe` en las rutas configuradas. Si el usuario utiliza rutas propias, debe ajustar las variables `MODEL_PATH` y `LLAMA_SERVER`.

Si el ejecutable `llama-server.exe` existe pero no llega a arrancar, conviene revisar algunos factores del equipo. En ciertos casos, Windows o el antivirus pueden bloquear la ejecución de archivos descargados desde Internet; también puede ocurrir que el puerto 8000 ya esté ocupado por otro proceso, que el proyecto se encuentre en una carpeta sincronizada como OneDrive o que el ordenador no disponga de recursos suficientes para cargar el modelo local. En estas situaciones, lo más recomendable es cerrar procesos anteriores, mover el proyecto a una ruta sencilla como `C:\TFG`, permitir la ejecución del archivo si Windows lo bloquea y volver a iniciar `START_LLAMA_SERVER.bat`.

A.9.6. Problemas al generar informes PDF

Si el PDF no se genera, se debe verificar que el análisis correspondiente se ha ejecutado previamente y que existen resultados válidos para exportar. Algunas opciones del informe, como incluir gráficas o interpretación IA, requieren que esos elementos estén disponibles antes de confirmar la descarga. En resumen, el informe solo puede guardar aquello que ya se ha calculado.

Si el problema está relacionado con las gráficas, puede estar asociado a la conversión de figuras mediante `kaleido`. En ese caso, conviene revisar que las dependencias se hayan instalado correctamente desde `requirements.txt`.

A.10. Flujo completo recomendado

El flujo de uso recomendado para trabajar con la aplicación es el siguiente. Seguir este orden ayuda a avanzar sin saltarse pasos importantes:

1. Iniciar la aplicación con `python cargaDataset.py` o con `START_DASH_APP.bat`.
2. Abrir en el navegador la dirección `http://127.0.0.1:8050`.
3. Cargar el archivo CSV desde la pantalla principal.
4. Preprocesar el dataset para generar la versión limpia.
5. Revisar el dataset en la sección de visualización.
6. Explorar las covariables para obtener una primera lectura descriptiva.
7. Ejecutar las técnicas de análisis de supervivencia necesarias.
8. Solicitar interpretaciones mediante IA si el servidor local está activo.
9. Exportar los resultados relevantes a PDF.

Este orden permite avanzar desde la carga de datos hasta la obtención de resultados exportables de forma controlada. Aunque el usuario puede navegar libremente por la interfaz, la mayoría de módulos requieren que el dataset haya sido cargado y preprocesado previamente. La verdad es que, si se respeta este flujo, la aplicación resulta bastante más cómoda de usar.

ANEXO B

MANUAL DE CÓDIGO FUENTE

B Anexo del Manual de código fuente

B.1. Finalidad del manual de código fuente

Este anexo tiene como finalidad facilitar la lectura, comprensión y mantenimiento del código fuente de la aplicación. A diferencia del manual de usuario, no explica cómo utilizar el dashboard desde la interfaz, sino cómo está organizado internamente el proyecto y qué papel cumplen sus principales variables, funciones y módulos. La idea es sencilla: que una persona distinta del autor pueda abrir el repositorio, orientarse sin demasiada fricción y saber dónde mirar antes de modificar cualquier parte del sistema.

El manual está pensado como una guía de apoyo para desarrolladores o futuros estudiantes que necesiten revisar, corregir o ampliar la aplicación. Por ello, no se documenta cada línea de código de forma exhaustiva. Eso haría el documento demasiado pesado y, además, poco práctico. En su lugar, se recogen las variables más importantes, las responsabilidades generales de los archivos principales y algunos fragmentos representativos de las funciones más delicadas.

Las capturas incluidas no pretenden mostrar todo el código fuente. Se han seleccionado únicamente aquellas partes que ayudan a entender mejor el funcionamiento interno del sistema: el preprocesamiento del dataset, el ajuste de un modelo más complejo como Random Survival Forest y la comunicación con el servidor local de inteligencia artificial. La verdad es que, en un proyecto con varios módulos de análisis, callbacks, exportación PDF e IA, esta selección resulta más útil que llenar el anexo con capturas de funciones repetitivas o demasiado sencillas.

B.2. Criterios de organización del código

El código se ha organizado intentando mantener una separación clara entre responsabilidades. La aplicación combina interfaz web, procesamiento de datos, modelos estadísticos, generación de gráficas, exportación de informes, traducciones e interpretación mediante inteligencia artificial. Si toda esa lógica estuviera mezclada

ANEXO B. ANEXO DEL MANUAL DE CÓDIGO FUENTE

en un único archivo, cualquier cambio pequeño sería mucho más difícil de controlar. Por eso, cada bloque funcional se sitúa en archivos específicos.

Como criterio general, los archivos tienen nombres descriptivos y agrupan una responsabilidad principal. Por ejemplo, los modelos estadísticos se separan en módulos propios, los callbacks se concentran en archivos específicos y los textos traducibles se mantienen en `translations.py`. Esta división ayuda a localizar mejor cada parte del sistema y evita que la interfaz, el análisis y la exportación queden mezclados sin control.

Los comentarios del código se utilizan especialmente en bloques donde la lectura puede resultar menos evidente: carga diferida de datasets, tratamiento de variables no binarias, llamadas al servidor local de inteligencia artificial, generación de informes PDF o recuperación ante errores de convergencia. No se comentan operaciones triviales, porque eso puede generar ruido. Se priorizan comentarios que expliquen decisiones, no instrucciones obvias.

Cualquier ampliación futura debería respetar esta estructura. Si se añade una nueva técnica de análisis, lo más adecuado no sería insertar toda la lógica dentro de un callback, sino crear un módulo propio, devolver resultados en un formato reutilizable y conectar después esa salida con la interfaz, la exportación y las traducciones necesarias. Es una forma de trabajar más ordenada y, sobre todo, más segura.

B.3. Diccionario de variables principales

En esta sección se recogen las variables más relevantes para entender el funcionamiento del sistema. Algunas proceden directamente del dataset académico y otras son variables internas utilizadas por la aplicación. No se incluyen todas las variables temporales o auxiliares del código, sino aquellas que aparecen de forma recurrente y que ayudan a comprender el flujo general de datos.

B.3.1. Variables principales del dataset

Variable	Significado	Uso dentro de la aplicación
<code>id_student</code>	Identificador único del estudiante.	Permite agrupar registros y seleccionar una fila representativa por estudiante durante el preprocesamiento.
<code>date</code>	Tiempo o momento observado dentro del seguimiento académico.	Se utiliza como variable temporal en los modelos de supervivencia.
<code>final_result</code>	Resultado final asociado al estudiante.	Se transforma en indicador del evento de abandono o permanencia según el criterio definido en el preprocesamiento.
<code>gender_F</code>	Variable binaria asociada al género.	Se usa como covariable personal en análisis comparativos y modelos estadísticos.
<code>disability_N</code>	Variable binaria asociada a la existencia o no de discapacidad registrada.	Permite estudiar diferencias de supervivencia o riesgo según esta característica.
<code>age_band</code>	Conjunto de columnas que representan tramos de edad.	Permite incorporar la edad como variable categórica codificada en varias columnas.
<code>highest_education</code>	Conjunto de columnas asociadas al nivel educativo previo.	Se utiliza para estudiar si la formación previa está relacionada con la permanencia o el abandono.
<code>studied_credits</code>	Número de créditos estudiados por el estudiante.	Es una variable numérica no binaria; puede requerir agrupación por intervalos o cuantiles según la técnica aplicada.

Tabla 37: Variables principales procedentes del dataset académico

Estas variables son especialmente importantes porque alimentan los modelos de supervivencia. Si alguna de ellas falta, contiene valores inesperados o se transforma

ANEXO B. ANEXO DEL MANUAL DE CÓDIGO FUENTE

de manera incorrecta, los resultados pueden dejar de ser fiables. Por ese motivo, el preprocesamiento valida la existencia de columnas críticas y controla los casos en los que los datos no son suficientes para continuar.

ANEXO B. ANEXO DEL MANUAL DE CÓDIGO FUENTE

B.3.2. Variables internas de la aplicación

Variable	Significado	Uso dentro del código
df	DataFrame con datos cargados o datos base.	Se utiliza como estructura principal para almacenar el dataset antes o durante distintos procesos de análisis.
df_limpio	DataFrame tras aplicar el preprocesamiento.	Es la versión preparada para ejecutar los modelos estadísticos y mostrar datos ya validados.
df_json	Representación serializada del dataset.	Permite guardar y recuperar el dataset en componentes <code>dcc.Store</code> de Dash entre callbacks.
language	Idioma activo de la interfaz.	Controla si los textos, mensajes, etiquetas e informes se generan en castellano o inglés.
contents	Contenido del archivo cargado desde la interfaz.	Se utiliza durante la carga del CSV para leer el archivo subido por el usuario.
summary_df	Tabla resumen generada por un análisis.	Recoge métricas o resultados principales para mostrarlos en pantalla o exportarlos a PDF.
figure	Figura generada por Plotly.	Representa gráficas de supervivencia, importancia de variables, comparaciones o curvas de modelos.
analysis	Estructura con resultados de un análisis.	Suele agrupar figura, tabla, métricas e interpretación para reutilizar la salida en distintos puntos del sistema.
store_data	Datos almacenados temporalmente en componentes de Dash.	Permite conservar resultados entre interacciones sin recalcular todo desde cero.
LLAMA_SERVER_URL	Dirección del servidor local de inteligencia artificial.	Indica el endpoint al que se envían las peticiones para generar interpretaciones automáticas.
MODEL_NAME Trabajo Fin de Grado	Nombre del modelo de lenguaje configurado.	Identifica el modelo Qwen utilizado por el servidor local para generar explicaciones.

Tabla 38: Variables internas más relevantes para entender el flujo de la aplicación

La distinción entre variables del dataset y variables internas es importante. Las primeras representan información académica sobre los estudiantes. Las segundas permiten que la aplicación mueva esa información entre carga, preprocesamiento, análisis, visualización, IA y exportación. Dicho de forma sencilla: unas describen los datos; las otras ayudan a que el sistema funcione.

B.4. Funciones principales por archivo

En este apartado se muestran algunas funciones representativas del código fuente. No se incluyen capturas de todos los archivos, ya que el objetivo del manual no es reproducir el código completo, sino señalar los puntos más importantes para comprender el funcionamiento interno de la aplicación. Las imágenes seleccionadas corresponden a fases clave: preparación del dataset, ajuste del modelo Random Survival Forest y comunicación con el módulo local de inteligencia artificial.

B.4.1. Preprocesamiento del dataset

La función de preprocesamiento se encuentra en `preprocesamiento.py`. Su responsabilidad principal es preparar el dataset antes de ejecutar los modelos. Para ello, valida que existan las columnas necesarias, transforma el resultado final en una variable interpretable por los modelos de supervivencia, ordena los registros por estudiante y fecha, y genera una versión limpia de los datos. Esta función es especialmente importante porque todos los análisis posteriores dependen de que la información llegue en un formato coherente.

Además, esta función actúa como una primera barrera de control. Si el dataset está vacío, si faltan columnas críticas o si los registros no permiten construir una observación válida por estudiante, el sistema evita continuar como si todo estuviera correcto. Dicho de forma sencilla: antes de calcular, el código intenta asegurarse de que los datos tienen sentido.

ANEXO B. ANEXO DEL MANUAL DE CÓDIGO FUENTE

```

# Función para preprocesar el dataset
def preprocess_data(df_tmp):
    """
    Preprocesa el dataset eliminando duplicados, conservando columnas esenciales,
    y validando la integridad de los datos.
    """
    # Validación 1: Datos vacíos
    if df_tmp is None or len(df_tmp) == 0:
        raise ValueError("ERROR: El dataset está vacío. No hay filas para procesar.")

    # Validación 2: Columnas críticas
    columnas_criticas = ['id_student', 'date', 'final_result']
    columnas_faltantes = [col for col in columnas_criticas if col not in df_tmp.columns]
    if columnas_faltantes:
        raise ValueError("ERROR: Faltan columnas críticas: {}".format(columnas_faltantes))

    print("[OK] Validaciones iniciales pasadas.")
    print(" - Filas del archivo: {}".format(len(df_tmp)))
    print(" - Columnas encontradas: {}".format(len(df_tmp.columns)))

    df_tmp.columns = df_tmp.columns.str.strip()

    # Convertir la columna 'final_result' en la columna binaria: 1 = aprobado, 0 = no aprobado
    try:
        df_tmp['final_result'] = df_tmp['final_result'].astype(str).str.strip().str.lower()
        df_tmp['final_result'] = df_tmp['final_result'].apply(lambda x: 1 if x == 'aprobado' else 0)
    except Exception as e:
        raise ValueError("ERROR: No se pudo procesar la columna 'final_result'. {}".format(e))

    # Ordenar por 'id_student' y 'date' en orden descendente (de más reciente a más antiguo)
    df_tmp = df_tmp.sort_values(by=['id_student', 'date'], ascending=[True, False])

    # Eliminar la columna 'Unnamed: 0' si no es necesaria
    if 'Unnamed: 0' in df_tmp.columns:
        df_tmp = df_tmp.drop(columns=['Unnamed: 0'])

    # Obtener columnas de actividad antes de definir la función
    # IMPORTANTE: Filtrar solo las columnas que existen en el dataframe
    todas_columnas_actividad = df_tmp.columns
    columnas_actividad_validas = [col for col in todas_columnas_actividad if col in df_tmp.columns]

    # Función para seleccionar la fila adecuada por estudiante
    def seleccionar_fila_grupo():
        if grupo['final_result'].iloc[0] == 0:
            fila_200 = grupo['date'].iloc[0]
            if not fila_200.empty:
                return fila_200.iloc[0]
            else:
                return grupo.iloc[0] # Si no hay fecha 200, se toma la más reciente
        else:
            # Si hay columnas de actividad, buscar fila con actividad
            if len(columnas_actividad_validas) > 0:
                grupo_con_actividad = grupo[columnas_actividad_validas]
                if not grupo_con_actividad.empty:
                    return grupo_con_actividad.iloc[0]
            # Si no tiene actividad o no hay columnas de actividad, tomar fila con fecha
            fila_0 = grupo['date'].iloc[0]
            if not fila_0.empty:
                return fila_0.iloc[0]
            else:
                return grupo.iloc[0]

    # Filas para agrupar por estudiante
    df_final = df_tmp.groupby('id_student', group_keys=False).apply(seleccionar_fila)

    # Convertir columnas a int (excepto las de actividad que pueden ser float)
    # Nota: 'date' con 'dt' es necesaria para permitir valores NaN
    for col in df_final.columns:
        if col not in columnas_actividad_validas:
            try:
                # Intentar convertir a int64 (número entero)
                df_final[col] = pd.to_numeric(df_final[col], errors='coerce').astype('int64')
            except Exception as e:
                print("Advertencia: No se pudo convertir '{col}' a int: {}".format(e))
                # Se deja la columna como está si hay problemas

    # Columnas a conservar: id_student, date, final_result, atributos binarios y nuevas atributos no-binarios
    columnas_a_conservar = [
        'id_student', 'date', 'final_result',
        'gender', 'disability',
        'age_band_0-20', 'age_band_20-30', 'age_band_30-40', 'age_band_40-50',
        'highest_education_A_level_or_equivalent', 'highest_education_Higher_education',
        'highest_education_lower_than_A_level', 'highest_education_No_formal_qualification',
        'highest_education_Post_graduate_qualification',
        'studied_credits'
    ]

    # VALIDACIÓN 3: Verificar que existen todas las columnas esperadas
    columnas_faltantes = [col for col in columnas_a_conservar if col not in df_final.columns]
    if columnas_faltantes:
        print("ADVERTENCIA: Faltan columnas esperadas: {}".format(columnas_faltantes))
        print(" - Se conservan solo las columnas disponibles")
        columnas_a_conservar = [col for col in columnas_a_conservar if col in df_final.columns]

    # Verificar que existen datos de supervivencia
    columnas_a_eliminar = [col for col in df_final.columns if col not in columnas_a_conservar]

    # Las columnas
    df_final = df_final.drop(columns=columnas_a_eliminar)

    # # # ERROR #7: Validación de valores NaN en columnas críticas
    critical_cols = ['id_student', 'date', 'final_result']
    nan_in_critical = []
    for col in critical_cols:
        if col in df_final.columns:
            nan_count = df_final[col].isna().sum()
            if nan_count > 0:
                nan_in_critical.append((col, nan_count))

    if nan_in_critical:
        print("ADVERTENCIA: Valores NaN en columnas críticas:")
        for col, count in nan_in_critical:
            print(" - {col}: (count) valores NaN")
        # Filtrar filas con NaN en columnas críticas
        df_final = df_final.dropna(subset=critical_cols)
        print(" - Filas con NaN removidas. Filas restantes: {}".format(df_final.shape[0]))

    # [OK] ERROR #7: Reporte de NaN en todas las columnas
    nan_summary = df_final.isna().sum()
    if nan_summary.any():
        print("ADVERTENCIA: Valores NaN por columna:")
        for col, count in nan_summary.items():
            if count > 0:
                print(" - {col}: (count) valores NaN")

    # [OK] Mostrar resumen de la limpieza realizada
    print("[OK] Preprocesamiento completado.")
    print(" - Filas procesadas: {}".format(df_final.shape[0]))
    print(" - Columnas conservadas: {}".format(df_final.shape[1]))
    print(" - Columnas eliminadas: {}".format(len(columnas_a_eliminar)))
    print(" - Distribución final_result: {}".format(df_final['final_result'].value_counts().to_dict()))
    print(" - Columnas de highest_education presentes: {}".format([col for col in df_final.columns if 'highest_education' in col]))
    print(" - Todas las columnas conservadas: {}".format(df_final.columns))

```

Figura 95: Fragmentos de la función de preprocesamiento del dataset

B.4.2. Ajuste de Random Survival Forest

El ajuste de Random Survival Forest se implementa en `rsf.py`. Esta parte del código construye el objeto de supervivencia, entrena el modelo y obtiene métricas asociadas a su rendimiento. Es una de las funciones más relevantes del proyecto porque introduce una técnica basada en aprendizaje automático y requiere preparar los datos de una forma más específica que los modelos paramétricos.

A diferencia de Weibull o Exponencial, Random Survival Forest necesita una matriz de características y una estructura especial para representar el tiempo y el evento. Por eso, el código no solo llama al modelo, sino que prepara las entradas, ajusta el bosque, calcula puntuaciones de riesgo y conserva métricas como el índice de concordancia o la puntuación *out-of-bag* cuando está disponible.

```
def _fit_rsf_model(df: pd.DataFrame):  
    feature_df, durations, events = _build_feature_matrix(df)  
    if feature_df.empty or durations.nunique() < 2:  
        return None  
  
    if len(feature_df) < 5 or feature_df.shape[1] < 1:  
        return None  
  
    y = Surv.from_arrays(event=events.astype(bool).to_numpy(), time=durations.to_numpy(dtype=float))  
  
    rsf = RandomSurvivalForest(  
        n_estimators=200,  
        min_samples_split=10,  
        min_samples_leaf=5,  
        max_features="sqrt",  
        n_jobs=-1,  
        random_state=42,  
        oob_score=True,  
    )  
    rsf.fit(feature_df, y)  
  
    risk_scores = rsf.predict(feature_df)  
    train_c_index = concordance_index_censored(events.astype(bool).to_numpy(), durations.to_numpy(dtype=float),  
    oob_score = getattr(rsf, "oob_score_", None)  
  
    return {  
        "model": rsf,  
        "feature_df": feature_df,  
        "durations": durations,  
        "events": events,  
        "risk_scores": risk_scores,  
        "train_c_index": float(train_c_index),  
        "oob_score": None if oob_score is None else float(oob_score),  
    }
```

Figura 96: Fragmento de código encargado del ajuste de Random Survival Forest

B.4.3. Llamada al servidor local de inteligencia artificial

La comunicación con el modelo de inteligencia artificial se gestiona en `ollama.AI.py`. La función mostrada prepara la petición HTTP, envía el prompt al servidor local y procesa la respuesta recibida. Este bloque es importante porque conecta la aplicación Dash con `llama-server`, por lo que también debe controlar posibles errores de conexión o falta de disponibilidad del servidor.

La función recibe el texto que debe interpretar el modelo, construye el cuerpo de la petición con el nombre del modelo, los mensajes y los parámetros de generación, y después espera la respuesta del servidor local. Si la comunicación falla, la aplicación debe responder de forma controlada, ya que la IA es una ayuda interpretativa, pero no debe bloquear el resto del dashboard.

```
def _call_llm(prompt, max_tokens, temperature=DEFAULT_TEMPERATURE, timeout=REQUEST_TIMEOUT_S, language='es'):
    """Invoca llama-server con parámetros homogéneos y salida académica consistente."""
    payload = {
        "model": MODEL_NAME,
        "messages": [
            {"role": "system", "content": _academic_system_prompt(language)},
            {"role": "user", "content": prompt}
        ],
        "temperature": temperature,
        "top_p": DEFAULT_TOP_P,
        "max_tokens": max_tokens,
        "stream": False,
    }
    response = requests.post(LLAMA_SERVER_URL, json=payload, timeout=timeout)
    response.raise_for_status()
    result = response.json()
    content = result['choices'][0]['message']['content'].strip()
    return _rewrite_to_prose_if_needed(content, max_tokens=max_tokens, language=language, timeout=timeout)
```

Figura 97: Fragmento de código utilizado para llamar al servidor local de inteligencia artificial

B.5. Resumen de archivos principales

Los archivos principales del proyecto ya se han mencionado anteriormente en la memoria. En concreto, aparecen en el capítulo de **Diseño del sistema**, dentro de la descripción de la arquitectura y de la vista de clases y componentes, y también en el capítulo de **Implementación**, especialmente en los apartados dedicados a la estructura del código fuente, la integración con callbacks, los modelos, la inteligencia artificial y la exportación de informes. Por tanto, este apartado no pretende repetir toda esa explicación, sino reunir los archivos en una vista rápida orientada al mantenimiento del código.

A continuación se resume la responsabilidad de los archivos más importantes del proyecto. Esta tabla no sustituye a la lectura del código, pero ayuda a localizar rápidamente dónde se encuentra cada parte de la aplicación. Es especialmente útil cuando se quiere modificar una funcionalidad concreta sin revisar todo el repositorio desde cero.

ANEXO B. ANEXO DEL MANUAL DE CÓDIGO FUENTE

Archivo	Responsabilidad	Funciones o elementos relevantes
<code>cargaDataset.py</code>	Punto de entrada de la aplicación Dash.	Inicialización de la app, navegación, componentes <code>dcc.Store</code> y arranque del servidor local.
<code>layout.py</code>	Definición visual de las páginas.	Construcción de pantallas, botones, modales, secciones de análisis y componentes de interfaz.
<code>analysis_callbacks.py</code>	Conexión entre interfaz y lógica de análisis.	Registro de callbacks, carga de datos, ejecución de modelos y actualización de resultados.
<code>preprocesamiento.py</code>	Preparación del dataset.	<code>load_dataset</code> y <code>preprocess_data</code> .
<code>exponential.py</code>	Modelo Exponencial.	<code>build_exponential_analysis</code> .
<code>weibull.py</code>	Modelo de Weibull.	<code>build_weibull_analysis</code> .
<code>rsf.py</code>	Random Survival Forest.	<code>build_rsf_analysis</code> , <code>_fit_rsf_model</code> y <code>build_rsf_profile_analysis</code> .
<code>survival_plots.py</code>	Gráficas complementarias.	Curvas comparativas, forest plots y visualizaciones de apoyo.
<code>ollama_AI.py</code>	Interpretación mediante IA local.	Construcción de prompts y llamadas al servidor <code>llama-server</code> .
<code>pdf_callbacks.py</code>	Gestión de exportación desde Dash.	Apertura de modales, validación de opciones y descarga de informes.
<code>pdf_exporter.py</code>	Construcción de informes PDF.	<code>SurvivalAnalysisPDFExporter</code> y funciones de exportación.
<code>translations.py</code>	Internacionalización.	Diccionario de traducciones y función <code>get_translation</code> .
<code>config.py</code>	Configuración general.	Rutas, servidor de IA, nombre del modelo y carga diferida de datasets.

Tabla 39: Resumen de archivos principales del código fuente

B.6. Flujo interno de ejecución

El funcionamiento interno de la aplicación sigue un flujo bastante claro. Primero se inicia la aplicación Dash desde el archivo principal. Después, el usuario carga y preprocesa el dataset desde la interfaz. A partir de ese momento, los callbacks recuperan los datos almacenados, llaman al módulo de análisis correspondiente y devuelven a la pantalla tablas, gráficas o mensajes. Si el usuario lo solicita, esos mismos resultados pueden enviarse al módulo de IA o al generador de informes PDF.

1. `cargaDataset.py` inicializa la aplicación Dash y registra los callbacks principales.
2. El usuario carga un archivo CSV desde la interfaz definida en `layout.py`.
3. El callback de carga recibe el archivo, lo valida y lo almacena temporalmente.
4. `preprocesamiento.py` genera el dataset limpio que utilizarán los modelos.
5. El usuario selecciona una técnica de análisis desde la interfaz.
6. `analysis_callbacks.py` recupera el dataset y llama al módulo correspondiente.
7. El módulo estadístico devuelve una estructura con tabla, figura, métricas o interpretación.
8. Dash actualiza la página con los resultados generados.
9. Si procede, `ollama_AI.py` genera una explicación textual mediante el servidor local de IA.
10. Si el usuario exporta, `pdf_callbacks.py` y `pdf_exporter.py` construyen el informe PDF.

Este flujo explica por qué es importante mantener separadas las responsabilidades. Los callbacks actúan como puente, pero no deberían concentrar toda la lógica estadística. Del mismo modo, los modelos no deberían encargarse directamente de construir la interfaz ni de generar el informe completo. Cada parte tiene su papel, y esa separación facilita que el proyecto pueda crecer sin volverse inmanejable.

B.7. Criterios para ampliar el código

Si en el futuro se quisiera añadir una nueva técnica de análisis, lo más recomendable sería seguir el patrón ya utilizado en los modelos incorporados. Primero se debería crear un archivo propio para la técnica, con una función principal que reciba el dataset limpio y devuelva resultados reutilizables. Después habría que añadir la vista correspondiente en la interfaz, registrar los callbacks necesarios, incluir traducciones y, si procede, preparar la exportación PDF.

Paso	Acción recomendada
Crear módulo de análisis	Añadir un archivo propio, por ejemplo <code>nuevo_modelo.py</code> , con funciones centradas en la técnica.
Validar entradas	Comprobar que existen columnas necesarias, datos suficientes y tipos compatibles antes de ajustar el modelo.
Devolver resultados reutilizables	Preparar una salida con figura, tabla resumen, métricas e interpretación si corresponde.
Actualizar interfaz	Añadir la nueva pantalla o sección en <code>layout.py</code> .
Registrar callbacks	Conectar los botones y controles con la función de análisis desde <code>analysis_callbacks.py</code> .
Añadir traducciones	Incluir las nuevas etiquetas y mensajes en <code>translations.py</code> , tanto en castellano como en inglés.
Actualizar exportación	Si el modelo genera informes, ampliar <code>pdf_callbacks.py</code> y <code>pdf_exporter.py</code> .
Probar casos válidos y erróneos	Verificar que la técnica funciona con datos correctos y que responde bien cuando faltan datos o parámetros.

Tabla 40: Pasos recomendados para incorporar una nueva funcionalidad

La clave está en no mezclar responsabilidades. Si una nueva funcionalidad empieza a crecer demasiado dentro de un callback, probablemente convenga mover parte de esa lógica a un módulo independiente. Es una pequeña decisión de diseño, pero evita que el código se vuelva difícil de seguir con el tiempo.

B.8. Puntos delicados para el mantenimiento

Algunas partes del proyecto requieren especial atención cuando se modifica el código. No son errores necesariamente, pero sí zonas donde un pequeño cambio puede tener efectos en cadena. Por eso conviene revisarlas con más cuidado.

- **Dataset bruto y dataset limpio:** no deben confundirse. Los modelos deben trabajar con los datos preprocesados.

ANEXO B. ANEXO DEL MANUAL DE CÓDIGO FUENTE

- **Identificadores de Dash:** si se cambia el `id` de un componente en `layout.py`, también deben actualizarse los callbacks que lo usan.
- **Traducciones:** cualquier texto nuevo de interfaz debería añadirse en castellano e inglés dentro de `translations.py`.
- **Servidor de IA:** las funciones de IA dependen de que `llama-server` esté activo y escuchando en la URL configurada.
- **Exportación de gráficas:** las figuras de Plotly deben poder convertirse correctamente para insertarse en PDF mediante Kaleido.
- **Modelos estadísticos:** antes de ajustar un modelo conviene validar que existen datos suficientes y que las columnas necesarias no contienen valores incompatibles.
- **Random Survival Forest:** requiere una matriz de características consistente entre entrenamiento, visualización y simulación de perfiles.

Tener presentes estos puntos ayuda a evitar errores difíciles de detectar. Muchas veces la aplicación no falla por una gran modificación, sino por un detalle pequeño: un identificador mal escrito, una traducción inexistente, una columna que cambia de nombre o un servidor local que no está iniciado. Por eso, antes de tocar una funcionalidad, conviene localizar qué módulos participan en ella y revisar el flujo completo.